

AI for Everyone

From rules to deep learning

Onur Önder

AI for Everyone

Onur Önder

© 2026 Onur Önder. All rights reserved. Marketing and sales rights belong to Fittechs Yazılım Anonim Şirketi. No part of this book may be reproduced, published or transmitted in any form without the written permission of the author.

Published by the author.

Print edition ISBN: [ISBN]

Interactive digital edition ISBN: 978-625-00-5299-0

First edition: September 2026

Cover and interior design: Onur Önder

Copy editing: Onur Önder

Printed on demand by Amazon KDP.

Sales and contact: Fittechs Yazılım Anonim Şirketi, Gayrettepe Mah. Yıldız Posta Cad. No: 8/34, İstanbul, Türkiye. Email: support@fittechs.com. Web: onuronder.com

The interactive digital edition of this book, with all 45 live demos, is at book.onuronder.com.

Acknowledgments

This book was an idea that appeared out of nowhere. I asked myself why: shouldn't everyone have some idea of this, understand a little of what they are using, see what kind of era we are entering? Then I told myself that to understand artificial intelligence you first have to understand the computer and how it works; that is how the story began.

The ideas this book rests on are not mine. They belong to Turing and Hinton, to the people who wrote rules and the people who computed probabilities, to the researchers who wrote the different chapters of this story, and to everyone who shared openly what they had learned. I have listed as many of them as I could in the bibliography, leaving no one out that I could name; I owe my thanks to all of them.

Finally, to the ones who stood by me in hard times, whatever came: to my wife Öykü, my greatest supporter and the source of my energy to keep going, with her mind, her compassion and her love; to my children Kuzey and Poyraz; to my mother, who never withheld her tenderness and warmth and who thinks of the smallest details; and to my father, who rests in light and whose pride and love I have always felt...

To the eyes that read the text line by line, corrected it and were never shy to say "this part is not clear": Seda and Gözde.

Thank you all; every error that remains is mine.

Onur Önder

Preface

This book begins with a question: Can a machine think? The question is not new. Long before computers, four hundred years ago, it kept philosophers busy. What is new is that the machines that can try to answer it are now sitting on the table.

I am going to tell a two-century story, from the first calculating gears to today's chatbots. Its heroes are ideas: rules, search, probability, learning, neurons, attention. Each one was born where the one before it hit a wall. So the chapters follow the order of history: each one shows where the last idea ran out and where the next began.

In the digital edition of this book every idea is tried with your own hands: you run a Turing machine step by step, turn the inputs of a neuron, pull a picture out of noise. Paper cannot do that. So I have worked every experiment out on the page, with numbers and drawings, and then handed it over to you. The “Try it yourself” exercises are solved with pencil and paper, and the link to the live demo sits under every figure.

I wrote the book at two depths. The main text is for someone with no background at all. The “Technical depth” boxes are for anyone who wants the formulas and the terminology behind the idea. You can read the book while skipping the boxes; nothing will be missing. If curiosity strikes later, go back to them.

I have tried to explain what everyone who talks about artificial intelligence needs to know, and to leave no reader behind. I hope that when you finish, you will have given your own answer to the question we started with.

Onur Önder

Istanbul, September 2026

How to Read This Book

The book has eight chapters. Each chapter covers one era and the main idea of that era, and the chapters were written to be read in order. Still, every chapter stands on its own; you can also start wherever your curiosity takes you.

Two depths. The main text is written in plain language and is complete on its own. At the end of some sections there are boxes headed **Technical depth**. They retell the same idea with its formula, its terms and its math. The boxes can be skipped; the main text does not depend on them. The grid and the web on the cover tell the same story: one idea, two depths.

Margin notes. The short notes headed “Margin note” do not sit in the page margin; they stand in the flow of the text as small boxes, set in italics with a colored rule down the left edge. They give the link between a topic and today, or a detail that is easy to miss at first glance.

Figures. Each of the 45 live demos in the digital edition has become a figure block here. Every block follows the same order:

- *Setup*: what you see in the figure.
- *Step by step* (or *Self-test* for the figures that quiz you): the experiment, carried out on paper with real numbers.
- *What is happening?*: a one-paragraph explanation.
- *Try it yourself*: one to three questions to solve with pencil and paper. Answers are at the back of the book.

Figures that show a process work like a film strip: follow the frames from left to right, row by row.

Live demos. The QR code under each figure opens the live version of that experiment on your phone. No sign-in or purchase is needed; only that demo opens. The whole book, with all of its demos, is in the digital edition. A list of all demos and their links is at the back of the book.

Test yourself. Every chapter ends with a short quiz. The answer key is at the back of the book. Some figures are tests as well (“Self-test”); their answers and the reasoning are in the same place.

What to keep from this chapter. The bullet list at the end of each chapter sums up what should stay with you from that chapter. Once you have finished the book, reading only these lists is a good way to review.

Contents

Chapter 1 · Minds and Machines	13
1.1 Can a machine think?	14
1.2 What is intelligence?	15
1.3 Is thinking computation?	19
1.4 From Babbage to Turing	22
1.5 ENIAC and the birth of the modern computer	26
1.6 Narrow AI or General AI?	30
1.7 Why now? Moore's law	33
1.8 Test yourself	37
 Chapter 2 · The Age of Rules	 41
2.1 The age of rules	42
2.2 Representing knowledge with symbols	43
2.3 If... then...: rules and expert systems	47
2.4 Search and heuristic shortcuts	50
2.5 Handling uncertainty: probability and Markov	54
2.6 Neats and scruffies	58
2.7 Test yourself	61
 Chapter 3 · How Machines Learn	 63
3.1 How do machines learn?	64
3.2 Features and labels	65
3.3 Three kinds of learning	68
3.4 Classification and regression	71
3.5 Clustering and anomalies	75
3.6 How a model improves: loss and gradient descent	78
3.7 Overfitting and ensembles	82
3.8 Test yourself	85
 Chapter 4 · The Artificial Brain	 89
4.1 The artificial brain: deep learning	90
4.2 A single artificial neuron	91
4.3 Layers and the forward pass	94
4.4 Backpropagation: learning from error	97

4.5 Seeing images: convolutional networks (CNN)	102
4.6 Understanding sequences: recurrent networks (RNN)	105
4.7 The art of forgery: GANs	110
4.8 Test yourself	114
Chapter 5 · Today's AI	117
5.1 The generative era: from recognizing to creating	118
5.2 How a machine sees words: tokens	119
5.3 Turning meaning into numbers: embeddings	123
5.4 Attention: the heart of the Transformer	127
5.5 A language model: predict the next word	131
5.6 How a model is raised: the training pipeline	136
5.7 Making images: diffusion	140
5.8 Limits: hallucination, context and cost	144
5.9 Test yourself	149
Chapter 6 · Using and Building AI	153
6.1 Using and building AI	154
6.2 Prompt engineering: asking the right question	155
6.3 RAG: give the model your own data	159
6.4 Agents: think, use tools, observe	164
6.5 The architecture of an AI application	167
6.6 AI in the real world	172
6.7 Test yourself	176
Chapter 7 · AI and Society	179
7.1 AI and society	180
7.2 Bias: from data to decisions	181
7.3 Black box or white box?	185
7.4 Deepfakes and disinformation	189
7.5 Regulation: classifying risk	192
7.6 Alignment: what you said, or what you meant?	197
7.7 Test yourself	201
Chapter 8 · Philosophy and the Future	203
8.1 Philosophy and the future	204
8.2 The Turing test: can you tell?	205
8.3 The Chinese Room: understanding or processing?	208

8.4 From narrow AI to superintelligence	212
8.5 The singularity: myth or serious possibility?	215
8.6 Responsibility and rights	219
8.7 Test yourself	222
Answer Key	225
Glossary	251
Bibliography and Further Reading	263
Live Demos	267
Index	269

1

CHAPTER 1

Minds and Machines

Can a machine think?

1.1 Can a machine think?

Once upon a time... except this story isn't over, and you are in it. It began long before computers, with a question: What is thinking? If some kind of gear turns inside our heads, couldn't a machine turn the same gear? People have argued about this for centuries.

This first chapter goes back to the very beginning. What is intelligence? Can “thinking” be written down as a step-by-step recipe? The path runs from Babbage's gears to the birth of the modern computer, and by the end it will be clear why artificial intelligence (AI for short) is possible at all.

Margin note. AI isn't new: “can machines think?” has occupied philosophers since the 1600s. What's new is the hardware that can test it.

TECHNICAL DEPTH

AI was born at the intersection of computability theory and the philosophy of mind. The core claim: if mental processes can be described by formal rules, then in principle a machine can carry them out too.

This chapter starts from defining intelligence, lays the foundation stones of computation (binary representation, algorithms, the Turing machine, the stored-program architecture) and connects to the modern debate via the weak/strong AI distinction. That gives the later learning and neural-network ideas solid ground to stand on.

Start with the first question. What is this “intelligence” you want to hand to a machine?

1.2 What is intelligence?

It looks like an easy question, but “what is intelligence?” has no single answer. Is the puzzle-solver smart, or the person who instantly finds their way in a new city, or the friend who reads your sadness in one glance? Perhaps all of them; yet each is a very different skill.

The psychologist Howard Gardner made the same point: intelligence doesn’t fit in one number (IQ); it comes in several kinds. He proposed seven in 1983 and later added naturalistic intelligence. The theory is debated, but it is a useful way to see that intelligence is not one thing. Explore the kinds below. The interesting part is that today’s AI is a master of some and still a toddler at others.

Margin note. A calculator is far faster than you at particular calculations but can’t laugh at a joke. Being “smart” isn’t a single axis.

Setup. The figure shows eight cards, one for each kind of intelligence in Gardner’s theory. Seven come from his 1983 list; the naturalist kind was added later. The bar under each card shows roughly where today’s AI stands in that area. The bars are illustrative levels chosen for the explanation, not measurements: a full bar means “strong,” a half bar “medium,” a short stub “weak.”

Step by step. All eight cards, in the order of the figure:

Intelligence type	What it means	AI today	Bar (illustrative)
Linguistic	Using language, writing and narrative.	Strong	90%

Intelligence type	What it means	AI today	Bar (illustrative)
Logical-Mathematical	Numbers, logic and systematic reasoning.	Strong	90%
Spatial	Grasping space, shapes and visuals.	Medium	55%
Musical	Sensing rhythm, melody and sound patterns.	Medium	55%
Bodily-Kinesthetic	Skillful use of the body and hands.	Weak	22%
Interpersonal	Understanding others, empathy and communication.	Weak	22%
Intrapersonal	Knowing one's own emotions and goals.	Weak	22%
Naturalist	Recognizing living things and patterns in nature.	Medium	55%

The percentages are not measured performance; they are the bar lengths of the three levels in the figure. Tally the last column and you get two “strong,” three “medium,” three “weak.” What do the two strong ones have in common? Both work with symbols: letters, numbers, rules. A computer runs on that same material; the binary code in the next section is a string of symbols too.

The three weak ones ask for a different material: a body, the face of the person across from you, your own inner world. None of these turns into symbols easily. The middle three, space, music and nature, sit between the two. Part of sound and image can be turned into numbers; part of it still resists.

The order of the cards is not a ranking, either. They only count separate abilities; none of them is “more intelligence” than another. The

same holds for AI: being strong in language does not mean being strong in the other seven areas.

What is happening? For Gardner, intelligence isn't one number but eight distinct families of ability; each card in the figure is one of them. And today's AI is great with language and logic; at anything involving the body or emotions the result depends on the task and the system, and in most of them it is still far behind.

Figure 1.1 · Explore multiple intelligences

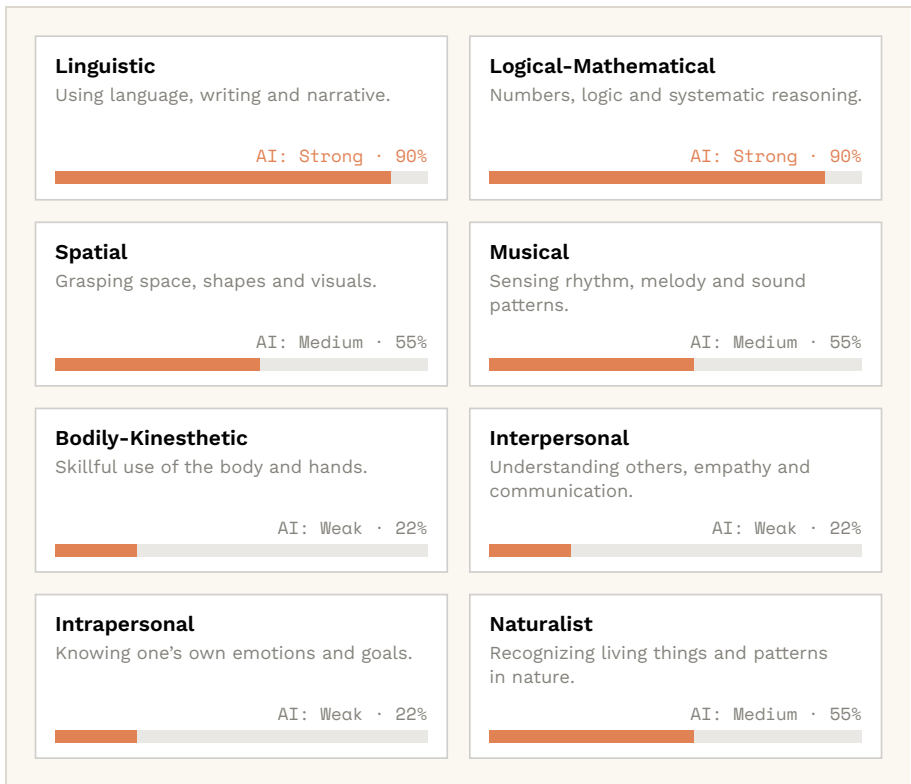


Figure 1.1 Live demo: book.onuronder.com/d/en/40ee8a0d04



Try it yourself. 1) Close the book and list the eight kinds of intelligence from memory. How many did you recall, and which ones slipped your mind? 2) Think about yesterday: from morning to night, which three kinds did you use most? According to the table, in how many of those is today's AI strong? 3) What do the three "weak" kinds in the table have in common?

TECHNICAL DEPTH

There is no agreed single definition of intelligence; working definitions cluster around "goal-directed adaptation, abstraction, learning and problem-solving capacity." The AI field uses a pragmatic one: systems that perform tasks requiring human intelligence.

Gardner's Theory of Multiple Intelligences (Frames of Mind, 1983) splits intelligence into relatively independent domains. The original 1983 list had seven: linguistic, logical-mathematical, spatial, musical, bodily-kinesthetic, interpersonal and intrapersonal. Gardner added the naturalistic domain later; the eight-item list used here is not identical to the original 1983 list. The theory is contested in psychometrics, but it usefully shows that intelligence is multi-dimensional: today's AI is wildly uneven across these dimensions.

Each card is one of the eight domains in Gardner's theory: AI is strong in language and logic, weak in the bodily and social/intrapersonal domains. The three levels in the figure are illustrative labels, not measured performance; the result changes with the task and the system.

If intelligence comes in pieces, can at least one of those pieces be written down as a step-by-step recipe? A kitchen is a good place to find out.

1.3 Is thinking computation?

Step into the kitchen for a moment. You hold a cake recipe: crack the eggs, whisk, add the flour, into the oven. Someone following that recipe to the letter will produce a cake without “understanding” baking at all. The idea beneath all of AI is the same: maybe “thinking,” too, is applying small, mechanical steps in order.

If so, a machine can do those steps too. Today’s digital computers use two things for it:

- A simple alphabet the machine can use (usually binary: 0 and 1)
- A clear list of steps (the recipe itself: an algorithm)

See how binary works for yourself below.

Margin note. The word “algorithm” comes from the name of the 9th-century Persian mathematician al-Khwarizmi. Binary itself was formalized by Leibniz in 1703.

Setup. The figure shows eight boxes in a row. Above each box is its place value; from left to right, 128, 64, 32, 16, 8, 4, 2 and 1. A lit box means 1, a dark box means 0. The top row shows the opening state, 01001001. The number underneath is the sum of the place values of the lit boxes. Under it, three worked rows show 5, 73 and 255 written the same way.

Step by step. First, the place-value chart:

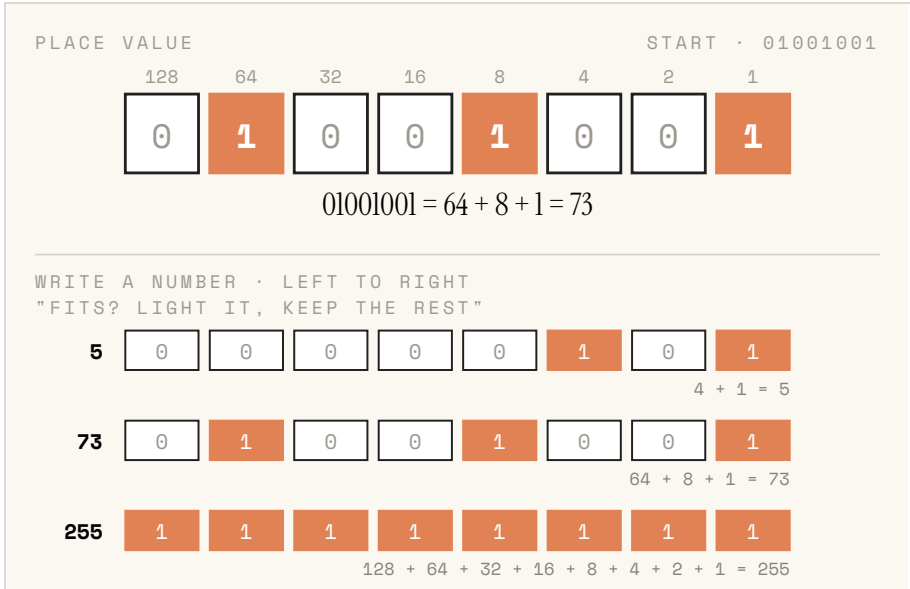
Box	1	2	3	4	5	6	7	8
Place value	128	64	32	16	8	4	2	1
Start	0	1	0	0	1	0	0	1

1. Decode the opening number. The lit boxes are 2, 5 and 8; their values are 64, 8 and 1. Add them: $64 + 8 + 1 = 73$. So 01001001 = 73.
2. Now go the other way and write the number 5 into the boxes. Start from the left and ask each box: “Does your value fit into the number?” 128, 64, 32, 16 and 8 do not fit; they all stay dark. 4 fits: light it, and 1 is left. 2 does not fit. 1 fits: light it, and 0 is left. The result is 00000101.
3. Write 73 the same way. 128 does not fit. 64 fits, 9 left. 32 and 16 do not fit. 8 fits, 1 left. 4 and 2 do not fit. 1 fits, 0 left. The result is 01001001, the same pattern as the opening state.
4. Finally, 255: light every box. $128 + 64 + 32 + 16 + 8 + 4 + 2 + 1 = 255$, that is, 11111111. This is the largest number eight boxes can say. One more, 256, needs a ninth box.

Did you spot the rule? Going from left to right, “if it fits, light it and carry on with the remainder” is all it takes. Every number has one pattern and one only; two different patterns never give the same number. Eight boxes hold 256 different numbers, from 0 to 255; computer people call that a byte.

What is happening? Eight boxes, eight bulbs: off means 0, on means 1. But who handed out the values? Start from the right: the rightmost box holds the smallest value, 1. Moving left, each box carries double its right-hand neighbor: 1, 2, 4, 8, 16, 32, 64, and 128 on the far left. Why always double? Because each new box must be able to say one more than all the boxes to its right combined; otherwise some numbers could never be written. Now add up the values of the lit boxes: there is your number. A computer’s whole world is this game of on and off.

Try it yourself. 1) Write the number 10 with eight boxes. 2) Which number is the pattern 10110000? 3) If only the leftmost box is lit, what is the number? If you added a ninth box, what would the largest number be?

Figure 1.2 · Crack the binary codeFigure 1.2 Live demo: book.onuronder.com/d/en/4642d3916a**TECHNICAL DEPTH**

This view is called computationalism: the mind is an information-processing system, and cognition is computation in the form of symbol manipulation. Its roots reach back to Hobbes (“reasoning is reckoning”); its modern form developed with Putnam and Fodor.

In practice, two building blocks are used. Binary representation: modern digital computers usually encode information in base-2 (bits); binary is not a precondition of computation, since decimal and analog machines compute too (ENIAC worked in decimal). Boolean algebra (Boole, 1854) and Shannon’s 1937 thesis showed

TECHNICAL DEPTH · CONTINUED

that logic can be realized with electrical switches. Algorithm: a finite sequence of well-defined steps (the term comes from the 9th-century mathematician al-Khwarizmi). Together they make algorithms mechanically executable; whether all thinking can be reduced to computation is a separate question.

Binary 01001001 = the sum of its place values = $64 + 8 + 1 = 73$. Each bit represents 128, 64, 32, ... from left to right. 8 bits (1 byte) encode any number from 0 to 255.

General rule: value = $\sum b_i \cdot 2^i$, with $i = 0 \dots 7$ counted from right to left; b_i is the 0 or 1 of that bit. n bits encode 2^n distinct numbers; for 8 bits, $2^8 = 256$.

The alphabet and the recipe are ready. Someone still has to carry the recipe out, and the first candidates, back in the 1800s, were made of gears.

1.4 From Babbage to Turing

In the 1800s, Charles Babbage envisioned a programmable calculating machine. He drew it as a giant of gears and levers: the Analytical Engine. Money and years ran out; he died without ever seeing it built. But his friend Ada Lovelace wrote the first “recipe” for that imaginary machine. So she counts as history’s first programmer.

About a century later, Alan Turing carried the dream a step further. Picture a single little box that reads a tape, writes on it and slides left or right. Given the right rules and enough tape, it can perform any calculation that can be written as an algorithm. It is called a Turing machine. Follow a small one that only adds 1 to a number as it “thinks,” frame by frame, in Figure 1.3.

Margin note. In 1948 Turing designed a chess algorithm called “Turochamp.” No machine of the era could run it, so Turing worked out the moves by hand on paper.

Figure 1.3 · A working Turing machine (+1)

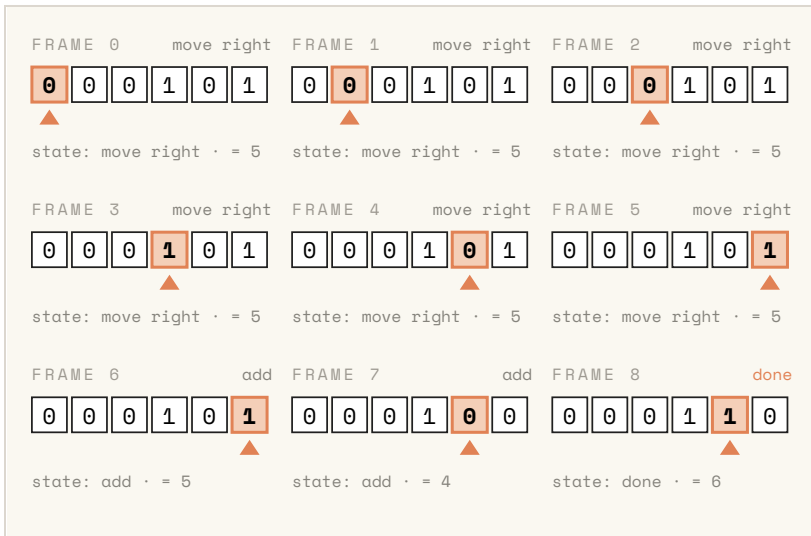


Figure 1.3 Live demo: book.onuronder.com/d/en/6c232ac487



Setup. The figure shows a tape of six cells; the arrow (▲) moving above it is the machine’s head. The tape reads 000101, which is 5 in binary. The machine has three states: “move right,” “add” (add 1 with carry) and “done.” The nine frames of the film strip show every step taken while 1 is added to 5.

Step by step. Read the frames in order; each frame gives the tape, the head position and the state. The cells are numbered 0 to 5 from

left to right.

0. Start. The head is on the leftmost cell (0), the state is “move right.” The tape reads 0 0 0 1 0 1, that is, 5.
1. The head slides one cell to the right, to cell 1. The tape does not change. The head (▲) steps right to find the number’s rightmost digit. As in adding by hand, the work starts from the far right.
2. The head is on cell 2. It is still only reading, not writing.
3. The head is on cell 3; there is a 1 here, but in this state the machine ignores it and moves on.
4. The head is on cell 4.
5. The head is on the last cell (5). The tape still reads 0 0 0 1 0 1.
6. No cell is left on the right. The machine changes state: “add” (add 1 with carry). The head stays put, the tape stays the same.
7. The head sees a 1 on the last cell. It turns the 1 into 0 and carries one to the left; the head moves to cell 4. The tape reads 0 0 0 1 0 0, for a moment 4. The head now works backward, adding 1 leftward: every 1 it sees becomes 0 with a carry, and at the first 0 it writes 1 and stops. The same “carry the one” rule you learned in school.
8. The head sees a 0 on cell 4. It writes a 1 there and stops; the state is “done.” The tape reads 0 0 0 1 1 0, that is, 6. Done. With only a handful of simple rules like “read, write, move left/right,” the machine added 1 to a binary number. That is all this machine can do; another calculation needs another rule table. Computing, for a Turing machine, means nothing more than this.

Eight moves in all: five steps to the right, one state change, one clear, one write. Five plus one made six; the tape says it as 000110.

What is happening? The machine walks the tape like an ant: first it heads right to find the end of the number, then turns back and adds 1. The way it carries the digit is no different from how you carry in addition on paper. This machine only adds 1; other rules give other

calculations. Any calculation that can be written as a recipe comes down, in the end, to moves this small.

Try it yourself. 1) Start the tape at 001111 (15) and draw the frames yourself on paper. How many 1s get cleared, in how many moves does the machine finish, and which number is left on the tape? 2) Start the tape at 010110 (22). When the head reaches the end and turns back, what is the first digit it sees? In how many moves does it finish? 3) What would happen with a tape of 111111 (63)? The machine sees a 1 even on the leftmost cell; it cannot move left, so it stops.

TECHNICAL DEPTH

Babbage's Analytical Engine (1837) was a general-purpose design with a "mill" (processing unit) and a "store" (memory), programmable via punched cards. Ada Lovelace's 1843 notes contain the algorithm for computing Bernoulli numbers, generally regarded as the first computer program.

Alan Turing's 1936 paper "On Computable Numbers" introduced the abstract Turing machine and the universal Turing machine; this is the formal basis of computability. Given enough time and tape, a universal Turing machine can run programs for any algorithmically computable task; the same paper shows that uncomputable problems exist as well. The simulation in Figure 1.3 is not universal but a particular machine: it adds 1 to a binary number, doing arithmetic with only read/write and state transitions.

The machine slides right to the end of the tape, then turns left and adds 1 to the binary number (carry logic): the 1s it sees become 0, and the first 0 becomes 1.

TECHNICAL DEPTH · CONTINUED

The transition table of the machine in Figure 1.3, taken from the source code:

State	Read	Write	Move	New state
move right	0 or 1	same	right	move right; at the end of the tape, “add 1”
add 1	1	0	left	add 1; at the start of the tape, “done”
add 1	0	1	stay	done

Three rows, a six-cell tape and a single head: enough to add 1 to any number from 0 to 62. For 63 the tape overflows and 0 remains.

Turing’s machine was a thought on paper. Pouring it into real tubes and wires is another story.

1.5 ENIAC and the birth of the modern computer

In 1945 a giant machine came to life: ENIAC; it was unveiled to the public in 1946. It weighed tons, ran on tens of thousands of tubes, and was one of the first general-purpose electronic digital computers. It had a grumpy streak: teaching it a new job meant days of unplugging and replugging cables by hand.

The fix was an elegant idea: keep the recipe next to the ingredients, in memory. Then, instead of rewiring the machine, you load new “instructions.” Nearly every computer today, including the phone in your pocket, runs on this arrangement. Follow the cycle in the figure below, then look at what its parts do.

Margin note. Reprogramming ENIAC took days; the cables had to be reconnected by hand. Putting the program in memory changed all of that.

Setup. The figure shows three boxes, Memory, Processor and Input / Output, in three frames side by side: “1 · Fetch,” “2 · Execute,” “3 · Store / Write.” In each frame the part at work in that phase is dark. After the third frame the cycle starts over. The figure is an illustrative cycle; in a real processor an instruction is not split into three phases of equal length, and not every instruction reaches the outside world.

Step by step. The parts first, as the figure describes them:

Phase	Active part	What it does
1 · Fetch	Memory	Stores both the program (instructions) and the data together. The real revolution is here: instructions are kept just like data.
2 · Execute	Processor	The ALU computes (adding, comparing); the control unit fetches instructions from memory in order and executes them.
3 · Store / Write	Register or memory; Input / Output on an output instruction	Save the result: it is written to a register or to memory. Only on an output instruction does the result go to Input / Output, the link to the outside world: keyboard, screen, sensors. Data comes in here, results go out here.

Now turn the cycle with a small program. Let memory hold two lines: the first line says “add 5 and 3,” the second says “write the result to the screen.” The numbers 5 and 3 sit in memory too.

1. Fetch. The control unit reaches into memory and takes the instruction on the first line: “add 5 and 3.” The instruction came

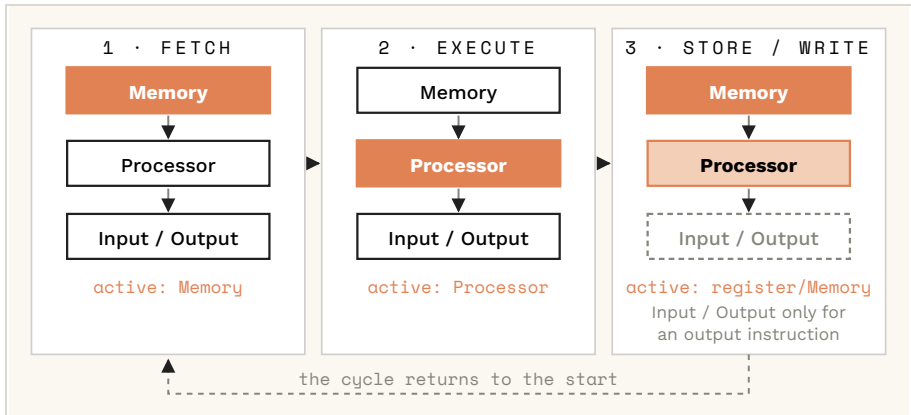
from memory the same way 5 and 3 did; it is data, not a cable.

2. Execute. The ALU does the addition: $5 + 3 = 8$. For now the result stays inside the processor.
3. Write. The 8 is put back into memory; in this round the Input / Output part has nothing to do, and nothing appears on the screen.
4. Fetch. The cycle has come back to the start; the control unit takes the second line: “write the result to the screen.”
5. Execute. The processor reads the 8 from memory and prepares it for output.
6. Write. An 8 appears on the screen. The two-line program is finished; if there is a next instruction, the cycle goes on.

Two rounds, six phases. On ENIAC, changing the first line meant days of pulling cables. Here the only thing to do is write a different instruction into the first line of memory, say “multiply 5 by 3.” The cycle stays the same; only the instruction it picks up changes. A stored program means no more than that: you change one line in memory, not the wiring.

What is happening? A computer performs a three-step dance non-stop: grab the next instruction from memory (fetch), do what it says (execute), save the result to a register or to memory, or display it if the instruction is an output (write). This is the loop beating at the heart of your phone and your computer alike.

Try it yourself. 1) You typed 7×6 into the calculator on your phone. Describe the three phases in your own words; which part is at work in each one? 2) Add a third line to the program above: “multiply the result by 2.” How many more phases are needed, and which number is written to memory? 3) Picture a processor that runs 3 billion cycles per second. Each cycle has three phases. How many phases in one second?

Figure 1.4 · The Fetch – Execute – Write cycleFigure 1.4 Live demo: book.onuronder.com/d/en/c4fdfa18f9**TECHNICAL DEPTH**

ENIAC (Mauchly & Eckert) was one of the first general-purpose electronic digital computers; it became operational in 1945 and was publicly unveiled in 1946. It used ~17,500 vacuum tubes and was programmed by rewiring plugboards (its first programmers were six women mathematicians). Internally it worked in decimal, not binary.

John von Neumann's 1945 EDVAC report popularized the stored-program principle: program and data live in the same memory. The architecture consists of a processor (ALU + control unit), memory and input/output, talking over a bus. The processor runs a continual “fetch–decode–execute” cycle. The “von Neumann bottleneck” is this design's well-known limit.

In the “fetch” phase, the control unit reads the next instruction from memory and decodes it; the program sits in memory just

TECHNICAL DEPTH · CONTINUED

like data (the stored-program principle). In “execute” the processor does the work; in “write” the result is saved to a register or to memory. Sending it to the screen is a separate input/output instruction; not every instruction uses an input or output device.

The three frames of Figure 1.4 are an illustrative simplification of the fetch–decode–execute cycle: decoding is folded into “Fetch,” and saving the result is shown as a separate frame at the end of execution.

The machine can now run any recipe. So what does “artificial intelligence” mean today, and how much intelligence is in it?

1.6 Narrow AI or General AI?

The all-understanding AI of the movies doesn’t exist yet. Today’s AI systems count as “narrow”: masters of the jobs they were trained for, beginners one step outside them. A chess engine can beat you, yet you can’t ask it for a soup recipe.

The hypothetical system with broad, transferable abilities that can learn across every domain like a human is called “artificial general intelligence” (AGI); nobody has built one yet. Doing many tasks does not by itself make a system general; consciousness is a separate question altogether. Sort the examples below: Which exists today, and which is still science fiction?

Margin note. A system with broad abilities that transfer across domains (general) and a system truly “understanding / being conscious” (strong) are different questions. Keeping them apart is key to the modern AI debate.

Setup. The figure shows three columns: “In use today,” “Hypothetical” and “Consciousness question.” Five cards sit in the middle, and the boxes next to them are empty; you fill them in with a pencil. The third column is for a separate question: consciousness is not the same thing as general ability (AGI).

Self-test. Put each card in a column. The first two columns have one test: is the system in use today, or is it a hypothesis nobody has built yet? Judge each card by whether it exists, not by how impressive it sounds. A card that asks about consciousness rather than ability goes in the third column. The fourth card is about general ability (AGI). The fifth is about consciousness, a separate question that is not part of the definition of AGI. Answers and reasons are at the back of the book.

#	Example	In use today	Hypothetical	Consciousness question
1	Chess engine	☐	☐	☐
2	Face recognition system	☐	☐	☐
3	Chatbot (language model)	☐	☐	☐
4	A machine that learns any profession like a human and has its own goals	☐	☐	☐
5	A self-aware, conscious AI	☐	☐	☐

After you have marked all five, count: how many cards are in each column? The fourth and fifth cards both describe something that does not exist today, but they do not ask the same question. Name the difference in one sentence and you have the main idea of the chapter.

What is happening? The rule is simple: any system that cannot step outside the jobs it was trained for is “narrow,” chatbots that write poems and code included. “General” means a machine with

transferable abilities that can learn across every domain like a human, and no such machine exists yet. A conscious machine is a separate question; being general does not require consciousness.

Try it yourself. 1) Write down three AI examples from your own daily life: navigation, translation, movie recommendations. Give each one a column. 2) Neither the fourth nor the fifth card exists today; are they the same thing? Apply the two questions in the margin note (“general” and “strong”) to each of them separately. Could a machine be general and yet not conscious? 3) What happens if you ask a chess engine for a soup recipe? Answer with a definition of narrow AI.

Figure 1.5 • Real today, or science fiction?

#	Example	In use today	Hypothetical	Consciousness question
1	Chess engine	☐	☐	☐
2	Face recognition system	☐	☐	☐
3	Chatbot (language model)	☐	☐	☐
4	A machine that learns any profession like a human and has its own goals	☐	☐	☐
5	A self-aware, conscious AI	☐	☐	☐

Figure 1.5 Live demo: book.onuronder.com/d/en/e69cfc21ea



TECHNICAL DEPTH

Two separate distinctions are in play here. Narrow versus general AI is a distinction of capability: narrow AI works on specific tasks, general AI means broad, transferable capabilities across domains; today’s systems, multi-task ones included, are narrow.

TECHNICAL DEPTH · CONTINUED

Weak versus strong AI is a philosophical distinction; the terms were coined by philosopher John Searle (1980, the Chinese Room argument). For Searle, weak AI is a program that models the mind without genuine understanding or consciousness; that is not the same thing as the narrow capability class.

An important distinction: Searle's "strong AI" is about whether the machine truly understands (has a mind); in everyday usage it is often conflated with "artificial general intelligence" (AGI). AGI is a hypothetical capability level that generalizes across domains; "strong AI" is a philosophical claim. They are not the same thing.

Careful: "general" (AGI, cross-domain generalization) and "strong AI" (Searle: does the machine truly understand / is it conscious?) are different questions. Even AGI carries no consciousness requirement: being general does not mean being conscious.

The ideas had been on the table for a hundred years. So why did even narrow AI arrive only in the last few years?

1.7 Why now? Moore's law

The ideas were ready by the mid-1900s; the machines were feeble. Do you know the old rice fable? One grain on the first square of a chessboard, double on every square after... Halfway across the board, the 32 squares hold $2^{32} - 1$ grains; at 25 milligrams a grain, about 107 tons. Toward the end of the board the amount becomes enormous. In 1965 Gordon Moore noticed the transistor count on chips doubling at a steady beat, just like that, about every year at

the time; in 1975 he revised the pace for the years ahead to roughly every two years.

Doubling looks innocent, but repeated it explodes. Do it yourself in the table of Figure 1.6: double the number every two years and see how it rockets. This compounding computation is what carries modern AI.

Margin note. If a sheet of paper 0.1 mm thick could be folded 42 times, it would be about 440,000 km thick, more than the distance to the Moon. That is exponential growth; chips grew at that pace for decades.

Setup. The figure starts in 1971. That year the first microprocessor, the Intel 4004, went on sale with 2,300 transistors. Each step in the figure means “two years passed, the count doubled.” The table stops after 13 doublings, in 1997; the two small charts on the right carry the numbers on to 2023. On the linear scale the curve hugs the floor and then shoots up; on the log scale it is a straight line.

Step by step. Follow the table row by row. The third column is 2^n , the fourth is $2,300 \times 2^n$. Each row is twice the one before; there is no other rule. This is not the historical Moore curve itself but an illustrative table built to make exponential growth felt.

Doubling (n)	Year	2^n	Transistors
0	1971	1	2,300
1	1973	2	4,600
2	1975	4	9,200
3	1977	8	18,400
4	1979	16	36,800
5	1981	32	73,600

Doubling (n)	Year	2 ⁿ	Transistors
6	1983	64	147,200
7	1985	128	294,400
8	1987	256	588,800
9	1989	512	1,177,600
10	1991	1,024	2,355,200
11	1993	2,048	4,710,400
12	1995	4,096	9,420,800
13	1997	8,192	18,841,600

Figure 1.6 · Feel exponential growth

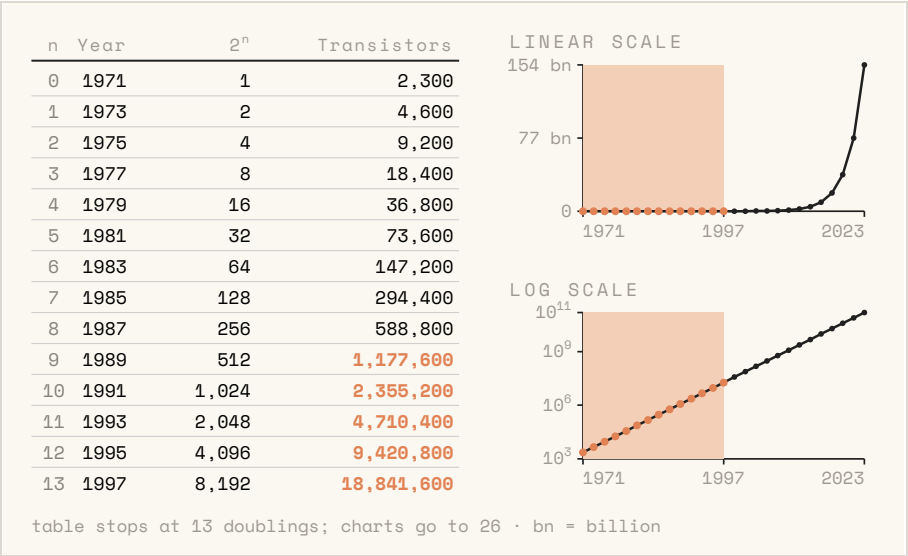


Figure 1.6 Live demo: book.onuronder.com/d/en/eb5ac74b07



The first rows look tame: 2,300, 4,600, 9,200. In ten years the count grows 32-fold but is still in the tens of thousands. In 1989 it crosses the one-million line: 1,177,600, shown as “1.2 million” in the live demo. In 1997 it is 18.8 million. The table stops after 13 doublings, 26 years; the charts go on to 26 doublings.

What if you keep going? After 20 doublings, in 2011, $2^{20} = 1,048,576$ and the count is 2,411,724,800, or 2.4 billion. After 26 doublings, in 2023, $2^{26} = 67,108,864$ and the count is 154,350,387,200: 154.4 billion. Between the first row and the last there is a factor of 67 million. In the rice fable, 107 tons had piled up by the 32nd square; chips reached the 26th square in 2023. A bigger count is not the same as more speed: more transistors do not bring a proportional speedup for every workload.

What is happening? Every row means “two years passed, the transistor count doubled.” Within a few rows the number takes off; that is what exponential growth is like. Transistor counts grew at this pace for decades; speed did not grow at the same rate for every workload, but that accumulation is what made today’s AI possible.

Try it yourself. 1) Extend the table by two rows: 2^n and the transistor count for 1999 and 2001. 2) Starting from 2,300, which row is the first to pass one million; how many years did it take? 3) If the doubling time were 3 years instead of 2, how many transistors would there be in 1995? Compare with the 1995 value in the table.

TECHNICAL DEPTH

Moore’s law is not a law of nature but an empirical observation and economic trend. Moore’s 1965 prediction was that the number of components on an integrated circuit would double about every year; in 1975 he revised it, for the years ahead, to a doubling about every two years. It is not a law of physics that says computing power must grow at the same rate; more transistors do not imply a proportional speedup for every workload. Even so,

TECHNICAL DEPTH · CONTINUED

this exponential growth made the heavy matrix computation deep learning needs economical.

The exponential trend held for ~50 years (1971 Intel 4004: ~2,300 transistors → 2020s: tens of billions). As some transistor structures approach atomic scales, manufacturing and physical limits (heat among them) become important, and the trend has been slowing in recent years; the industry now leans on parallelism and specialized AI hardware such as GPUs/TPUs.

Exponential growth: $\times 2$ roughly every 2 years; n doublings give a 2^n -fold increase.

The formula behind Figure 1.6: $N(t) = N_0 \cdot 2^{(t/2)}$, where t is the number of years since 1971 and $N_0 = 2,300$. For $t = 52$, $N = 2,300 \cdot 2^{26} \approx 1.5 \times 10^{11}$. Real chips sit a little below this curve; the doubling time has stretched since the 2000s.

The building blocks of the chapter are in place: intelligence, binary code, algorithm, Turing machine, stored program, the narrow-versus-general distinction, exponential growth. Six questions follow.

1.8 Test yourself

Answers are at the back of the book.

1. What did Howard Gardner propose?
 - a) That the brain is a computer
 - b) That intelligence has many kinds, not one
 - c) That IQ fully measures intelligence
 - d) That machines can never think

2. Binary code is made of which symbols?
 - a) A to Z
 - b) Dots and dashes
 - c) 0–9
 - d) 0 and 1
3. What can a universal Turing machine fundamentally do?
 - a) Only play chess
 - b) Any computable task, via simple rules, given enough time and tape
 - c) Only addition
 - d) Only store text
4. The core idea of the von Neumann architecture?
 - a) Inventing the internet
 - b) Keeping program and data in the same memory
 - c) Rewiring for each new task
 - d) Using binary instead of decimal
5. What kind are today's AI systems?
 - a) Strong AI
 - b) Narrow AI
 - c) General AI (AGI)
 - d) Conscious AI
6. What does Moore's law say?
 - a) Internet speed is constant
 - b) AI will surpass humans
 - c) Transistor count ~doubles every 2 years
 - d) Computers get cheaper yearly

What to keep from this chapter

- Intelligence is not one number; across Gardner's eight kinds, today's AI is strong in language and logic, weak in body and emotion.

- Today's computers run on two building blocks: binary code (0 and 1) and an algorithm (a clear list of steps). Binary is a common choice, not a condition of computation; whether all thinking is computation is a separate question.
- Eight boxes hold every number from 0 to 255; adding up the place values of the lit boxes is all it takes.
- A Turing machine works with nothing but read, write and move one cell; a universal one, given enough tape and the right program, performs any computable task. The particular machine in the figure only adds 1; adding 1 to 5 took eight moves.
- Stored program: an instruction sits in memory just like data; the processor fetches, decodes and executes each instruction and saves the result to a register or to memory. Sending it to the screen is a separate input/output instruction.
- Today's AI systems are narrow, even the ones that do many tasks; general AI (AGI) does not exist yet; consciousness is a question separate from generality.
- Moore's law: the transistor count doubles every year in the 1965 prediction and roughly every two years in the 1975 revision; a bigger count is not the same as more speed. The table's 2,300 of 1971 reach 154 billion after 26 doublings.

2

CHAPTER 2

The Age of Rules

Before learning: hand-written intelligence

2.1 The age of rules

Today, AI means systems that learn from data. Yet the story's first great act was the exact opposite: the masters of that era tried to make the machine memorize the world. Whatever it needed to act intelligently (all the knowledge, all the rules) they wrote out by hand, one line at a time.

This approach is called “classical” or “symbolic” AI. Logic, rules, search and expert systems were that era's toolbox. This chapter visits it: how did these ideas work, what did they achieve, and why did they one day hit a wall? You will try every bit of it with your own hands.

Margin note. Early AI researchers thought: “If we write enough rules, the machine becomes intelligent.” It worked beautifully for some tasks, and not at all for others.

TECHNICAL DEPTH

Classical AI (symbolic AI, or GOF AI, short for “Good Old-Fashioned AI”) treats intelligence as rule-based manipulation of formal symbols. The core assumption: knowledge about the world can be represented explicitly, and reasoning can run as logical operations over those representations.

This paradigm dominated from the 1950s to the 1980s and produced powerful tools such as logic programming, search algorithms and expert systems. This chapter builds them up, then shows why the “knowledge-acquisition bottleneck” and

TECHNICAL DEPTH · CONTINUED

brittleness increased interest in learning-based approaches (Chapter 3) that were already being studied.

Start with the most basic question of all: how does a machine know that Tom is a cat?

2.2 Representing knowledge with symbols

How would you teach a machine about Tom? It cannot see him or pet him; it knows him only through the sentences you write: “Tom is a cat,” “a cat is a mammal,” “a mammal is an animal.” Classical AI stores knowledge in that form: explicit symbols and the links between them.

The best part is that the machine follows those links and reaches facts nobody ever told it. It never heard the sentence “Tom is an animal”; but walking the chain link by link, it finds that out itself. Figure 2.1 asks four questions; read how the machine “thinks” there, step by step.

Margin note. Symbolic AI’s strength: it can derive many new facts from a few rules. Its weakness: someone has to write those rules first.

Setup. The figure shows five boxes in a single row: Tom, Cat, Mammal, Animal, Living thing. Every arrow between two boxes means “is a”: Tom is a Cat, a Cat is a Mammal, and so on. The first arrow ties an individual to its class: Tom is an instance of the class Cat. The other arrows tie a class to a larger class: Cat is a subclass of Mammal. The figure draws both with the same arrow. These four arrows are the machine’s entire knowledge; it knows nothing else. Under the chain are four queries; for each one the machine walks the chain from the start and states its verdict.

Step by step. Follow the query “Is Tom a Mammal?” first.

1. The machine starts at Tom. The word it is looking for is “Mammal.” Is Tom himself a Mammal? No; but an arrow leaves Tom: Cat.
2. It moves on to Cat. Is Cat “Mammal”? No; an arrow leaves Cat as well: Mammal.
3. It arrives at Mammal. The word it was looking for is found. In the figure the first three boxes are shaded and the verdict reads “Yes: ‘Mammal’ found in the chain (transitivity).”

Figure 2.1 • Inference along a knowledge chain

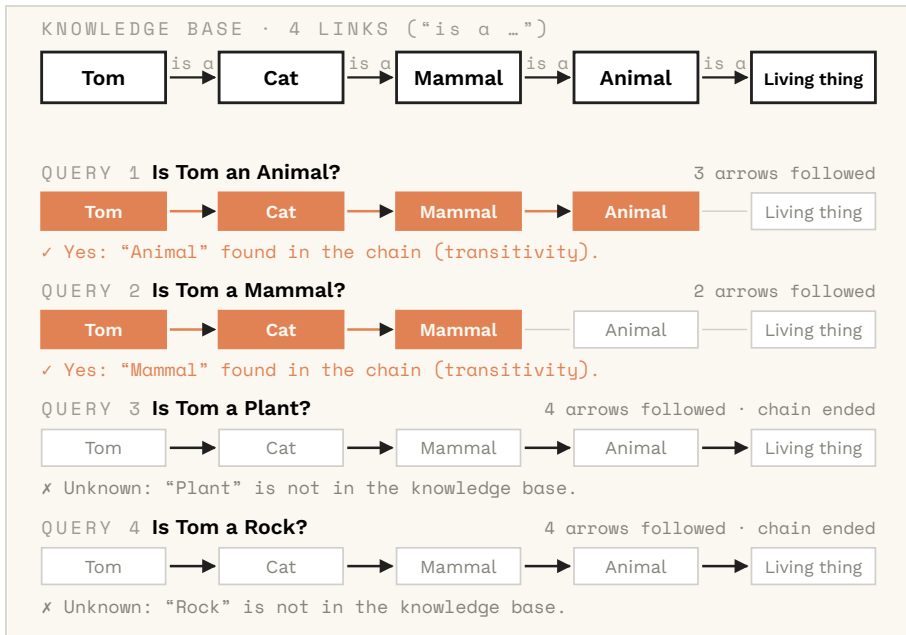


Figure 2.1 Live demo: book.onuronder.com/d/en/48e9aced3e



It followed two arrows. Nobody wrote the sentence “Tom is a Mammal”; the machine derived it.

Now the query “Is Tom a Plant?”

1. Tom: not a Plant; arrow to Cat.
 2. Cat: no; arrow to Mammal.
 3. Mammal: no; arrow to Animal.
 4. Animal: no; arrow to Living thing.
 5. Living thing: no. No arrow leaves Living thing; the chain has ended.
- The verdict reads “Unknown: ‘Plant’ is not in the knowledge base.”

All four queries:

Query	Arrows followed	Verdict
Is Tom an Animal?	3	Yes
Is Tom a Mammal?	2	Yes
Is Tom a Plant?	4, chain ended	Unknown
Is Tom a Rock?	4, chain ended	Unknown

The verdict is “Unknown,” not “No.” The machine does not claim that Tom is not a plant; it only says that its arrows cannot reach that fact. Staying silent about what it does not know is a virtue of symbolic systems.

What is happening? All the machine holds is “what kind of thing is what”: Tom is a cat, a cat is a mammal... When you ask, it follows the chain link by link and reaches a fact nobody ever told it. And if it isn’t in the chain, it admits it: “unknown.”

Try it yourself. 1) Add one more arrow at the end of the chain: a Living thing is an Entity. What does the machine say to “Is Tom an Entity?” and how many arrows does it follow? 2) Turn the question around: “Is a Cat a Tom?” The arrows point one way only; what does the machine answer? Do you think that answer is right? 3) What would you have to add to the knowledge base for the machine to say “Tom is not a Plant”?

TECHNICAL DEPTH

In symbolic AI, knowledge is encoded via knowledge representation: semantic networks, frames, ontologies or logical propositions. Entities and their relations (e.g. is-a, has-a) are defined explicitly.

Inference is the application of rules over these representations. The demonstration in Figure 2.1 uses transitivity in an is-a hierarchy. An ontology keeps two links apart: Tom is an instance of the class Cat (instance-of); Cat is a subclass of Mammal (subclass-of). If “Tom instance-of Cat” and “Cat subclass-of Mammal,” then “Tom instance-of Mammal” is derived, so Tom is a mammal. The figure draws both links as one arrow type, “is a.” This is the essence of symbolic reasoning.

The knowledge base contains only consecutive “is-a” links. For each query the inference engine follows the chain using the transitivity rule: if the target is in the chain, “Yes”; otherwise, “Unknown.”

Written formally, the transitivity rule between classes reads: $\text{subclass-of}(A, B) \wedge \text{subclass-of}(B, C) \rightarrow \text{subclass-of}(A, C)$. For an instance: $\text{instance-of}(a, B) \wedge \text{subclass-of}(B, C) \rightarrow \text{instance-of}(a, C)$. The engine applies this rule over and over along the chain; on a five-node chain it either reaches the target or runs off the end within four steps.

The chain only ever followed “is a” links. Getting from what a machine knows to what it should do takes another kind of sentence: if it is raining, then what?

2.3 If... then...: rules and expert systems

At the heart of classical AI beats one tiny sentence: “IF this is true, THEN do that.” This is a rule. Stack enough rules and systems are born that act like masters of a field. These are expert systems: apprentices that hold a doctor’s knowledge as question-and-answer rules.

In the 1980s these apprentices worked everywhere, from medicine to engineering. Below is a tiny one: switch the conditions on and off in your head; see in Figure 2.2 which rule “fires” and how the advice is born.

Margin note. One rule’s output can trigger another rule; this is “chaining.” From a single fact the system can build a long chain of reasoning.

Setup. At the top of the figure are three facts: “It’s raining” is on, “It’s cold” and “It’s windy” are off. Under them sit five rule cards, R1 to R5; the IF condition is on the left of each card, the THEN advice on the right. A rule that fires is shaded; R5 carries a “chain” mark because it depends on another rule. The advice of every fired rule is lined up at the bottom. At the start only R1 fires: “Take an umbrella.” A small table in the figure lists three scenarios; the text below walks through them.

Step by step. The system does the same three jobs every time. It looks at the facts that are on, compares each rule's IF part with those facts, and fires the rules that match. Three scenarios:

Scenario	Facts on	Rules fired	Advice line
A (initial)	Rain	R1	Take an umbrella
B	Rain, Wind	R1, R5	Take an umbrella · Careful: the umbrella may flip!
C	Cold, Wind	R2, R3	Wear a coat · Wear a scarf

Figure 2.2 · A tiny expert system

FACTS · SCENARIO A

☒ It's raining

☐ It's cold

☐ It's windy

IFTHEN

R1It's raining→Take an umbrella🔥

R2It's cold→Wear a coat

R3Cold AND windy→Wear a scarf

R4No rain AND not cold→You can dress light

R5chain"Umbrella" fired AND windy→Careful: the umbrella may flip!

SYSTEM ADVICE
Take an umbrella

THREE SCENARIOS

Scenario	Facts on	Fired	Advice line
A (initial)	Rain	R1	Take an umbrella
B	Rain, Wind	R1, R5	Take an umbrella · Careful: the umbrella may flip!
C	Cold, Wind	R2, R3	Wear a coat · Wear a scarf

Figure 2.2 Live demo: book.onuronder.com/d/en/41c4cadb5f



In scenario A only rain is on. R1's condition is "It's raining": it holds, so R1 fires. R2 and R3 ask for cold, R5 asks for wind; all three stay silent. R4 says "No rain"; it is raining, so R4 is silent too.

In scenario B wind is on as well. The IF part of R5 names no fact but the result of another rule: "Umbrella" fired. So the engine runs two rounds. In the first round R1 fires and drops an "umbrella" note into working memory. In the second round R5 sees that note, combines it with the wind and fires. Two facts gave two pieces of advice; the second is built on the first. This is chaining. Set rain to off and R1 goes dark, no note is dropped, and R5 stays silent too: wind alone is not enough.

In scenario C rain is off, cold and wind are on. R1 and R5 are silent. Cold fires R2; cold AND windy fires R3. R4 gets past its "no rain" condition, but its "not cold" condition fails; it stays silent.

What is happening? The system checks every condition that is on against its rules: whichever rule's "IF" part holds, that rule fires and gives its advice. R5 is the odd one out: it waits for another rule's result. One rule feeding the next is chaining.

Try it yourself. 1) All three facts are on: which rules fire, and how many pieces of advice are on the advice line? 2) Only "It's windy" is on. What does the system advise? Does that advice make sense to you; which rule is missing? 3) If R5 depended on "It's windy" alone, what would be lost?

TECHNICAL DEPTH

Production rules take the IF-THEN form; an inference engine triggers rules whose conditions match facts in working memory. Forward chaining goes from facts to conclusions; backward chaining works from a goal toward evidence.

Expert systems (e.g. MYCIN, 1970s) used this architecture: a knowledge base (rules) plus an inference engine. In Figure 2.2 you will also see a fact produced by one rule trigger another rule

TECHNICAL DEPTH · CONTINUED

(chaining). The limit: rules must be hand-written, and exceptions explode.

R5 is a chain: it triggers only if R1 fired (umbrella) and it is windy.

The engine of the demonstration makes two passes. The first pass evaluates only the rules that depend on facts and adds the derived facts they produce ($R1 \rightarrow \text{umbrella}$) to working memory. The second pass evaluates every rule again against this extended memory. For longer chains the same loop repeats until no new fact is produced.

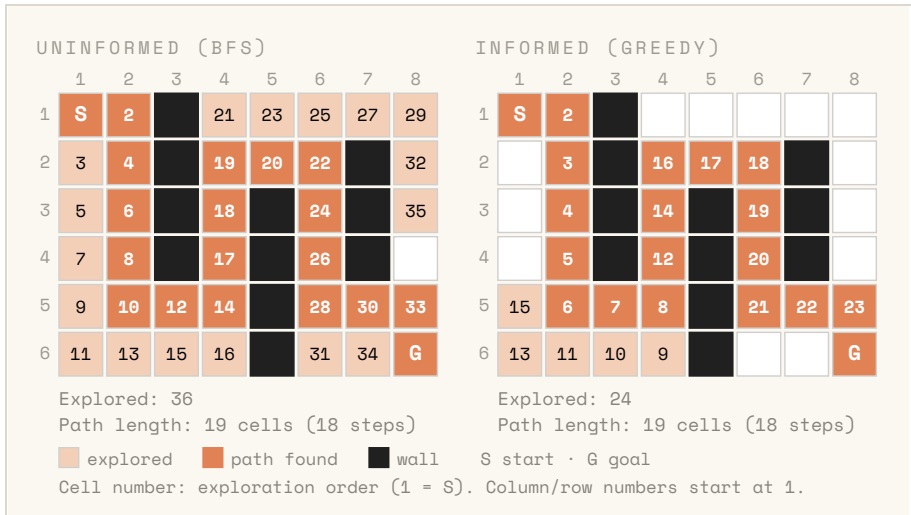
Rules say what to do. When there are thousands of options, though, which one should be tried first? Classical AI had a second tool for that: search.

2.4 Search and heuristic shortcuts

Finding the way out of a maze, picking a chess move, plotting a route through town... all versions of the same game: a crowd of possibilities lies before you, and you are hunting the one that reaches the goal.

The most patient method is trying every possibility one by one, but that can be terribly slow. A heuristic takes a shortcut: it guesses “which direction looks more promising?” Figure 2.3 races the two: the uninformed one scans everywhere; the informed one points its nose at the goal.

Margin note. Greedy heuristics buy speed, but at a price: they can sometimes miss the best solution. They prefer “good enough” over “perfect.” More careful methods such as A^ win the guarantee back with a suitable heuristic.*

Figure 2.3 · Pathfinding: uninformed vs informedFigure 2.3 Live demo: book.onuronder.com/d/en/b1946ef054

Setup. The figure shows a grid of 8 columns and 6 rows; 11 of the 48 cells are black walls. S (start) is top left, G (goal) bottom right. The walls form three vertical barriers: four cells from the top in column 3, four from the bottom in column 5, three in the middle of column 7. So the path passes the first barrier at the bottom, the second at the top, the third at the bottom. Both panels show the same grid: uninformed search left, informed right. Light orange cells are the explored cells, dark orange cells the path; each cell's number is its exploration order.

Step by step. Uninformed search (BFS):

1. It starts at S and queues S's neighbors; first in, first out.
2. It spreads in rings: cells one step from S, then two, then three.
With no sense of direction, it scans the left corridor, the bottom row and the middle corridor alike.

3. It stops when G leaves the queue. The counter reads 36 cells explored, 36 of the 37 empty cells; only column 8, row 4 was never touched.
4. The path is read backward: 19 cells (S and G included), that is, 18 steps. This is the path with the fewest steps possible; since every step on the grid costs the same, it is the cheapest path too.

Greedy informed search:

1. It scores every cell by its street distance to the goal: column difference plus row difference. For S, $7 + 5 = 12$.
2. It always takes the lowest-scoring cell from the queue. It goes down the left corridor, passes under the first barrier and reaches column 4.
3. Here it is fooled: the bottom row shares G's row, so it looks close. Because the wall in column 5, row 6 blocks the way to the right, it tries columns 4, 3, 2 and 1 of the bottom row, then column 1, row 5: five cells wasted.
4. Following the scores, it climbs column 4 to row 2, comes down column 6 and reaches G. The counter reads 24 cells.
5. The path: again 19 cells, 18 steps.

Method	Cells explored	Path (cells)
Uninformed (BFS)	36	19
Greedy informed	24	19

The greedy search visited a third fewer cells and still found the shortest path. That is luck, not a guarantee: a low-scoring dead end cost it five cells, and a more devious maze could mean a long detour. BFS's 36 cells are the price of the guarantee. A*, which adds the distance already traveled to the score, keeps both the guarantee and the savings with the same heuristic.

What is happening? You are racing two searchers. The uninformed one patiently scans in every direction; when every step costs the

same, it finds the path with the fewest steps in the end, but it visits many squares. The greedy informed one always runs toward the goal; it visits few squares but sometimes misses the shortest path. That is the trade-off between speed and guarantee; methods such as A* win the guarantee back by counting the distance already traveled.

Try it yourself. 1) Score two cells: column 4, row 2 and column 1, row 6. Which looks closer to the goal? Which one is on the path? 2) S scores 12, but the shortest path is 18 steps. Where does the difference come from? 3) Remove the wall in column 5, row 6: how many steps is the shortest path now?

TECHNICAL DEPTH

Many classical AI problems are modeled as state-space search. Uninformed search, such as breadth-first search (BFS), scans systematically without any direction information; when all edges have the same cost it guarantees a minimum-step and therefore minimum-cost path, but it expands many nodes. Unequal edge costs require a different method.

Informed search estimates closeness to the goal with a heuristic function $h(n)$; greedy best-first uses only h (fast but no optimality guarantee), while A* uses $g(n)+h(n)$: with an admissible h (one that never exceeds the true distance) it finds the shortest path in tree search; graph search also needs h to be consistent, or nodes reached by a better path to be reopened. In Figure 2.3, compare how many cells BFS and heuristic search each explore.

BFS (uninformed) expands layer by layer with a FIFO queue and guarantees the shortest path on an equal-cost grid. Informed (greedy best-first) search picks the node minimizing the Manhattan distance $h(n)$ to the goal; it opens far fewer cells but does not guarantee the shortest path.

On this grid, with zero-based coordinates, $h(n) = |x - 7| + |y - 5|$. Results: BFS expanded 36 nodes, greedy search 24; both found

TECHNICAL DEPTH · CONTINUED

the 18-step shortest path. Because h never exceeds the true distance at any node (it is admissible) and changes by at most 1 between neighboring cells (it is consistent), A^* with the same h also finds the shortest path in graph search and usually expands fewer nodes than BFS.

Search assumed that the world is certain: a wall is a wall, and the goal stays where it is. Tomorrow's weather offers no such certainty, and knowledge that is not certain needs a different tool.

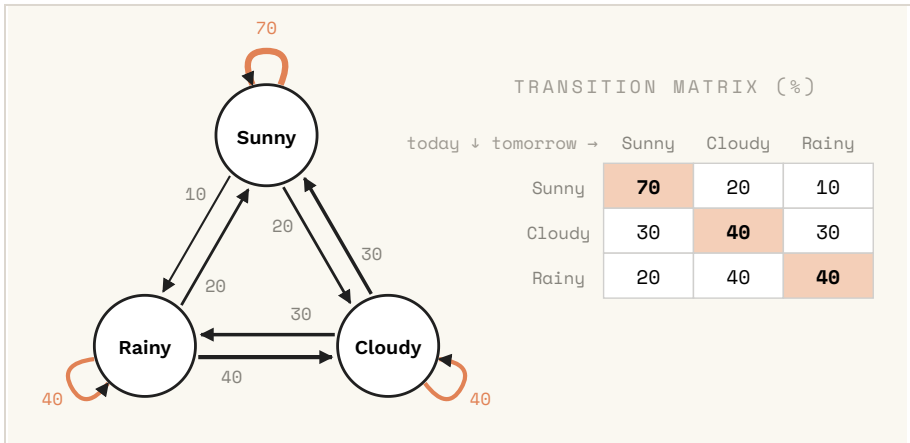
2.5 Handling uncertainty: probability and Markov

Strict rules stumble in the real world, because the world is uncertain. “If it rains, take an umbrella” is easy to say; but will it rain? Nobody knows for certain. At best, you can name the odds.

Classical AI found an elegant answer: move from state to state by probability. The Markov chain is the most famous example: a weather game played with a loaded die. The table under Figure 2.4 holds a seven-day example: follow the weather there as it changes according to its odds.

Margin note. The Markov property: “given the present, the future does not depend on the earlier past; how you got here doesn’t matter.” It looks simple, yet it is everywhere, from weather to Google search.

Setup. The figure shows three state boxes, Sunny, Cloudy and Rainy, joined by arrows; the number on each arrow is the chance of that move, in percent. Next to them the same numbers form the transition matrix: rows are today, columns are tomorrow; every row adds up to 100.

Figure 2.4 · A weather Markov chainFigure 2.4 Live demo: book.onuronder.com/d/en/95270c7a90

today \ tomorrow	Sunny	Cloudy	Rainy
Sunny	70	20	10
Cloudy	30	40	30
Rainy	20	40	40

Step by step. First the loaded die. Say today is sunny; the first row reads 70 / 20 / 10. Picture a die with one hundred faces: 1 to 70 mean sun, 71 to 90 cloud, 91 to 100 rain. The machine draws a number from 1 to 100; the range it lands in is tomorrow's weather. The die is loaded: after sun, more sun has a 70 percent share; rain gets only 10.

Now a seven-day chain. The live demo draws the numbers at random; the seven draws here were chosen so that you can follow every step. In each row, find today's ranges, then place the number drawn.

Day	Today	Ranges (S / C / R)	Number drawn	Tomorrow
1	Sunny	1-70 / 71-90 / 91-100	42	Sunny
2	Sunny	1-70 / 71-90 / 91-100	83	Cloudy
3	Cloudy	1-30 / 31-70 / 71-100	55	Cloudy
4	Cloudy	1-30 / 31-70 / 71-100	91	Rainy
5	Rainy	1-20 / 21-60 / 61-100	77	Rainy
6	Rainy	1-20 / 21-60 / 61-100	12	Sunny
7	Sunny	1-70 / 71-90 / 91-100	64	Sunny

After seven days the tally is three sunny, two cloudy, two rainy: 43 / 29 / 29 percent. Moving to rain on day 4, the machine never looked at the three days before; “today is cloudy” and the number 91 were enough.

Finally, the long run. Seven days is not much; because every state can be reached from every other in this chain, after hundreds of days the shares settle, whatever the starting day, and you can find them without any die. Let the long-run shares be S (sunny), C (cloudy), R (rainy). A sunny day arrives by three routes: after sun ($0.7 \cdot S$), after cloud ($0.3 \cdot C$), after rain ($0.2 \cdot R$). If the shares are steady, that sum must again be S:

$$S = 0.7 \cdot S + 0.3 \cdot C + 0.2 \cdot R \quad C = 0.2 \cdot S + 0.4 \cdot C + 0.4 \cdot R \quad R = 0.1 \cdot S + 0.3 \cdot C + 0.4 \cdot R, \text{ and } S + C + R = 1.$$

Solution: $S = 6/13$, $C = 4/13$, $R = 3/13$; about 46 / 31 / 23 percent. Check: $0.7 \cdot 6 + 0.3 \cdot 4 + 0.2 \cdot 3 = 4.2 + 1.2 + 0.6 = 6$. Run the live demo long enough and its bars sway around these numbers.

What is happening? Tomorrow's weather is predicted by looking only at today; once today is known, yesterday doesn't matter. Each day

rolls a die, but the die is loaded: after a sunny day, more sun is likely. As the days pile up, the shares settle into the same proportions every time.

Try it yourself. 1) Today is rainy and the number drawn is 35: what is tomorrow's weather? 2) Today is sunny. What is the chance of rain two days from now? Hint: work out tomorrow's three possibilities separately, then add them. 3) If the Sunny row became 90 / 5 / 5, which way would the long-run split move? Guess first, then check with the first equation.

TECHNICAL DEPTH

Probabilistic models are used to reason under uncertainty. A Markov chain is a stochastic process where, given the current state, the next state does not depend on earlier history (the Markov property: conditional independence given the present); transitions are defined by a probability matrix whose rows each sum to 1.

A finite, irreducible, aperiodic chain converges from any starting state to a unique stationary distribution; the existence of a stationary distribution alone does not guarantee convergence. The idea extends to hidden Markov models, PageRank and the Markov decision processes of reinforcement learning. In Figure 2.4, observe how the long-run distribution forms.

Each new day samples from P's current row; the transition matrix P itself never changes.

The stationary distribution π solves two equations:

$$\pi P = \pi \quad \sum \pi_i = 1$$

For this P, $\pi = (6/13, 4/13, 3/13) \approx (0.462, 0.308, 0.231)$. Every entry of the matrix is positive, so the chain is irreducible and aperiodic; that is why this finite three-state chain converges to the same π from any start. From a sunny start the expected

TECHNICAL DEPTH · CONTINUED

distribution evolves as follows: day 1 (0.70, 0.20, 0.10), day 2 (0.57, 0.26, 0.17), day 3 (0.51, 0.29, 0.20), day 5 (0.47, 0.30, 0.23). Convergence is largely complete within five days; the counter in the demonstration is a sample, so it fluctuates around these values.

Probability loosened the rules a little; but a human still writes the table, the rules and the chain by hand. How much of that load can a person carry? On that question AI researchers split into two camps.

2.6 Neats and scruffies

For years, AI researchers were split into two rival camps. The “neats” wanted every step proven with clean mathematics. The “scruffies” shrugged: “If it works, it’s good; we’ll find the theory later.”

This quarrel isn’t only history; it continues today. Sort the statements below into the right camp. At the end it will be clear why classical AI hit its wall, and how that collision raised interest in an idea already under study: machines that “learn.”

Margin note. Classical AI’s lesson: hand-writing all the world’s rules is impossible. The solution? Instead of giving the machine rules, teach it to find them in data itself. That is the next chapter.

Setup. The figure shows four statement cards, one under the other; next to each card are two boxes: Neat and Scruffy. The cards are short, like slogans; each is a single sentence that could have come out of a researcher’s mouth.

Self-test. Pick the camp for each statement: Neat or Scruffy?

#	Example	Neat	Scruffy
1	Prove everything with formal logic	☐	☐
2	Use working shortcuts, theorize later	☐	☐
3	Mathematical rigor is essential	☐	☐
4	The real world is messy; be flexible	☐	☐

Figure 2.5 · Which approach?

#	Example	Neat	Scruffy
1	Prove everything with formal logic	☐	☐
2	Use working shortcuts, theorize later	☐	☐
3	Mathematical rigor is essential	☐	☐
4	The real world is messy; be flexible	☐	☐

Figure 2.5 Live demo: book.onuronder.com/d/en/2fd259af6d



The words can mislead: “rigor” and “prove” come from one camp, “shortcuts” and “flexible” from the other, but the test is the attitude. Does the speaker want it proven first, or working first? The first is Neat, the second Scruffy. Answers and reasons are at the back of the book.

What is happening? Two camps, two personalities: neats want every step proven with mathematics; scruffies say “make it work first, theory comes later.” Both turned out right in places; today’s AI is a blend of the two.

Try it yourself. 1) Sort the five methods of this chapter into the two camps: the knowledge chain, the expert system, uninformed search, informed search, the Markov chain. Which ones promise an exact result, and which settle for “good enough”? 2) The heuristic score (Figure 2.3) is which camp’s invention? The guarantee that A* adds to it moves it to which camp? 3) Find an example from your own life: a moment when you first made something work and thought about the theory later.

TECHNICAL DEPTH

The “neat” vs “scruffy” distinction (attributed to Roger Schank) names a methodological tension in AI: between formal, provable, principled approaches (neats; logic and probability theory, for instance) and heuristic, engineering-driven, empirical ones (scruffies).

Classical symbolic AI hit two fundamental limits: the knowledge-acquisition bottleneck (hand-writing every rule doesn’t scale) and brittleness (collapsing on unforeseen cases). These limits increased interest in learning-based methods that were already being studied; symbolic and learning-based approaches developed alongside one another (Rosenblatt’s perceptron dates from 1958). That is how the shift to statistical AI (Chapter 3), which learns from data instead of hand-coding knowledge, picked up speed.

Classical AI’s wall, in two ideas: the knowledge-acquisition bottleneck and brittleness.

The quarrel proved both camps partly right; the next chapter tells how. First, six questions.

2.7 Test yourself

Answers are at the back of the book.

1. How does classical (symbolic) AI represent knowledge?
 - a) By learning from data
 - b) Only with images
 - c) With explicit symbols and rules
 - d) By random guessing
2. What is “if it’s raining, take an umbrella”?
 - a) A probability
 - b) A neural network
 - c) A rule (IF-THEN)
 - d) A dataset
3. Key property of greedy heuristic search?
 - a) They always guarantee the optimal solution
 - b) They guess at random
 - c) They learn from data
 - d) They speed up the search but don’t guarantee the best
4. In a Markov chain, the next state depends on?
 - a) Only the current state
 - b) The entire history
 - c) The future
 - d) Nothing
5. Classical AI’s biggest challenge was?
 - a) Hand-writing all rules, and the messiness of the world
 - b) Being too fast
 - c) Depending on the internet
 - d) Being too cheap

6. What do “Neat” and “Scruffy” describe?

- a) Two approaches in AI research
- b) Two robot types
- c) Two programming languages
- d) Two computer brands

What to keep from this chapter

- Classical AI stores knowledge as explicit symbols and hand-written rules.
- An “is a” chain (instance to class, class to larger class) derives facts nobody wrote down, thanks to transitivity; for anything not in the chain it says “unknown.”
- An expert system fires the IF-THEN rules that match the facts that are on; one rule’s result can trigger another rule.
- On an equal-cost grid, uninformed search guarantees the path with the fewest steps but visits many cells; greedy informed search visits fewer but gives no guarantee; A* combines the two with an admissible heuristic.
- In a Markov chain, once today is known, tomorrow does not depend on the earlier past; in a finite chain where every state can be reached and no cycle locks in, the long-run distribution settles at one fixed ratio.
- Neats want proof first, scruffies want results first; modern AI took something from each.
- Hand-writing rules does not scale and is brittle; these limits raised interest in the learning research that ran alongside the symbolic era.

3

CHAPTER 3

How Machines Learn

From rules to patterns

3.1 How do machines learn?

How would you teach a child what a cat is? By writing rules, “four legs, whiskers, a tail...”? No; you point at cats, and the child works out the rest. The last chapter ran into the wall of rule-writing. So what if machines could learn the way children do?

That is the entire idea of machine learning: show plenty of examples and let it catch the pattern itself. This chapter goes behind the curtain: how is the data prepared, and how does a model grow a little more skillful with every try? You will see it all happen step by step.

Margin note. Classical AI: “Here are the rules, apply them.” Machine learning: “Here are the examples, find the rule yourself.” This small idea made all of modern AI possible.

TECHNICAL DEPTH

Machine learning (ML) learns a function from statistical patterns in data instead of explicitly programmed rules. Where classical AI asks “how do we represent knowledge?” ML asks “how do we estimate the input-to-output mapping from data?”

The basic setup: a model maps inputs to outputs through its parameters (weights); a loss function measures how wrong the predictions are; an optimization method (e.g. gradient descent) updates the parameters to reduce the loss. This chapter builds that loop and the core task types.

Learning from examples starts with knowing what an example is. When you hand a machine an “example,” what are you giving it?

3.2 Features and labels

Picture a detective: clues in one hand, and the case’s final verdict in the other. A machine learning from examples looks at the same pair. The clues are called features: measurable facts describing an example. The verdict is called the label. For an email the clues might be “word count,” “link or password request,” “mentions free”; the label is “Spam” or “Normal.”

Reading hundreds of solved cases, the model learns which clue travels with which verdict. Each example in Figure 3.1 shows its clues and its correct answer.

Margin note. “Garbage in, garbage out” applies here too: if the features are poor, even the best model can’t learn. Understanding the data often matters more than choosing the model.

Setup. The figure shows four short emails and three cues for each one: does it mention “free,” is there a link or password request, does it use urgent language. A link or password request means the email asks the reader to open a link or hand over a password. A filled mark (✓) means the cue is present; an open circle (○) means it is absent. The three cue columns are headed “Features (input).” The rightmost column, headed “Label (output),” gives the correct answer: Spam or Normal. All four examples are in the table below.

Step by step.

#	Email	mentions "free"	link or password request	urgency language	Label
1	"You won a free iPhone! Click now"	✓	✓	✓	Spam
2	"Meeting tomorrow at 10:00"	○	○	○	Normal
3	"URGENT: your account will close, enter your password"	○	✓	✓	Spam
4	"I've attached the draft report"	○	○	○	Normal

Figure 3.1 • See the features and the label

		FEATURES (INPUT)			LABEL (OUTPUT)
#	Email	mentions "free"	link or password request	urgency language	Label
1	"You won a free iPhone! Click now"	✓	✓	✓	Spam
2	"Meeting tomorrow at 10:00"	○	○	○	Normal
3	"URGENT: your account will close, enter your password"	○	✓	✓	Spam
4	"I've attached the draft report"	○	○	○	Normal

✓ has cue ○ no cue

Figure 3.1 Live demo: book.onuronder.com/d/en/9b4cdaf0e7



1. Read the table row by row. Each row is one example: the three cue columns are that example's features, the last column is its label. The model sees only this pair, never the email itself.
2. Email 1 carries all three cues, and its label is Spam. Email 3 never says "free," but it asks for a password and hurries you. It is Spam too.
3. Emails 2 and 4 carry none of the three cues. Both are Normal.
4. Then compare the cue columns with the label. When the cue count is zero, the label is always Normal; when it is two or three, always Spam. "Free" alone does not decide: email 3 came out Spam without it. The link or password request and the urgent language, though, are present in both Spam examples.
5. Nobody wrote this relation; you read it off the table. The model does the same, only with thousands of rows instead of four. The "two cues means spam" rule drawn from four rows is a provisional guess; a fifth example could break it.

What is happening? An email's clues (features) and its correct answer (label) sit side by side: mentions of "free," a link or password request, urgent language... Seeing many examples, the machine learns by itself which clues go with "Spam"; nobody writes the rule, it draws it from the examples.

Try it yourself. 1) Mark the three cues of the email "Your password has expired; if you do not renew it today, your account will be deleted." By the relation in the table, what is its label? 2) The email "Free coffee, let's meet in the kitchen tomorrow at noon" is in fact Normal. If this row is added to the table, should the model trust the "free" cue more, or less? 3) Write a rule that agrees with all four rows. Does your rule also work on the email in question 2?

TECHNICAL DEPTH

In supervised learning, each example is a pair of a feature vector x and a label y . Features can be numeric or categorical; the

TECHNICAL DEPTH · CONTINUED

model tries to learn the mapping $f(x) \approx y$. Good feature engineering is one of the most decisive steps for performance in classical ML.

The label's type sets the task: a categorical label $\rightarrow$ classification, a quantitative (continuous) label $\rightarrow$ regression. Categories may be coded as numbers; that does not make them regression targets. Figure 3.1 builds intuition for telling spam from normal email with simple features.

Feature vector x = [mentions "free"; link or password request; urgency language], label y = "Spam." In supervised learning the model tries to estimate the mapping $f(x) \approx y$ from these (x, y) pairs, i.e. a decision rule like $P(\text{spam} | x)$.

In the spam example every email came with its correct answer. Can a machine learn without an answer key, or with no teacher at all?

3.3 Three kinds of learning

The school of machines has three kinds of teacher. The first walks around with an answer key: "this is spam, this isn't" (supervised). The second gives no answers at all: "you sort it out," and the machine groups the data itself (unsupervised). The third never lectures; the machine enters the game and collects rewards and penalties as it tries (reinforcement).

Now take each task below to the right teacher. For each, ask: Is there an answer key, is the machine sorting alone, or is it learning by trying?

Margin note. Hint: “Are correct answers given?” → Supervised. “Is the machine grouping on its own?” → Unsupervised. “Is it trying things and collecting reward?” → Reinforcement.

Figure 3.2 · Which kind of learning?

#	Example	Supervised	Unsupervised	Reinforcement
1	Telling cats from dogs using labeled photos	☐	☐	☐
2	Grouping customers by similarity	☐	☐	☐
3	A robot learning to walk by trial and error	☐	☐	☐
4	Predicting house prices from past (labeled) data	☐	☐	☐
5	Learning to achieve a high score by playing a game	☐	☐	☐
6	Clustering unlabeled news by topic	☐	☐	☐

Figure 3.2 Live demo: book.onuronder.com/d/en/79ea5021f2



Setup. The figure shows six tasks and three columns: Supervised, Unsupervised, Reinforcement. Next to each task sit three empty boxes, one per column; you mark the right one in pencil. There is no worked solution; the answers, with their reasons, are at the back of the book.

Self-test. For each task, ask three questions in order. Are the correct answers given up front? Is the machine grouping the data on its

own? Or is it trying things and collecting reward? Then mark one of the three boxes.

#	Example	Supervised	Unsupervised	Reinforcement
1	Telling cats from dogs using labeled photos	☐	☐	☐
2	Grouping customers by similarity	☐	☐	☐
3	A robot learning to walk by trial and error	☐	☐	☐
4	Predicting house prices from past (labeled) data	☐	☐	☐
5	Learning to achieve a high score by playing a game	☐	☐	☐
6	Clustering unlabeled news by topic	☐	☐	☐

Once all six are marked, look back at the list. Three of the tasks say “labeled” or “unlabeled” outright; in the others, what told you whether a label exists? And tasks 3 and 5 have something in common: where does the correct answer come from in each? Each column gets two tasks, and that is by design.

What is happening? There are three ways to teach a machine: show it the right answers (supervised), give no answers and let it group things itself (unsupervised), or let it try and collect reward/penalty (reinforcement). Place each task on the right path.

Try it yourself. 1) Put these three tasks into the same three columns. A translation program learning from millions of translations made by humans; a chess program growing stronger by playing against itself; a supermarket finding “bought together” product groups from its receipts. 2) Turn the three questions of the self-test into a decision tree: which question comes first, and at which answer do you stop? 3) Write a task that fits two columns at once, and use it to explain

the sentence “these boundaries are not sharp” from the technical text.

TECHNICAL DEPTH

Supervised learning learns $f(x) \approx y$ from labeled (x, y) pairs (classification/regression). Unsupervised learning extracts structure from x alone (clustering, dimensionality reduction). Reinforcement learning is an agent learning a policy that maximizes a reward signal in an environment.

These boundaries are not sharp: semi-supervised and self-supervised learning sit in between. Modern large language models are largely trained with self-supervised pretraining, generating the labels from the data itself. Sort the tasks of Figure 3.2 into the three core types.

Two tasks landed in the Supervised column: telling cats from dogs and predicting house prices. Both come with labels, yet one asks for a category and the other for a number. What does that difference change?

3.4 Classification and regression

Supervised learning asks two basic questions. One says “how much?”: What does this house cost? That is regression; its answer is a number. The other says “which one?”: Is this email spam or not? That is classification; its answer is a category.

Try both yourself below. In regression you’ll find the “best line” through the middle of the points; in classification you’ll draw a border between the two groups.

Margin note. Simple rule: if the model predicts a quantity it's regression, if it picks a category it's classification. "How much?" is regression, "Which one?" is classification; a category stays a category even when it is coded as a number.

Figure 3.3 • Two core tasks

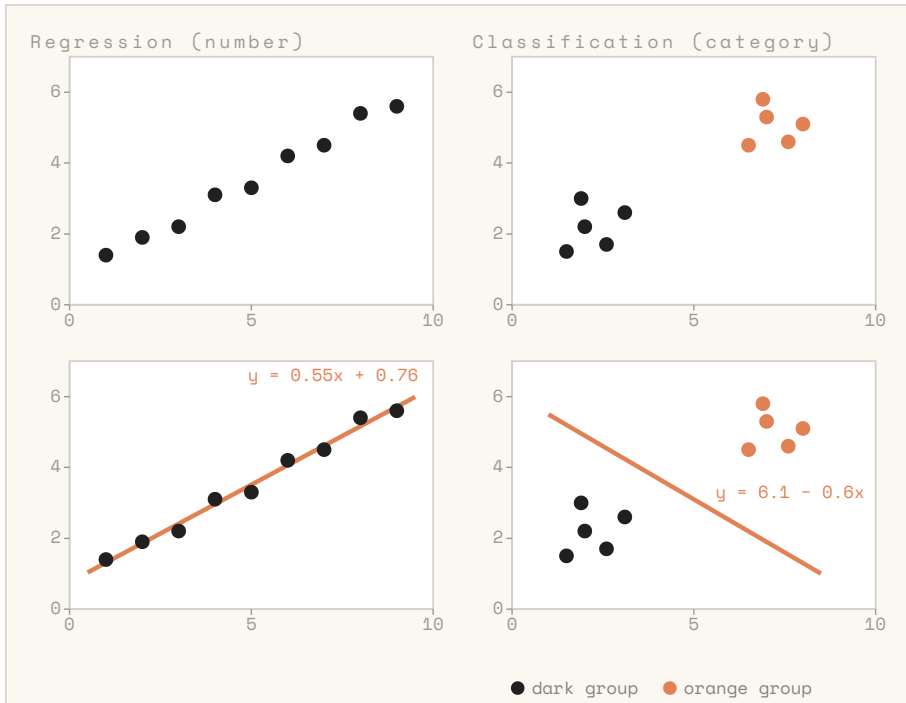


Figure 3.3 Live demo: book.onuronder.com/d/en/d10b8aa300



Setup. The figure shows the two tasks side by side. On the left, “Regression (number)”: nine points and the best line fitted to them. On the right, “Classification (category)”: five green points, five orange points and the boundary drawn between them. The top square of

each panel shows the raw data; the bottom square shows the line or the boundary drawn in.

Step by step.

1. The regression data is nine points: (1, 1.4), (2, 1.9), (3, 2.2), (4, 3.1), (5, 3.3), (6, 4.2), (7, 4.5), (8, 5.4), (9, 5.6). As x grows, y grows, but the points do not sit on a single line.
2. The least-squares line comes from two means: the x values average 5, the y values 3.51. Slope $m = 33 / 60 = 0.55$; intercept $b = 3.51 - 0.55 \cdot 5 = 0.76$. The line: $y = 0.55x + 0.76$.
3. How close is the line to each point? The measure is the vertical gap at the same x : the observed y minus the y the line gives. That gap is called the residual; it is not the shortest (perpendicular) distance to the line. At $x = 3$ the line says 2.41, the point is 2.2; the residual is -0.21 . At $x = 8$ the line says 5.16, the point is 5.4; the residual is $+0.24$. None of the nine residuals exceeds 0.25. The sum of the squared residuals is 0.22.
4. Why is this sum the measure of “best”? Take a line drawn by eye, $y = 0.5x + 1$: the sum of squared vertical residuals rises to 0.37. Among all possible lines, the least-squares line is the one that makes this sum smallest.
5. The classification data is two groups. Green: (1.5, 1.5), (2, 2.2), (2.6, 1.7), (3.1, 2.6), (1.9, 3). Orange: (6.5, 4.5), (7, 5.3), (7.6, 4.6), (6.9, 5.8), (8, 5.1).
6. The boundary is the line through the points (1, 5.5) and (8.5, 1): $y = 6.1 - 0.6x$. Check: at $x = 3.1$ the boundary says 4.24; the green point at 2.6 is below it. At $x = 6.5$ it says 2.2; the orange point at 4.5 is above it. All ten points are on the right side.
7. Two answers, two kinds. Regression gives a number: for $x = 10$, $0.55 \cdot 10 + 0.76 = 6.26$. Classification gives a side: below the boundary is green, above it is orange.

What is happening? Two core jobs. Regression predicts a number: the “best line” runs between the points; the best line is the one that

makes the sum of the squared vertical gaps, each point's gap at its own x , as small as possible. Classification draws a border separating one group from the other.

Try it yourself. 1) By the line $y = 0.55x + 0.76$, what is the prediction for $x = 6.5$? 2) On which side of the boundary does the point $(4.5, 3.5)$ fall: green or orange? And $(5, 3)$? 3) Add a far-off point such as $(9, 9)$ to the regression data. Does the slope go up or down? Is it a problem that the line chases a single point?

TECHNICAL DEPTH

Regression predicts a continuous target; its simplest form is the least-squares line minimizing the sum of squared errors. Classification predicts a categorical target and learns a decision boundary separating the classes (e.g. logistic regression, support vector machines).

Figure 3.3 fits a least-squares line for regression and shows a linear decision boundary separating two clusters for classification. In reality, decision boundaries need not be linear.

The closed form of the least-squares line: $m = \sum(x_i - \bar{x})(y_i - \bar{y}) / \sum(x_i - \bar{x})^2$, $b = \bar{y} - m\bar{x}$. For the data of Figure 3.3 the numerator is 33 and the denominator 60. The quantity minimized is the sum of squared vertical residuals, $\sum(y_i - m \cdot x_i - b)^2$, not the perpendicular distance to the line.

So far every point came with its label or its number. Take the labels away and nothing is left but the points. Can the machine still find an order in them?

3.5 Clustering and anomalies

Imagine being handed a huge box of buttons and told “sort these.” Nobody says which belongs where; still, you put like with like. That is clustering: the machine groups unlabeled data by similarity, all by itself. And there are always a few odd buttons that fit nowhere; anomaly detection catches those. It is one of the tools banks use to catch a suspicious transaction.

The points in Figure 3.4 carry no labels at all. The machine sorts them into two clusters by nearest center. The point far from both clusters is marked as the outlier.

Margin note. What unsupervised learning can do: without anyone saying “these belong together,” the machine finds the structure itself. Anomaly detection is one of the tools banks use in fraud analysis.

Setup. The figure shows eleven gray points and two X marks. The X marks are the cluster centers: center A at the top left (2.5, 7), center B at the bottom right (7, 3). The left square, “before: unlabeled points,” is the state before grouping; every point is the same gray. In the right square, “after: nearest-center groups,” each point has taken the color of its nearest center, and one point is marked as the “outlier.” In this illustration both centers and the outlier are chosen in advance; the machine only computes the distances.

Step by step. Grouping rests on a single question: which center is this point closer to? The distance comes from Pythagoras: $\sqrt{((x - x_m)^2 + (y - y_m)^2)}$. The outlier rule is fixed in advance: a point farther than 2.4 from its nearest center counts as an outlier and joins no cluster. The order is: first assignment, then the threshold, then the center update with the remaining members.

Figure 3.4 • Group unlabeled data

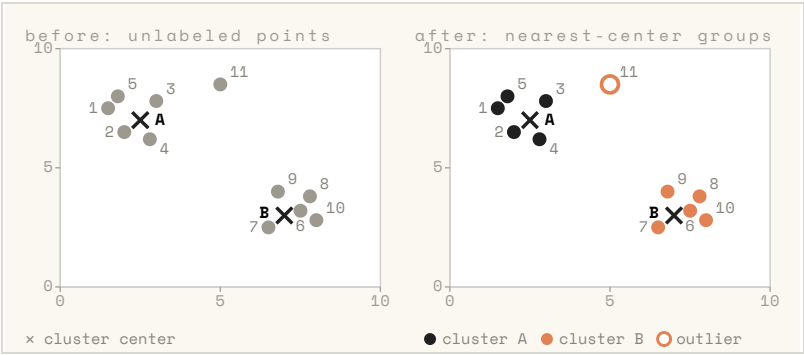


Figure 3.4 Live demo: book.onuronder.com/d/en/afcbfd2bb7



#	Point	Distance to A	Distance to B	Cluster
1	(1.5, 7.5)	1.12	7.11	A
2	(2, 6.5)	0.71	6.10	A
3	(3, 7.8)	0.94	6.25	A
4	(2.8, 6.2)	0.85	5.28	A
5	(1.8, 8)	1.22	7.21	A
6	(7.5, 3.2)	6.28	0.54	B
7	(6.5, 2.5)	6.02	0.71	B
8	(7.8, 3.8)	6.19	1.13	B
9	(6.8, 4)	5.24	1.02	B
10	(8, 2.8)	6.92	1.02	B
11	(5, 8.5)	2.92	5.85	outlier

1. Point 1, (1.5, 7.5): to A, $\sqrt{(1^2 + 0.5^2)} = 1.12$; to B, $\sqrt{(5.5^2 + 4.5^2)} = 7.11$. A is far closer; the point goes to A.

2. Point 9, (6.8, 4): to A, $\sqrt{(4.3^2 + 3^2)} = 5.24$; to B, $\sqrt{(0.2^2 + 1^2)} = 1.02$. It goes to B.
3. Point 4, (2.8, 6.2): 0.85 against 5.28, A again. Do the same calculation for the first ten points; in every one, the two distances differ by a factor of at least five. The decision is never hard.
4. Point 11, (5, 8.5), is different. Its nearest center is A, but the distance is 2.92, above the threshold of 2.4. The other members of A are at most 1.22 from their center; this point is more than twice as far. It is 5.85 from B. By the rule it is an outlier and joins no cluster.
5. That was a single round. Next comes the center update: with the outlier left out, the new center of A is the mean of points 1 to 5, (2.22, 7.2). That is only 0.34 from the old center; the clusters have already settled. One caution: standard k-means knows nothing about outliers; it assigns point 11 to A as well, and the center of A becomes (2.68, 7.42) with six members. The exclusion here is the result of the threshold rule we set.

What is happening? The points carry no labels at all. The machine ties each point to its nearest center (X); similar ones end up in the same cluster. The lone point far from both clusters gets flagged as an “outlier” (an odd example).

Try it yourself. 1) Compute the new center of cluster B: the mean x and mean y of points 6 to 10. How far did it move from the old center (7, 3)? 2) If the point (4.5, 5.5) were added to the data, which center would it go to? Looking at its distance, would you count it as an outlier? 3) The clustering used no labels at all. The “outlier” decision, though, rests on a number and a threshold: which ones, and who chose the threshold?

TECHNICAL DEPTH

Clustering discovers groups by similarity without labels; methods like k-means assign points to the nearest centroid and

TECHNICAL DEPTH · CONTINUED

update the centroids. Anomaly (outlier) detection identifies examples that deviate markedly from the majority distribution.

Figure 3.4 shows k-means' "assignment" step by assigning points to the nearer of two fixed centroids; it also highlights an outlier far from both clusters. Real k-means updates the centroids iteratively until convergence. In this illustration the cluster centers and the outlier are predefined; a real system needs a method and a threshold for detecting anomalies.

The two steps as formulas: assignment $c(i) = \operatorname{argmin}_k \|x_i - \mu_k\|^2$, update $\mu_k \leftarrow$ the mean of the points assigned to cluster k . In Figure 3.4, once the outlier is excluded by the threshold rule, one update moves center A from (2.5, 7) to (2.22, 7.2) and center B from (7, 3) to (7.32, 3.26); a second assignment round changes no point, and the algorithm has converged. Without the threshold, standard k-means assigns point 11 to A; the center of A becomes (2.68, 7.42) with six members, and the point stays in A in the second round.

In clustering the center moved once and the job was done. How does a model with millions of parameters take its "correct a little" step?

3.6 How a model improves: loss and gradient descent

How does a model get "better"? First you measure how wrong it is; that is the loss. Now picture yourself in a fog-covered valley. The bigger the loss, the higher up you stand; your goal is the valley floor. The fog hides the path, but one thing you can still feel: the slope under your feet.

Gradient descent is that walk: feel the slope, take a small step downhill, repeat. In the left panel of Figure 3.5 the ball descends

step by step and the loss melts away. And if your stride is too long? The right panel shows what happens.

Margin note. Learning is this: ask “how wrong am I?”, correct a little, repeat, millions of times over. Most of today’s models, neural networks above all, are trained this way, with gradients; some methods, such as decision trees, learn another way.

Setup. The valley is a parabola: $L(x) = 0.18 \cdot (x - 5)^2 + 0.1$. The horizontal axis is the model’s single parameter x ; the vertical axis is the loss. The floor is at $x = 5$, where the loss is 0.1. The ball starts at $x_0 = 0.6$, on the left slope, where the loss is 3.58. The left panel shows six steps at the low learning rate ($\eta = 0.18$), the right panel six steps at the high rate ($\eta = 4.6$).

Step by step. The rule is the same at every step: $x \leftarrow x - \eta \cdot L'(x)$. The slope is $L'(x) = 0.36 \cdot (x - 5)$: negative to the left of the floor, positive to the right. Subtracting a negative slope makes x larger, so the ball moves right, that is, downhill.

step	x ($\eta = 0.18$)	$L(x)$	x ($\eta = 4.6$)	$L(x)$
0	0.60	3.58	0.60	3.58
1	0.89	3.15	7.89	1.60
2	1.15	2.77	3.11	0.75
3	1.40	2.43	6.24	0.38
4	1.63	2.14	4.19	0.22
5	1.85	1.88	5.53	0.15
6	2.06	1.66	4.65	0.12

1. Low rate, step 1: the slope is $0.36 \cdot (0.6 - 5) = -1.58$. New $x = 0.6 - 0.18 \cdot (-1.584) = 0.6 + 0.285 \approx 0.89$. The loss fell from 3.58 to 3.15.

2. Low rate, steps 2 to 6: each step is a little shorter than the last: 0.29, 0.26, 0.25, 0.23, 0.22, 0.21. The slope shrinks as the floor nears. After six steps $x = 2.06$, loss 1.66. The floor is still far off; bringing x within 0.1 of it takes close to sixty steps. Safe, but slow.
3. High rate, step 1: the same slope, but the step is $4.6 \cdot 1.584 \approx 7.29$ units. The ball misses the floor and flies up the far slope to $x = 7.89$. The loss still fell: 1.60.
4. High rate, steps 2 to 6: the slope is now positive (+1.04), so the ball jumps left, to 3.11. Then 6.24, 4.19, 5.53, 4.65. Once right of the floor, once left: oscillation. Each jump is 0.66 times the one before, so the swings die down and the ball still closes in on the floor. Loss: 0.12.
5. Compare: after six steps the low rate stands at 1.66, the high rate at 0.12. In this valley the big stride won, but it was close to the edge: had $\eta \cdot 0.36$ passed 2, each jump would have grown and the ball would have flown out of the valley.

What is happening? Improving a model is like descending to a valley floor: the bigger the “loss,” the higher up you are. Each step nudges the ball downhill and the loss shrinks. But if the step is too big (a high learning rate), the ball overshoots the bottom and flies up the far slope; the step size matters as much as the direction.

Try it yourself. 1) Work out step 7 at the low rate: the slope at $x = 2.06$, the new x and the new loss. 2) Take two steps with $\eta = 6$ ($\eta \cdot 0.36 = 2.16$). Is the ball getting closer to the floor or farther away? 3) Is there a learning rate that reaches the floor of this valley in a single step? Hint: require $x - \eta \cdot 0.36 \cdot (x - 5) = 5$.

TECHNICAL DEPTH

Training means adjusting the parameters to minimize a loss function $L(\theta)$. Gradient descent moves against the gradient at every step: $\theta \leftarrow \theta - \eta \cdot \nabla L(\theta)$, where η is the learning rate.

TECHNICAL DEPTH · CONTINUED

The learning rate is the most delicate dial: too small and convergence is slow; too large and it can oscillate around the minimum or diverge. Figure 3.5 shows descent on a convex loss curve and the overshoot a large learning rate causes. Deep-network loss surfaces are generally nonconvex, and gradient-based methods such as stochastic gradient descent are the norm in training; some models, such as linear regression, have convex losses, and some, such as tree-based methods, do not use gradients at all.

In this quadratic valley the step rule is linear: $x_{t+1} - 5 = (1 - 0.36\eta)(x_t - 5)$. The factor is 0.935 at $\eta = 0.18$ (one-directional, slow), -0.656 at $\eta = 4.6$ (damped oscillation), and its absolute value exceeds 1 for $\eta > 5.56$ (divergence). Every row of the table follows from the previous one through this single factor.

Figure 3.5 · Descending the loss valley

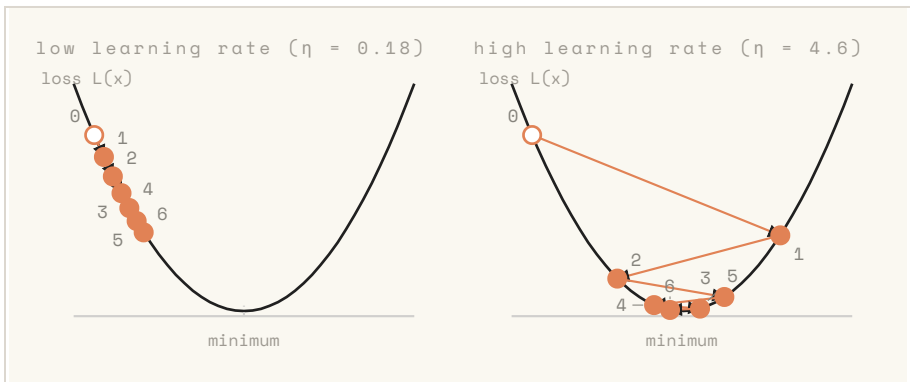


Figure 3.5 Live demo: book.onuronder.com/d/en/a6d0d3d672



The loss drops toward zero and the model looks better and better. Does zero error on the training data mean zero error on new data, though?

3.7 Overfitting and ensembles

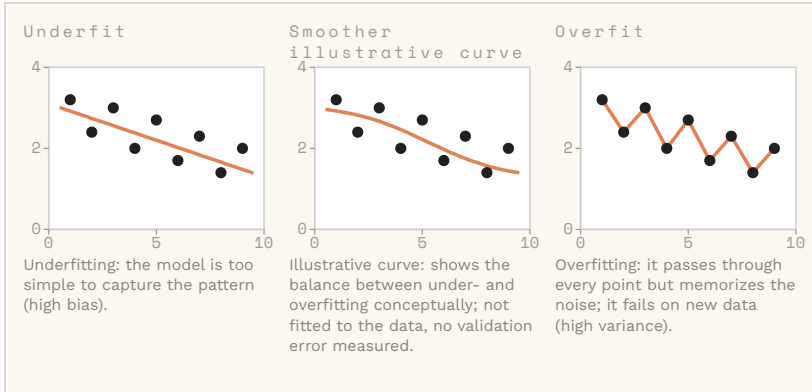
You know the class memorizer: word-perfect on every past exam question, lost the moment the question changes a little. Models sometimes learn “too well” in the same way: they memorize the training examples and flunk the new ones. That is overfitting. The opposite exists too: a model too simple to catch the pattern at all (underfitting). A good model stands in between; it doesn’t memorize, it grasps.

Now fit three models to the same data, like three students: the lazy one, the balanced one and the memorizer. Which one, do you think, will answer a question it has never seen?

Margin note. Memorizing isn’t learning. A student who only memorizes past exam questions flunks a new one. A good model learns the pattern underneath, not the examples themselves.

Setup. All three panels hold the same nine points: (1, 3.2), (2, 2.4), (3, 3.0), (4, 2.0), (5, 2.7), (6, 1.7), (7, 2.3), (8, 1.4), (9, 2.0). The overall trend is downward, but the points jump up and down from one x to the next; that is measurement noise. The left panel, “Underfit,” is a straight line; the middle, “Smoother representative curve,” a gently waving curve; the right, “Overfit,” a broken line joining the nine points one by one. Under each panel stands that model’s verdict. The middle curve was not trained on the data; it was drawn by hand for the explanation.

Step by step.

Figure 3.6 · Same data, three modelsFigure 3.6 Live demo: book.onuronder.com/d/en/d09eae1a3

1. Underfit: the line $y = 3.1 - 0.18x$. At $x = 5$ it says 2.2 against a point at 2.7; the gap is 0.5. At $x = 9$ it says 1.48 against 2.0. The line runs above one point and below the next; it never follows the jolts. Verdict: “Underfitting: the model is too simple to capture the pattern (high bias).”
2. Smoother representative curve: $y = 3.0 - 0.16x + 0.15 \cdot \sin(0.6x)$. It says 2.22 at $x = 5$ and 1.44 at $x = 9$. It does not chase the zigzag; it carries only the downward trend and a slight wave. Verdict: “Bias–variance balance: neither so simple that it misses the pattern nor so complex that it memorizes the noise.” To call it a good generalizer, its error on data not used for training would have to be measured; the curve was drawn by hand, so that measurement does not exist here.
3. Overfit: the broken line passes through all nine points; the training error is zero. But its shape gives it away: from 5 to 6 it drops by 1.0, from 6 to 7 it rises by 0.6. Those ups and downs are noise, not pattern; a fresh measurement would not repeat them. Verdict:

“Overfitting: it passes through every point but memorizes the noise; it fails on new data (high variance).”

4. The exam: hide points 5 and 7 and draw the same broken line through the remaining seven. At $x = 5$ the line joins (4, 2.0) to (6, 1.7) and says 1.85; the truth is 2.7, an error of 0.85. At $x = 7$ it says 1.55; the truth is 2.3, an error of 0.75. The simple line errs by 0.5 and 0.46 at the same two points. On an unseen question the memorizer does worse than the lazy one.
5. Training error on its own misleads. A model has to be tested on data it has never seen; that is what “validation” means. A fair comparison trains every candidate on the same seven points and tests it on the same two. Here that was done for the broken line and the straight line; the middle curve, drawn by hand, never sat the exam.

What is happening? Three different models are fitted to the same data. The too-simple one misses the pattern (underfitting); the too-complex one memorizes every point but stumbles on new data (overfitting). The best is in between: the model that also predicts examples it has never seen. These curves show the idea; which one generalizes well can only be told by the error measured on data not used for training.

Try it yourself. 1) What do the three models say for $x = 10$? Look closely at the broken line. 2) Repeat the hide-and-test exam with points 2 and 8: what errors do the broken line and the straight line make at those two points? 3) The model that passes through all nine points boasts of “zero error.” What does that number prove, and what does it not prove?

TECHNICAL DEPTH

Overfitting is when a model also learns the noise in the training data and loses performance on unseen data; underfitting is when it is too simple to capture the pattern. These are the two

TECHNICAL DEPTH · CONTINUED

ends of the bias–variance tradeoff, usually managed with train/validation splits, regularization and cross-validation.

Ensemble learning combines the predictions of many models (voting, bagging, boosting); when their errors complement one another it can improve generalization, but it does not guarantee beating every individual model, and the gain is measured on validation data. Random forests and gradient boosting are the best-known examples.

The hide-and-test exam of Figure 3.6 is a single-fold train/validation split: two points form the validation set, seven the training set. Cross-validation is the k -fold version of this: the data is split into k parts, each part is held out in turn, the rest trains the model and the errors are averaged; holding out each point one at a time (here $k = 9$) is a special case.

One idea ran through this chapter: grasp the pattern instead of memorizing the examples. The six questions below check whether it stuck.

3.8 Test yourself

Answers are at the back of the book.

1. What is the core difference between ML and classical AI?
 - a) It learns from data instead of hand-written rules
 - b) It never makes mistakes
 - c) It runs faster
 - d) It needs the internet

2. What is an example's "correct answer" called?
 - a) Label
 - b) Gradient
 - c) Feature
 - d) Model
3. Which task groups unlabeled data?
 - a) Clustering (unsupervised)
 - b) Classification
 - c) Reinforcement
 - d) Regression
4. What does gradient descent do?
 - a) Generates labels
 - b) Deletes data
 - c) Slows the model
 - d) Updates parameters step by step to reduce loss
5. What is overfitting?
 - a) Not learning at all
 - b) Learning too fast
 - c) Memorizing training data and failing on new data
 - d) Compressing data
6. How does an agent learn in reinforcement learning?
 - a) By reward and penalty, trial and error
 - b) From labeled examples
 - c) By memorizing rules
 - d) By clustering data

What to keep from this chapter

- Machine learning writes no rules; it draws the rule from the examples itself.

- Every example has two parts: the features that describe it and the correct answer, the label.
- With labels it is supervised learning, without them unsupervised, and with rewards reinforcement learning.
- “How much?” is regression (a house price), “which one?” is classification (spam or not); a category coded as a number is still classification.
- Clustering groups unlabeled data by similarity; a point far from every cluster counts as an outlier by a threshold set in advance.
- Gradient descent measures the loss, reads the slope, takes a small step and repeats this millions of times.
- A model that memorizes the training data flunks new data; a good model grasps the pattern.

4

CHAPTER 4

The Artificial Brain

From neuron to network

4.1 The artificial brain: deep learning

Billions of tiny messengers live inside your head: neurons. Each listens to its neighbors' whispers; when the whispers grow loud enough, it shouts and passes the news along. Researchers looked at this simple game and wondered: What if we built a rough mathematical imitation of it?

Artificial neural networks were born from that question. Stack “artificial neurons” into layers and out comes a powerful learning machine. As the layers multiply, the name changes too: deep learning. This chapter sits beside a single messenger first, then moves to a whole network, and on to the special networks that handle images and sequences.

Margin note. An “artificial neuron” is not a real copy of the brain; it is a very crude mathematical analogy. A single neuron does little; the patterns formed by millions of them together are what count.

TECHNICAL DEPTH

Artificial neural networks are only loosely inspired by biological neurons; at heart they are stacks of nonlinear transformations arranged in layers. Each artificial neuron adds a bias term to a weighted sum of its inputs and passes the result through an activation function.

Deep learning is the stacking of many hidden layers; this depth lets the network learn increasingly abstract representations from raw data (edge → shape → object). Without activation functions, stacked linear layers collapse into a single linear function;

TECHNICAL DEPTH · CONTINUED

nonlinearity is what makes depth meaningful. The field entered its modern era with AlexNet's leap on ImageNet in 2012.

The smallest piece of this whole structure is a single artificial neuron. What does it compute on its own, and how does it turn three numbers into a decision?

4.2 A single artificial neuron

Think of the artificial neuron as a gatekeeper. A few messages arrive at its door; the keeper gives each a level of importance (called a weight), adds them up, then adds its own temperament on top (the bias term). It passes the total through a filter and turns it into an output; that filter is what “activation” means. Whether the bell rings is a separate rule: does the output clear a threshold?

In Figure 4.1 the inputs are strengthened and weakened in turn. Follow how the total changes and when the keeper rings the bell; that moment is the neuron “firing.”

Margin note. All a neuron does is “weigh the inputs, sum them, pass them through an activation function.” Repeated millions of times, an operation this simple can recognize faces and write text.

Setup. The figure shows three inputs: x_1 , x_2 and x_3 . Each carries a value between 0 and 1; at the start, 0.60, 0.30 and 0.80. Next to each input sits a fixed weight: $w = [0.7, -0.5, 0.9]$. The box in the middle adds them up and adds the bias term $b = -0.3$ on top. The box on the right applies the activation, sigmoid or ReLU; the activation is the neuron's numerical output. “Fired” is a separate decision rule of this illustration. When the sigmoid output clears 0.5, or the ReLU output

clears 0, the panel on the right reads “The neuron fired!”; in the table the other cases are called “silent.”

Figure 4.1 • Run the neuron

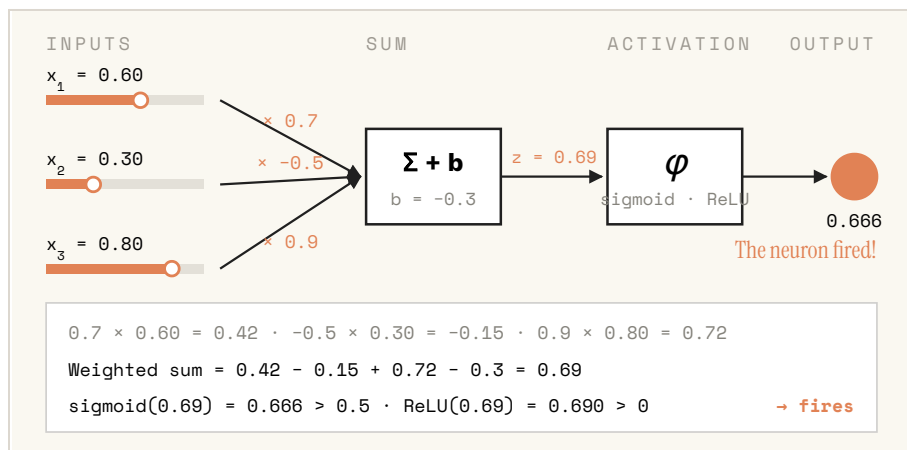


Figure 4.1 Live demo: book.onuronder.com/d/en/239357fa8c



Step by step. Work through the keeper’s arithmetic with the opening values.

1. Multiply each input by its weight: $0.7 \times 0.60 = 0.42$; $-0.5 \times 0.30 = -0.15$; $0.9 \times 0.80 = 0.72$.
2. Add the three: $0.42 - 0.15 + 0.72 = 0.99$. Add the bias term: $0.99 - 0.3 = 0.69$. The line “Weighted sum = 0.69” in the figure is this number.
3. With sigmoid selected, the output is $1 / (1 + e^{-0.69}) = 0.666$. That is above the decision threshold of 0.5, so the panel on the right says “The neuron fired!”
4. With ReLU, the output is $\max(0, 0.69) = 0.690$: a positive sum passes through unchanged. It is above zero, so the neuron fires again.

5. Raise x_2 to 1.00. Its weight is negative, so the sum drops: $0.42 - 0.50 + 0.72 - 0.3 = 0.34$. Sigmoid gives 0.584; the neuron still fires, but more weakly.
6. Put x_2 back to 0.30 and set x_3 to zero. The sum becomes $0.42 - 0.15 + 0 - 0.3 = -0.03$. Sigmoid gives 0.493, ReLU 0.000; in both cases the neuron stays silent. Silent does not mean an output of zero: the sigmoid still gives 0.493; it only stayed below the 0.5 threshold.

x_1	x_2	x_3	Weighted sum	Sigmoid	ReLU	State
0.60	0.30	0.80	0.69	0.666	0.690	fires
0.60	1.00	0.80	0.34	0.584	0.340	fires
0.60	0.30	0.00	-0.03	0.493	0.000	silent
0.00	0.00	0.00	-0.30	0.426	0.000	silent

In the last row every input is zero, and still the sum is -0.30 . That is the bias term at work: by temperament, the keeper starts out closed. x_2 always pulls the sum down; x_1 and x_3 push it up, and x_3 pushes hardest because its weight is largest.

What is happening? A neuron does something simple: it multiplies each input by an “importance weight,” adds them up, then adds a small bias term. The activation function turns that total into the output: sigmoid gives a number between 0 and 1, ReLU zeroes a negative total and passes a positive one through unchanged. Calling the neuron “fired” once the output passes 0.5 is a decision rule of this illustration. Change the inputs and the sum and the output change together.

Try it yourself. 1) Set all three inputs to 1.00. What are the weighted sum and the sigmoid output; does the neuron fire? 2) Only $x_3 = 1.00$, the others 0. Compute the sum and the ReLU output. 3) Only $x_2 = 1.00$, the others 0. Does the sigmoid output pass 0.5?

TECHNICAL DEPTH

An artificial neuron computes $z = \sum w_i x_i + b$, then applies an activation function: $a = \phi(z)$. Common choices are sigmoid (0–1), tanh (–1–1) and ReLU = $\max(0, z)$. The weights tune each input’s importance and the bias term shifts the sum; both are learned in training.

The activation’s role is decisive: without added nonlinearity, the network, however many layers, would equal a single linear transformation. ReLU has become the de facto standard in deep networks thanks to its simplicity and its preservation of gradient flow.

The sigmoid in the figure is $\sigma(z) = 1 / (1 + e^{-z})$; at $z = 0$ it gives 0.5, which is why the firing threshold in the figure was set at 0.5. For ReLU the threshold is $z > 0$. The threshold is a decision rule placed on top of the output, not part of the activation: the sigmoid is not zeroed below it (0.493 for -0.03), and ReLU passes a positive input through unchanged.

A single neuron can draw only one line through the space of its inputs. When dozens are stacked in layers, how does the signal travel through them?

4.3 Layers and the forward pass

One gatekeeper alone can’t achieve much. But house the keepers on the floors of a building and everything changes. News flows from the ground floor to the top; the “hidden” floors in between refine the raw news a little more at every stop.

News flowing from entrance to exit is called the forward pass. Figure 4.2 shows two input patterns; follow the signal as it climbs floor by floor and see which neurons light up.

Margin note. The secret of depth: early layers learn simple features (edges), later layers learn their combinations (shapes, objects). No one programs this by hand; the network discovers it itself.

Figure 4.2 · A live neural network (forward pass)

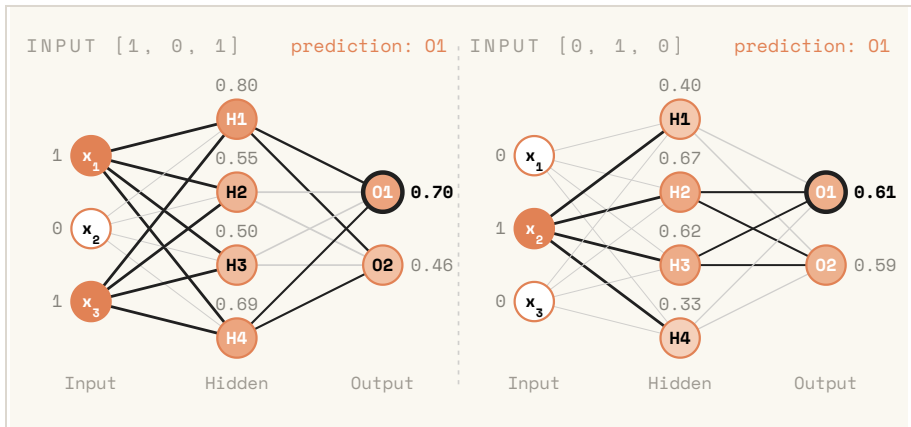


Figure 4.2 Live demo: book.onuronder.com/d/en/b87326a6b3



Setup. The figure has two panels, each with three layers. On the left of a panel are three input boxes, each on (1) or off (0). In the middle sit four hidden neurons, H1 to H4; on the right, two output neurons, O1 and O2. Every connection has a fixed weight; the values appear in the calculation. There is no bias term. The weights were chosen by hand; the two outputs carry no class meaning. Each neuron sums the incoming signals by their weights and passes the total through a sigmoid. The darker a neuron, the higher its activation. The left panel shows the opening state: x_1 and x_3 on, x_2 off. In the right panel only x_2 is on.

Step by step. Compute the left panel, the input [1, 0, 1]. The input weights of the hidden neurons are, in order: H1 [0.6, -0.4, 0.8], H2 [0.5, 0.7, -0.3], H3 [-0.6, 0.5, 0.6], H4 [0.3, -0.7, 0.5].

1. H1: $0.6 \times 1 + (-0.4) \times 0 + 0.8 \times 1 = 1.40$; sigmoid 0.80. The brightest neuron in the layer.
2. H2: $0.5 + 0 - 0.3 = 0.20$; sigmoid 0.55.
3. H3: $-0.6 + 0 + 0.6 = 0.00$; sigmoid 0.50. Even with a sum of zero, the neuron stays at half brightness.
4. H4: $0.3 + 0 + 0.5 = 0.80$; sigmoid 0.69.
5. Output O1, weights [0.7, -0.5, 0.6, 0.4]: $0.7 \times 0.80 - 0.5 \times 0.55 + 0.6 \times 0.50 + 0.4 \times 0.69 = 0.86$; sigmoid 0.70.
6. Output O2, weights [-0.4, 0.6, 0.5, -0.6]: $-0.32 + 0.33 + 0.25 - 0.41 = -0.15$; sigmoid 0.46.
7. $O1 > O2$. The illustration calls this the network's "prediction"; in the figure it is the darkest box. O1 and O2 stand for no class; the illustration does not show that a task has been learned.

In the right panel only x_2 is on. The same procedure gives this table:

Input	H1	H2	H3	H4	O1	O2	Prediction
[1, 0, 1]	0.80	0.55	0.50	0.69	0.70	0.46	O1
[0, 1, 0]	0.40	0.67	0.62	0.33	0.61	0.59	O1

When the input changes, the brightness pattern of the hidden layer flips: on the left H1 stands out, on the right H2 and H3 shine. The winner does not change, though; in the second case O1 leads by only 0.02. Nobody has trained these weights, so the network's "opinion" is still arbitrary.

What is happening? The signal moves left to right, layer by layer: each neuron sums what reaches it and passes it on to the next layer. The brighter a neuron, the stronger its response. The brightest box on the far right is what we call the network's "prediction"; because the weights were chosen by hand, that prediction means nothing yet.

Try it yourself. 1) With every input off ($[0, 0, 0]$), what value do the four hidden neurons take? Compute the outputs as well. 2) For the input $[1, 1, 1]$, find H1's sum and its sigmoid value. 3) Does O2 win for any of the eight possible input patterns? Guess first, then test two of them by calculation.

TECHNICAL DEPTH

In a feedforward network each layer takes the previous layer's activations: $a^{(l)} = \phi(W^{(l)}a^{(l-1)} + b^{(l)})$. The input layer carries the raw data, hidden layers carry intermediate representations, and the output layer carries the prediction. The weight matrices and bias-term vectors are the network's learned parameters.

The network in Figure 4.2 computes a real forward pass with fixed weights chosen by hand: the computation runs from input to output, and neuron brightness reflects the activation values. The highest output is called the network's "prediction" only for the sake of the illustration; the outputs have no class meaning, and the demonstration does not show that a task has been learned. Training is the act of tuning these weights, and that comes next.

In the network of Figure 4.2, $W^{(1)}$ is 4×3 and $W^{(2)}$ is 2×4 ; $b^{(1)} = b^{(2)} = 0$. There are $12 + 8 = 20$ learnable weights in all; ϕ is the sigmoid in both layers.

This network gives the same answer for all eight input patterns, because nobody has tuned its weights. Can the error itself be used to correct them?

4.4 Backpropagation: learning from error

The network starts out guessing at random; forgive its first-day clumsiness. So how does it become a master? First it compares its guess with the right answer and measures how wrong it was: the

error. Then that error walks backward, from exit to entrance, telling every connection it passes: “adjust your small share of the blame.”

That backward walk is called backpropagation; it works out each connection’s share of the error. The correction is a separate step: each weight is nudged in proportion to its share (gradient descent). Follow Figure 4.3 round by round: the error flows back and the output edges toward the right answer. As the error shrinks, the backward whisper fades too; there is less blame left to hand out.

Margin note. The forward pass means “make a guess”; backpropagation means “learn from the error.” Repeat those two steps millions of times and that is deep learning; there is nothing more to it.

Setup. The figure is laid out like a film strip; each frame is one training round. The network is small but the arithmetic is real: two inputs ($x_1 = 1.0$, $x_2 = 0.5$), two hidden neurons, one output; all sigmoid. The target is 0.8; the loss is $L = \frac{1}{2}(\hat{y} - 0.8)^2$; the learning rate is $\eta = 2$. The starting weights were chosen for the explanation: hidden layer $W_1 = [[0.3, -0.2], [0.4, 0.1]]$, bias terms 0; output weights $w_2 = [0.5, -0.3]$, $b_2 = 0$. On the left of a frame is the network with an arrow running back from output to input: the gradient flowing backward. In the middle, a vertical bar shows the network’s output ($\hat{y}$), and the line above it marks the target (0.8). Under each frame is that round’s error (target $- \hat{y}$). In round 0 the weights have not been updated yet; every round is one forward pass, one gradient computation and one update.

Step by step. Read the frames in order; work out the first round yourself.

1. Round 0, forward pass: hidden neuron 1 sums $0.3 \cdot 1.0 + (-0.2) \cdot 0.5 = 0.20$, sigmoid 0.5498; neuron 2 sums $0.4 \cdot 1.0 + 0.1 \cdot 0.5 = 0.45$, sigmoid 0.6106. The output sum is $0.5 \cdot 0.5498 - 0.3 \cdot 0.6106 + 0 = 0.0917$; $\hat{y} = \text{sigmoid}(0.0917) = 0.5229$.
2. Round 0, loss: the error is $0.8 - 0.5229 = 0.2771$; $L = \frac{1}{2} \cdot 0.2771^2 = 0.0384$.

3. Round 0, backpropagation (computing the gradient): at the output neuron $\delta_2 = (\hat{y} - 0.8) \cdot \hat{y} \cdot (1 - \hat{y}) = -0.2771 \cdot 0.5229 \cdot 0.4771 = -0.0691$. The gradient of each output weight is that number times the hidden activation: $\partial L / \partial w_2 = [-0.0691 \cdot 0.5498, -0.0691 \cdot 0.6106] = [-0.0380, -0.0422]$; $\partial L / \partial b_2 = -0.0691$. The chain goes one step further back: for hidden neuron j , $\delta_j = \delta_2 \cdot w_{2j} \cdot h_j(1 - h_j)$, and the gradient is $\delta_j \cdot x_j$.
4. Round 0, update (optimization): $w \leftarrow w - \eta \cdot \text{gradient}$. $w_2 = [0.5 + 2 \cdot 0.0380, -0.3 + 2 \cdot 0.0422] = [0.5760, -0.2156]$; $b_2 = 0 + 2 \cdot 0.0691 \approx 0.1383$. Hidden layer $W_1 = [[0.3171, -0.1914], [0.3901, 0.0951]]$. Backpropagation computed the gradient; this step is what changes the weights.
5. Round 1: a forward pass with the new weights gives $\hat{y} = 0.5817$, error 0.2183, loss 0.0238. The output came 5.9 points closer in a single round.
6. Rounds 2 to 8: the same three steps repeat; the numbers are in the table. In round 8, $\hat{y} = 0.7390$, error 0.0610, loss 0.0019.
7. The steps are shrinking: round 1 raised $\hat{y}$ by 0.059, round 8 by 0.010. As the error shrinks so does the gradient, and since the update is proportional to it, the step gets shorter.

Round	w_2	b_2	$\hat{y}$	Error ($0.8 - \hat{y}$)	Loss
0	[0.5000, -0.3000]	0.0000	0.5229	0.2771	0.0384
1	[0.5760, -0.2156]	0.1383	0.5817	0.2183	0.0238
2	[0.6354, -0.1513]	0.2445	0.6258	0.1742	0.0152
3	[0.6818, -0.1021]	0.3261	0.6585	0.1415	0.0100
4	[0.7183, -0.0639]	0.3897	0.6832	0.1168	0.0068
5	[0.7477, -0.0335]	0.4403	0.7022	0.0978	0.0048
6	[0.7716, -0.0090]	0.4812	0.7172	0.0828	0.0034
7	[0.7914, 0.0111]	0.5148	0.7292	0.0708	0.0025
8	[0.8080, 0.0279]	0.5427	0.7390	0.0610	0.0019

There is no ready-made rule in the table; each row follows from the previous one through a gradient and an update. The error fell from 0.28 to 0.06 in eight rounds; it did not reach zero, and at this pace it would take a long time to. This is training error measured on a single example: how the network does on new examples still has to be tested. In real training over millions of examples the curve does not descend this smoothly, and now and then it even rises; but the loop is the same: forward pass, loss, gradient, update.

Figure 4.3 · Learn from error (real training)

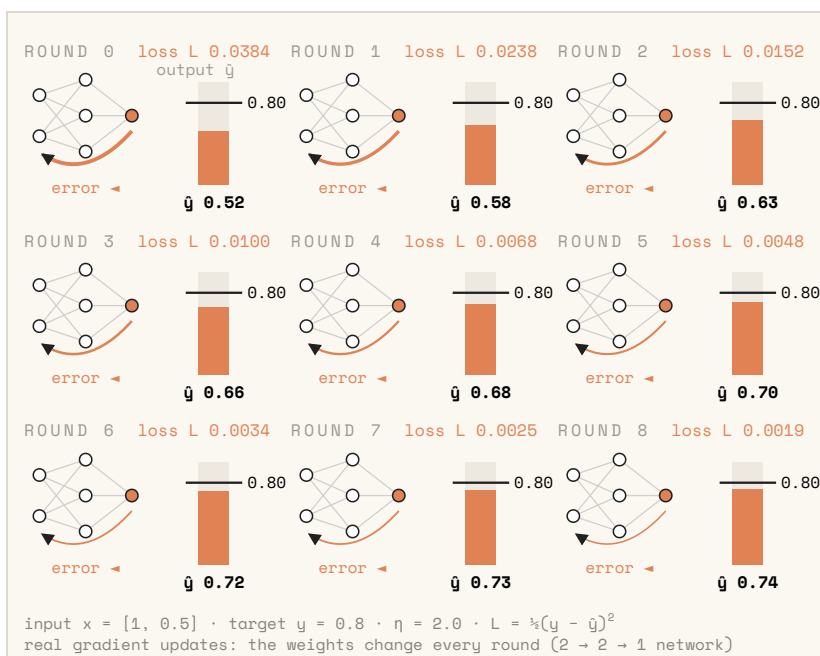


Figure 4.3 Live demo: book.onuronder.com/d/en/b198800899



What is happening? The network starts by guessing with untrained weights, so the error (loss) is high. Each round, backpropagation computes how the loss changes with every connection (the

gradient), from output back to input; then gradient descent nudges every weight in the direction that reduces the error. Bit by bit, the output closes in on the right answer. Training error has decreased on this example; performance on new examples still needs testing.

Try it yourself. 1) Compute the output with the weights of round 8 yourself: hidden activations $h = [0.5957, 0.5994]$, $w_2 = [0.8080, 0.0279]$, $b_2 = 0.5427$. What are the output sum and $\hat{y}$; do they agree with the table? 2) In round 0 the gradient of b_2 is -0.0691 . With a learning rate of 1 instead of 2, what would b_2 be after the first update, and how does the step change? 3) In round 8 the error is 0.061. Can you say “the network has learned”? Think about what this error measures and what it does not.

TECHNICAL DEPTH

Backpropagation efficiently computes the gradient of the loss with respect to every weight using the chain rule; gradients are carried backward layer by layer, from output to input. Gradient descent then updates the weights: $W \leftarrow W - \eta \cdot \partial L / \partial W$.

This method (popularized by Rumelhart, Hinton and Williams in 1986) is the key to training deep networks. The animation in Figure 4.3 runs a real training loop on a single example: forward pass, loss, gradient by the chain rule, update by gradient descent; real training repeats the same loop over millions of examples. Backpropagation computes the gradient; the optimization step is what changes the parameters.

The network in the figure is 2-2-1 with sigmoid activations throughout; the loss is $L = \frac{1}{2}(\hat{y} - y)^2$, $\eta = 2$. At the output neuron $\delta_2 = (\hat{y} - y) \cdot \hat{y}(1 - \hat{y})$; for hidden neuron j , $\delta_j = \delta_2 \cdot w_{2j} \cdot h_j(1 - h_j)$; the gradients are $\partial L / \partial w_{2j} = \delta_2 \cdot h_j$, $\partial L / \partial b_2 = \delta_2$ and $\partial L / \partial w_{1ji} = \delta_j \cdot x_i$. Every row of the table was computed with these formulas; there is no ready-made curve.

Backpropagation trains networks of every kind. Feeding the tens of thousands of pixels of a photo into a flat layer is wasteful, though; an image calls for a layout of its own.

4.5 Seeing images: convolutional networks (CNN)

An image holds tens of thousands of pixels; listening to each one separately would be madness. Convolutional neural networks (CNNs) are craftier: like sweeping a flashlight around a dark room, they glide a small “filter” across the image, hunting local patterns; an edge here, a corner there.

In Figure 4.4 the flashlight sweeps the image step by step. At every stop it examines a tiny region and marks its findings on a “feature map.” Swap the flashlight (the filter) and see which edges it catches this time.

Margin note. The same small filter is used across the whole image (weight sharing). So it can spot an edge in the top-left and the bottom-right alike, and it learns with far fewer parameters.

Setup. On the left is a 7×7 image; dark cells hold 1, light cells 0. The image is a plus sign: the fourth row and the fourth column are dark throughout. In the middle is the 3×3 kernel, that is, the filter. The vertical edge kernel has $[1, 0, -1]$ in each of its three rows. The horizontal edge kernel has $[1, 1, 1]$ on top, zeros in the middle and $[-1, -1, -1]$ at the bottom. On the right is the 5×5 feature map: 25 stops, left to right and top to bottom, one cell per stop. Plus means a change from dark to light, minus from light to dark; darkness shows the magnitude. Two stops of each kernel are picked out.

Step by step. Slide the vertical edge kernel along the first row, stop by stop.

1. Stop 1: rows 1 to 3, columns 1 to 3. All zeros; the sum is 0.

2. Stop 2: columns 2 to 4. Its right column (column 4 of the image) is 1 in all three rows. Three rows of $1 \times 0 + 0 \times 0 + (-1) \times 1$ give -3 . Cell 2 of the map: a minus at full darkness; the figure picks out this stop.
3. Stop 3: columns 3 to 5. The dark column lands in the middle, where the kernel is zero. The sum is 0.
4. Stop 4: columns 4 to 6. The dark column is now on the left, weight $+1$. The sum is $+3$; the figure picks out this stop too.
5. Stop 5: columns 5 to 7. All zeros again.
6. In the second row (window rows 2 to 4) the window meets the horizontal arm, where left and right cancel. The values fall to -2 and $+2$.

All 25 stops:

Vertical kernel

0	-3	0	3	0
0	-2	0	2	0
0	-2	0	2	0
0	-2	0	2	0
0	-3	0	3	0

Horizontal kernel

0	0	0	0	0
-3	-2	-2	-2	-3
0	0	0	0	0
3	2	2	2	3
0	0	0	0	0

The vertical kernel marks only the two sides of the vertical line; the horizontal arm vanishes. The horizontal kernel does the opposite; its picked-out stops are 6 (-3) and 16 ($+3$). The same nine numbers travel the whole image; wherever the edge is, it gets found. Shift the

image by one cell and the map shifts by one cell too: that is equi-variance, not invariance.

Figure 4.4 • Convolution: slide the filter

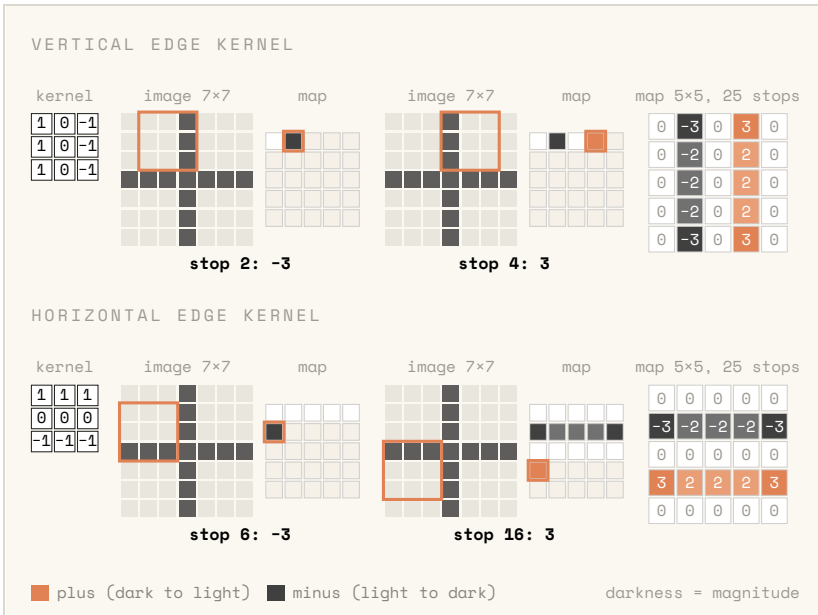


Figure 4.4 Live demo: book.onuronder.com/d/en/39c97bee31



What is happening? A small “filter” roams the image step by step, like a magnifying glass. At every stop, if the little region holds the pattern it is looking for (say a vertical edge), it marks that spot bright. The result is a map showing where in the image that pattern lives.

Try it yourself. 1) With the vertical kernel, compute stop 22: rows 5 to 7, columns 2 to 4. 2) With the horizontal kernel, compute the center stop: rows 3 to 5, columns 3 to 5. 3) With a 5×5 kernel instead of 3×3 , how many cells would the feature map have?

TECHNICAL DEPTH

CNNs detect local patterns with shared-weight convolution kernels; the kernel slides across the image, and at each position the sum of element-wise products produces a feature map. Pooling reduces dimensionality; as layers deepen, the network learns hierarchical representations, from edges to shapes to objects.

Weight sharing and local connectivity greatly reduce the parameter count; under ideal conditions convolution produces translation-equivariant feature maps (shift the input and the map shifts with it), and operations such as pooling can provide approximate invariance to small shifts. LeNet (LeCun) pioneered this architecture; AlexNet (2012) made CNNs visible at scale. Figure 4.4 shows a real convolution operation.

The operation in the figure is $(I * K)(r, c) = \sum_i \sum_j K(i, j) \cdot I(r + i, c + j)$, with $i, j \in \{0, 1, 2\}$. There is no padding and the stride is 1, so a 7×7 image and a 3×3 kernel give a map of size $(7 - 3 + 1) = 5$. Nine weights are shared across 25 positions. The kernel is applied without flipping; in mathematical terms the operation is cross-correlation, which the deep-learning literature calls convolution. Because the vertical kernel is not symmetric, flipping it would reverse the signs in the map.

Convolution captures what sits next to what in space. What about what comes after what in time, such as the order of the words in a sentence?

4.6 Understanding sequences: recurrent networks (RNN)

Picture a child listening to a bedtime story: each new sentence is heard through the memory of the ones before, or the tale falls apart. Text, music and speech are the same: all sequences, and order

matters. “The dog bit the man” and “The man bit the dog” carry the same words yet tell utterly different stories. That is why recurrent networks (RNNs) carry a “memory.”

An RNN listens word by word, refreshing its memory at every step; it moves forward without losing the past. In Figure 4.5 the words are fed in one at a time, and the memory shifts each time.

Margin note. The essence of an RNN in one line: “process each new input together with the memory of everything seen so far.” Because the same cell is reused again and again, it views the sequence as a loop.

Setup. The figure has eight frames, from frame 0 to frame 7. Above each frame is the seven-word sentence “The cat ran because it was scared.” Processed words are shaded dark. Below them, four bars show the hidden state, that is, the network’s memory; each is a number between -1 and 1 (a tanh output), upward positive, downward negative. Under the frame is the count of words processed. In frame 0 all four bars are zero: $h_0 = 0$. The numbers are computed with $h_t = \tanh(W_x x_t + W_h h_{t-1})$; W_x (4×5) and W_h (4×4) are fixed weights chosen for the explanation, not trained.

Step by step. Read the frames from left to right; work out the first two steps yourself. Each word enters as a unit vector: “The” is $[1, 0, 0, 0, 0]$, “cat” is $[0, 1, 0, 0, 0]$, and so on. W_x has five columns, so the sixth and seventh words reuse the first two. $W_x x_t$ is therefore one column of W_x .

1. Frame 0, “0 words processed”: $h_0 = [0, 0, 0, 0]$. All four bars sit at zero.
2. Frame 1, “The” processed: column 1 of W_x is $[0.9, -0.6, 0.2, -0.8]$; $W_h h_0 = 0$. $h_1 = \tanh([0.9, -0.6, 0.2, -0.8]) = [0.72, -0.54, 0.20, -0.66]$.
3. Frame 2, “cat” processed: column 2 of W_x is $[-0.4, 0.8, -0.5, 0.3]$.
Now the memory speaks too: $W_h h_1 = [0.56, -0.44, -0.14, 0.02]$. The

- total is $[0.16, 0.36, -0.64, 0.32]$; $h_2 = [0.16, 0.34, -0.57, 0.31]$. The second word did not erase the first; it mixed with it.
4. Frame 3, “ran”: $h_3 = [0.16, 0.32, 0.53, -0.58]$. The third component flipped from negative to positive.
 5. Frame 4, “because”: $h_4 = [-0.54, 0.11, 0.82, 0.68]$. Even a function word reshaped the memory from end to end.
 6. Frames 5 to 7, “it,” “was,” “scared”: $h_5 = [0.34, -0.81, -0.16, 0.53]$, $h_6 = [0.86, -0.57, -0.55, -0.39]$, $h_7 = [0.09, 0.67, -0.77, 0.20]$. At the end of the sentence the memory carries traces of all seven words; none sits in a place of its own, all are mixed into the same four numbers.

Step	Word	$h[1]$	$h[2]$	$h[3]$	$h[4]$
0	(empty)	0.00	0.00	0.00	0.00
1	The	0.72	-0.54	0.20	-0.66
2	cat	0.16	0.34	-0.57	0.31
3	ran	0.16	0.32	0.53	-0.58
4	because	-0.54	0.11	0.82	0.68
5	it	0.34	-0.81	-0.16	0.53
6	was	0.86	-0.57	-0.55	-0.39
7	scared	0.09	0.67	-0.77	0.20

The exact numbers matter less than the behavior: the memory is not only added to, it is reshuffled at every step. The same four numbers carry all seven words; the network does not open a new memory for each word, it updates the one it has. Because the weights were never trained, these numbers know nothing about the meaning of the words; in a trained RNN, W_x and W_h are learned from data and each word enters as a learned vector instead of a unit vector.

What is happening? The network reads the words one by one, carrying a “memory” as it goes. With each new word it updates that memory using both the new word and everything gathered so far. So

it keeps track of order; “the dog bit the man” and “the man bit the dog” are no longer the same thing to it.

Try it yourself. 1) In the formula $h_t = \tanh(W_h \cdot h_{t-1} + W_x \cdot x_t)$, if $h_0 = 0$, which term contributes nothing in the first step? 2) Feed the words in a different order, swapping the third and the fourth: The, cat, because, ran, it, was, scared. Which column of W_x does step 3 use now? h_2 stays the same; compute h_3 and compare it with the table. 3) Would a model that ignores word order produce the same memory for “the dog bit the man” and “the man bit the dog”? What does an RNN do instead?

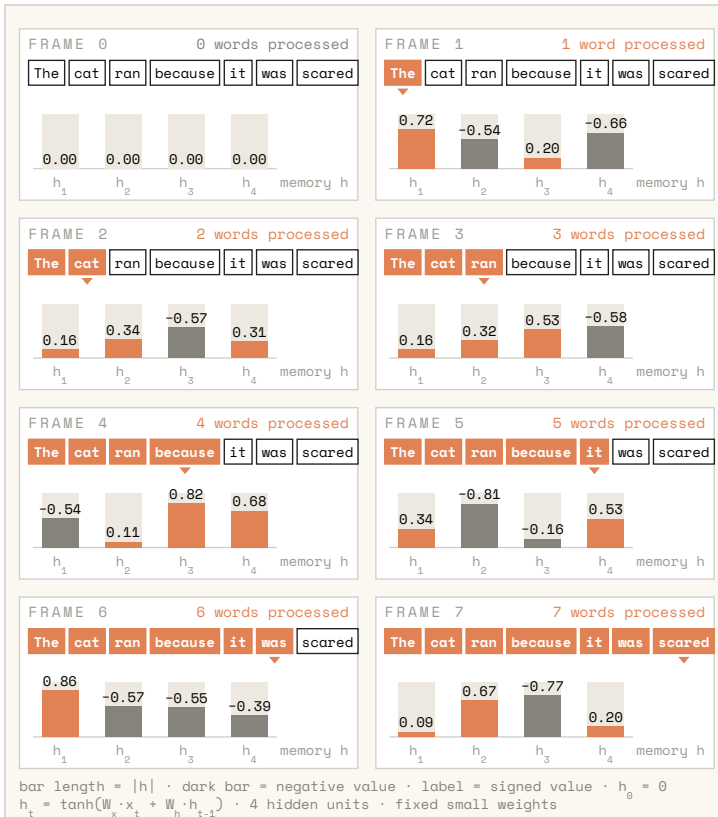
TECHNICAL DEPTH

An RNN updates its hidden state at each timestep with $h_t = \phi(W_h \cdot h_{t-1} + W_x \cdot x_t + b)$; the same weights are reused at every step (parameter sharing in time). That lets it process variable-length sequences through one context vector.

Classic RNNs struggle with long dependencies due to vanishing gradients; LSTM (Hochreiter & Schmidhuber, 1997) and GRU ease this with gate mechanisms. In most modern sequence tasks, RNNs have largely given way to attention-based transformers; they come in the next chapter. The animation in Figure 4.5 computes the hidden state step by step with the equation $h_t = \tanh(W_x x_t + W_h h_{t-1})$, using fixed, untrained weights.

Hidden state $h_0 = 0$. Each step updates it with $h_t = \tanh(W_h \cdot h_{t-1} + W_x \cdot x_t)$; the same weights are reused for every word.

The four bars in the figure stand for a 4-dimensional hidden state vector. The weights: rows of W_x [0.9, -0.4, 0.3, -0.7, 0.5], [-0.6, 0.8, -0.2, 0.4, -0.9], [0.2, -0.5, 0.7, 0.6, -0.3], [-0.8, 0.3, -0.6, 0.9, 0.1]; rows of W_h [0.5, -0.3, 0.2, 0.0], [0.1, 0.4, -0.5, 0.3], [-0.2, 0.6, 0.3, -0.4], [0.3, -0.1, 0.4, 0.5]. They are constants chosen for the explanation, not trained; in a real application x_t is the word’s embedding vector, and W_x and W_h are learned in training.

Figure 4.5 · Processing with memory (real recurrence)Figure 4.5 Live demo: book.onuronder.com/d/en/4feb42a9fb

So far the networks have only recognized: a neuron fired, an edge was found, a sequence was remembered. Can a network produce a face it has never seen, from scratch?

4.7 The art of forgery: GANs

Sometimes the goal isn't recognizing but creating: realistic faces, landscapes, sounds. The trick of generative adversarial networks (GANs) is locking a forger and a detective in the same room. The forger (generator) produces fakes; the detective (discriminator) tries to tell real from fake.

The two race without rest: as the detective catches fakes, the forger sharpens; as the forger sharpens, the detective's eye grows keener. In Figure 4.6, round by round, an image born as pure noise edges toward the real thing as the forger masters its craft.

Margin note. What makes a GAN work: “making a good fake” and “catching the fake” keep pushing each other. Like a forger racing a detective: as both improve, the result becomes strikingly realistic.

Setup. The figure is a film strip of nine frames, from round 0 to round 8. This strip is illustrative: both the images and the percentages were fixed in advance for the explanation, not measured from a trained network. On the left of each frame is the 8×8 image the generator drew: dark and light pixels. The “real” image the generator is trying to learn sits at the top: a filled circle in the middle, with 24 of the 64 pixels dark. On the right of each frame is the discriminator's decision: the probability that the image is fake, and a one-word verdict. Above 50 percent means “Fake!” and below means “Real?”.

Step by step. Read the frames in order.

1. Round 0: the generator draws pure noise; only 31 of the 64 pixels agree with the target, a coin toss. The detective is certain: fake probability 95 percent, verdict “Fake!”
2. Rounds 1 and 2: the probability falls to 84 percent, then 73. The first traces of the circle appear in the image; matching pixels 34,

then 36.

3. Rounds 3 and 4: 62 and 51 percent. Matching pixels 40 and 45. The detective still says “Fake!” but only just; 51 percent is a hair above the line.
4. Round 5: the probability drops to 40 percent, and the verdict becomes “Real?” The detective is fooled for the first time. Matching pixels: 50.
5. Rounds 6 and 7: 29 and 18 percent. The circle is now clearly visible; matching pixels 54 and 60.
6. Round 8: 7 percent, verdict “Real?” All 64 of the 64 pixels are in place; in the strip the generator has matched the target. The 7 percent is an illustrative number, not a measured probability.

Round	Fake probability	Verdict	Matching pixels
0	95%	Fake!	31 / 64
1	84%	Fake!	34 / 64
2	73%	Fake!	36 / 64
3	62%	Fake!	40 / 64
4	51%	Fake!	45 / 64
5	40%	Real?	50 / 64
6	29%	Real?	54 / 64
7	18%	Real?	60 / 64
8	7%	Real?	64 / 64

The probability falls 11 points every round; that is the fixed rule of the animation, not a training result. In real training both networks learn at the same time: the detective sharpens too, so the decline does not follow such a straight line. Nor is 7 percent the theoretical goal: at the ideal equilibrium the discriminator cannot tell real from fake and gives every image 50 percent.

What is happening? Two networks are racing: one (the generator) makes fake images, the other (the discriminator) tries to catch

whether they are fake or real. At first the forger is clumsy and easily caught. Each round it learns to forge a little better, and the “fake” probability drops, like a forger racing a detective. In this illustration both the image and the percentage are fixed in advance; in real training the two networks learn together, and at the ideal equilibrium the discriminator cannot tell real from fake.

Try it yourself. 1) In which round does the verdict flip from “Fake!” to “Real?” and what is the probability in that round? 2) How is the 7 percent of round 8 found; write the rule and compute it. 3) If the detective said 50 percent for every image, what would that be a sign of?

TECHNICAL DEPTH

GANs (Goodfellow et al., 2014) train two networks in an adversarial game: the generator G produces samples from random noise; the discriminator D tries to separate real from generated samples. G is trained to maximize its chance of fooling D , while D is trained simultaneously not to be fooled.

Training is a min-max game; at equilibrium the generator’s samples become indistinguishable from the real distribution. GANs were a breakthrough for photorealistic images; today diffusion models are a common alternative (Chapter 5). In the animation of Figure 4.6 the image and the percentage are scripted, not measured training results. At the ideal GAN equilibrium, the discriminator cannot distinguish real from generated samples.

The objective of the game is $\min_G \max_D V(D, G) = E_x[\log D(x)] + E_z[\log(1 - D(G(z)))]$; at the theoretical optimum, when the generator has captured the data distribution, the optimal discriminator gives every sample $D(x) = 1/2$. An output of 1/2 on its own does not show that this optimum has been reached; an untrained discriminator can give the same value. In the animation the fake probability is $p(r) = 95 - 11r$ percent, and in round r

TECHNICAL DEPTH · CONTINUED

roughly $r/8$ of the pixels lock onto the target; both are scripted, not measured from any training.

Figure 4.6 · Generator vs Discriminator

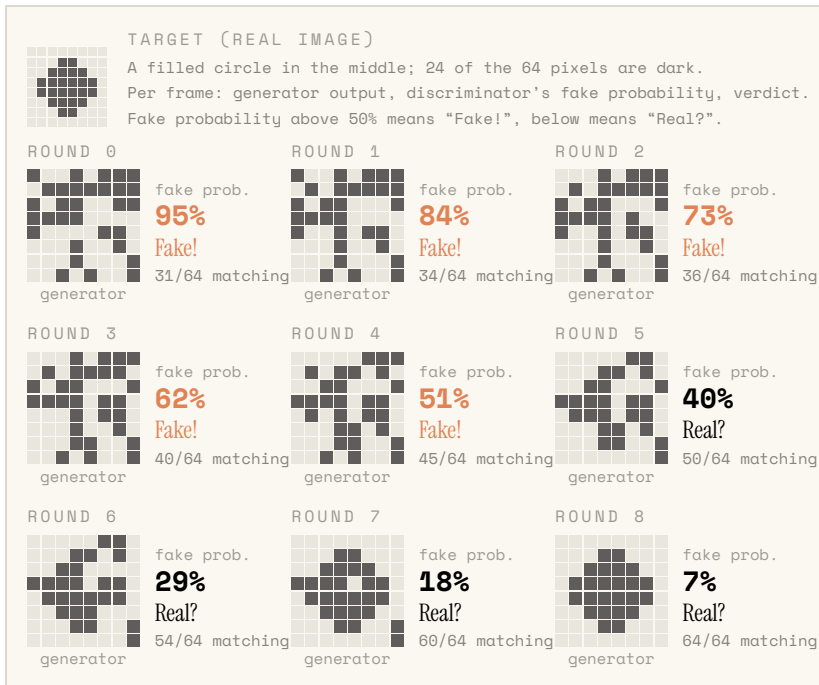


Figure 4.6 Live demo: book.onuronder.com/d/en/888b4fc273



The networks have learned both to recognize and to create. Today's systems that chat and draw take one more step, attention; that is the next chapter.

4.8 Test yourself

Answers are at the back of the book.

1. What does an artificial neuron compute?
 - a) Just the average of the inputs
 - b) A weighted sum of inputs + bias term, then an activation
 - c) A random number
 - d) It copies the inputs as-is
2. What makes a neural network “deep”?
 - a) Running very fast
 - b) Many hidden layers
 - c) Having a single neuron
 - d) Being connected to the internet
3. What if there were no activation function at all?
 - a) The network would get faster
 - b) The network would collapse into one linear function
 - c) Nothing would change
 - d) The network would get stronger
4. What does backpropagation do?
 - a) Adds new layers
 - b) Computes the loss gradient for every weight, from output back to input
 - c) Enlarges the image
 - d) Deletes the data
5. What data are CNNs especially strong at?
 - a) A single number
 - b) Passwords
 - c) Tables
 - d) Images

6. Which two networks compete in a GAN?

- a) Input and Output
- b) CNN and RNN
- c) Generator and Discriminator
- d) Teacher and Student

What to keep from this chapter

- An artificial neuron is a crude mathematical imitation of the brain; one neuron does little, and everything depends on the pattern that millions of them build together.
- A neuron weighs its inputs, sums them, adds a bias term and passes the total through an activation; with $w = [0.7, -0.5, 0.9]$ a sum of 0.69 gives 0.666 in sigmoid, and the neuron fires.
- In neurons arranged in layers, the signal flows from input to output; that is the forward pass, and the highest output counts as the network's prediction; in an untrained network that prediction means nothing.
- Without activation, a network of any number of layers would collapse into a single linear function.
- Backpropagation computes the gradient of the loss with respect to every weight, from output back to input; gradient descent updates the weights. In Figure 4.3 the error fell from 0.28 to 0.06 in eight rounds.
- Convolutional networks slide one small kernel across the whole image; the same nine numbers find an edge wherever it is.
- Recurrent networks process each word together with the memory of the ones before; in a GAN, the generator and the discriminator push each other toward the real thing.

5

CHAPTER 5

Today's AI

From tokens to language models, attention to diffusion

5.1 The generative era: from recognizing to creating

Until now, machines stayed on the “recognizing” side: Is this spam, is this a cat, what is this house worth? Like a painter’s apprentice spending years just studying canvases. Then one day the apprentice picked up the brush: Machines began to create. They write, they paint, they generate code, they hold conversations.

One of the turning points of this leap is the Transformer architecture and the idea of “attention” at its heart. Generative models (GANs, for one) and attention mechanisms were already being studied; the Transformer made it possible to train generative language models at scale. The story starts with the smallest piece: how does a sentence look to a machine? The answer lies in small chunks of text called “tokens.” From there the chapter moves to embeddings, which turn meaning into numbers, to the attention mechanism, and to how a language model writes word by word. Then comes training, the diffusion models behind image generation, and finally the limits of these systems.

Margin note. The answer to “why right now?” stands on three legs: plentiful data (the internet), powerful hardware (GPUs) and the right architecture (the Transformer). When the three came together, the generative era began.

TECHNICAL DEPTH

Generative AI marks the shift from discriminative to generative modeling. A discriminative classifier models the probability of a label y given an input x : $p(y|x)$. A generative model models the distribution of the content itself, x : $p(x)$, or $p(x|condition)$ when conditioned on an input; new samples are drawn from that

TECHNICAL DEPTH · CONTINUED

distribution. The Transformer architecture (Vaswani et al., 2017, “Attention Is All You Need”) was a major milestone in scaling generative language models; generative models (GANs, 2014, for one) and attention mechanisms were already being studied. The modern leap came from that architecture combined with the data and compute able to train it at scale.

This chapter builds the generative stack end to end: tokenization → embedding space → self-attention → autoregressive generation → the training pipeline (pretraining, fine-tuning, RLHF) → diffusion-based image generation → and the practical limits (hallucination, context window, cost). By the end, you will have an intuitive but accurate picture of why and how today's LLMs and generative models work.

The stack starts with the smallest piece. You read a sentence letter by letter, word by word; what does the machine read?

5.2 How a machine sees words: tokens

A language model never sees letters or words “as they are.” It first breaks text into tiny Lego bricks called tokens. A token is sometimes a whole word, sometimes a broken piece of one, sometimes just a comma.

Why break things up? Think of a Lego box: a limited set of bricks builds endless things. Memorizing every word in the world is impossible; but with a limited kit of pieces, any word can be built. Pick an example below and watch the machine take a sentence apart.

Margin note. The same text takes a different number of tokens in different languages. So asking an LLM something in Turkish can be more “expensive” than asking in English.

Figure 5.1 • Split a sentence into tokens

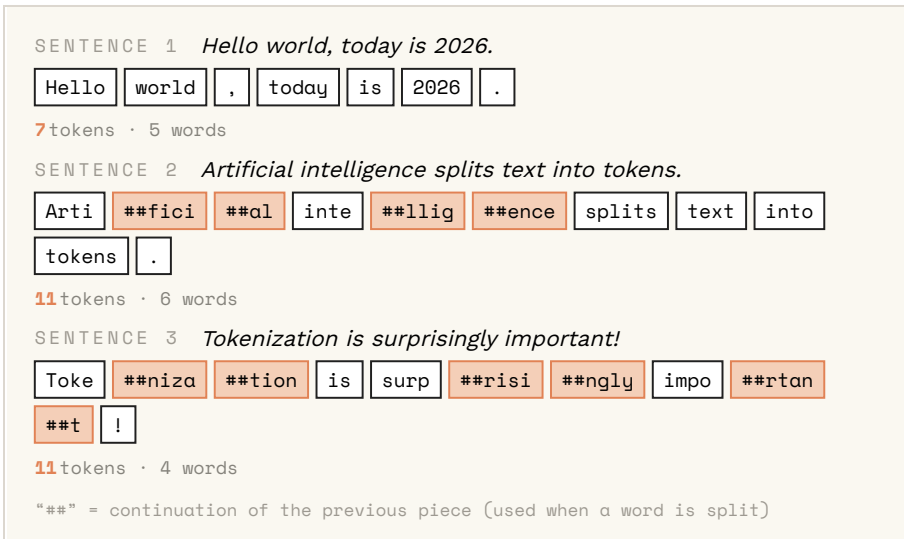


Figure 5.1 Live demo: book.onuronder.com/d/en/b2bf8192ef



Setup. The figure shows three example sentences and, under each, the same sentence taken apart into tokens. Every token sits in its own box. A plain box is a stand-alone piece; a shaded box is a continuation piece and starts with “##.” Two counts appear under each sentence: the number of tokens and the number of words. The splitter behind the figure follows one simple rule, given below; it is a toy rule, not a real tokenizer.

Step by step. The splitter's rule fits in three lines. A word of six letters or fewer stays a single token. A longer word is cut into four-character pieces; the first piece is written plain, the rest with “##.” A comma, a period or an exclamation mark is always a token of its own. Apply the rule to the three sentences:

Sentence	Tokens	Words	Tokens
Hello world, today is 2026.	Hello · world · , · today · is · 2026 · .	5	7
Artificial intelligence splits text into tokens.	Arti · ##fici · ##al · inte · ##llig · ##ence · splits · text · into · tokens · .	6	11
Tokenization is surprisingly important!	Toke · ##niza · ##tion · is · surp · ##risi · ##ngly · impo · ##rtan · ##t · !	4	11

In the first sentence every word is short: Hello, world and today have five letters each, so all of them stay whole. “2026” is a number, but to the splitter it is an ordinary four-character word. The comma after “world” and the final period each get a box of their own. Five words become seven tokens. In the second sentence “Artificial” has ten letters and breaks into three pieces, four, four and two. “intelligence” has twelve letters and breaks into three pieces of four. “splits” and “tokens” have six letters each; they sit on the limit and stay whole.

The third sentence is the striking one: four words, eleven tokens. “Tokenization” and the adverb after “is” have twelve letters each and take three pieces apiece (Toke · ##niza · ##tion; surp · ##risi · ##ngly). The word “important” has nine letters and ends in a one-letter tail (impo · ##rtan · ##t). Only “is” stays untouched.

Count tokens per word and the three sentences come out at 1.4, 1.8 and 2.75. The second and third sentences both make 11 tokens, one from six words, the other from four. The count depends less on how many words there are than on how many of them are long and rare.

A real tokenizer draws its boundaries differently; how many tokens come out depends on the tokenizer and its vocabulary. The direction is the same: Turkish suffixes make words longer, so the same text usually takes more tokens in Turkish than in English, and costs more.

What is happening? The model never sees a sentence whole; it first chops it into small pieces (tokens). This splitter keeps short words in one piece, splits long words into chunks marked “##,” and counts punctuation separately; real token boundaries depend on the tokenizer and its vocabulary. These pieces are the machine’s entire alphabet.

Try it yourself. 1) Split “Artificial intelligence learns.” with the same rule. How many tokens do you get? 2) How many pieces does the word “internationalization” break into? Write them out. 3) How many tokens per word does the third sentence carry? Compare it with the first sentence.

TECHNICAL DEPTH

Modern models use subword tokenization (approaches include BPE, WordPiece and unigram models; SentencePiece is a tool that trains them): frequent sequences become single tokens, rare words split into several pieces. Vocabularies typically hold 30K–100K+ tokens; each maps to an integer ID.

Roughly, 1 token $\approx$ 0.75 words in English; agglutinative languages like Turkish can use more tokens per word. Model context and cost are measured in tokens: both the context window and pricing depend on token count. Figure 5.1 is a simplified subword splitter (long words break into pieces marked with “##”).

The rule of Figure 5.1 (at most 6 characters for a single token, then four-character pieces) is not a stand-in for a real tokenizer vocabulary. In a real vocabulary the boundaries follow frequency statistics: a frequent word usually stays a single token, a rare

TECHNICAL DEPTH · CONTINUED

one splits into a few pieces. An exact token count for “Hello” or “Tokenization” cannot be given without fixing the tokenizer and its version and running it. The rule is simple, but the result points the same way: rare and long words take more pieces.

Every token now carries an ID number, and an ID number says nothing about “cat” being close to “dog.” Where does meaning come from?

5.3 Turning meaning into numbers: embeddings

Tokens enter the machine as numbers, but a dry ID number carries no “meaning.” Enter the embedding: Every word is given an address in a vast city. That address is a list of numbers carrying the word’s meaning.

In this city, words with similar meanings move into the same neighborhood. “Cat” and “dog” are next-door neighbors; so are “king” and “queen.” Pick a word in Figure 5.2 and meet its neighbors.

Margin note. Embeddings aren’t just for words: sentences, images and sounds can live in the same space. This is the foundation of multimodal models and semantic search.

Setup. The figure places nine words on a two-dimensional map. Three dashed circles mark three families of meaning: animals (cat, dog, bird), royalty (king, queen, prince) and food (apple, bread, cheese). Each point carries its word; the coordinates are in the table below. The points were placed by hand to show the idea of closeness; they are not an embedding from a trained model or the result

of PCA or t-SNE. On this map, points that sit close together count as close in meaning. Distance is measured with a plain ruler.

Step by step. The positions on the map and the two nearest neighbors of every word:

Word	Family	x	y	2 nearest neighbors (distance)
cat	animal	58	62	dog (28.6), bird (30.6)
dog	animal	84	50	cat (28.6), bird (52.8)
bird	animal	52	92	cat (30.6), dog (52.8)
king	royalty	198	54	queen (28.8), prince (34.1)
queen	royalty	222	70	king (28.8), prince (31.6)
prince	royalty	196	88	queen (31.6), king (34.1)
apple	food	128	132	cheese (17.2), bread (26.8)
bread	food	152	120	apple (26.8), cheese (42.8)
cheese	food	118	146	apple (17.2), bread (42.8)

The distance comes from Pythagoras: take the horizontal and vertical differences between two points, square them, add them, take the square root. For “cat” and “dog” the differences are 26 and 12; their squares are 676 and 144; the sum is 820; the square root is 28.6. Do this for all nine words and every word’s two nearest neighbors come from its own family, without exception. The figure’s own caption for “cat” reads: “The 2 nearest neighbors: dog (28.6), bird (30.6); both from the ‘animal’ family.”

Between families the distances grow. From “cat” to “apple” is 99 units; from “cat” to “queen” is 164. The map tells you who is a neighbor, who is a stranger, and by how much. There is a border zone as well: the third-nearest word to “prince” is “bread” (54.4), so the royalty district and the food district touch. What the machine calls “meaning” is nothing more than this geometry. One limit: real emb-

eddings have hundreds of dimensions; two points that look nearest on a two-dimensional map need not be the nearest pair in the real space.

What is happening? Every word becomes a point on a map. Words close in meaning sit close on the map too; here the 2 nearest points stand for the 2 most similar words. The points on this map were placed by hand to show the idea; a real model learns its placement from which words appear in similar sentences.

Figure 5.2 · The meaning map

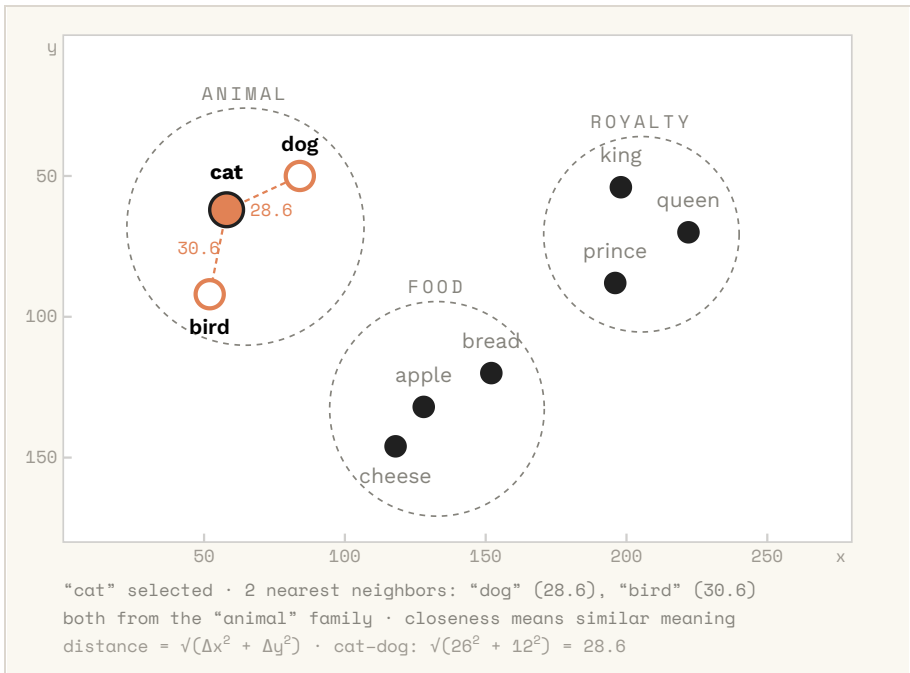


Figure 5.2 Live demo: book.onuronder.com/d/en/ab3b439524



Try it yourself. 1) Compute the distance between “bird” and “cheese.” Compare it with the largest within-family distance in the table. 2) Suppose you want to add the word “lion” to the map. Which region would you put it in? Propose a sensible x and y pair and find its two nearest neighbors. 3) Which word is third-nearest to “bread,” and from which family? What does that tell you about the map?

TECHNICAL DEPTH

An embedding maps a token to a dense vector (typically hundreds to thousands of dimensions). These vectors are learned in training; semantic and syntactic relations are reflected in the geometry. Similarity is usually measured with cosine similarity.

The famous example: with vector arithmetic, analogies like king – man + woman $\approx$ queen can emerge. The points in Figure 5.2 were placed by hand to illustrate similarity; they are not embeddings extracted from a trained model or the result of PCA or t-SNE. Real embeddings have far more dimensions, and the two points that look nearest in a two-dimensional projection need not be the most similar pair in the real space.

The distance in Figure 5.2 is the Euclidean distance: $d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$. Real embeddings are more often compared with cosine similarity: $\cos \theta = (a \cdot b) / (\|a\| \cdot \|b\|)$. This measure compares the direction of the vectors, not their length; a value near 1 means the same direction, 0 means perpendicular (unrelated), -1 means opposite directions.

Every word has an address. Inside a sentence, though, how does a word choose which neighbor to look at?

5.4 Attention: the heart of the Transformer

Read this sentence: “The cat ran away because it was scared.” Who is “it”? Without even noticing, you glanced back at “the cat.” The attention mechanism teaches a machine that very glance: Each word learns which other words to “look at” for its meaning.

Pick a word in Figure 5.3; see how much it “attends” to the others by the depth of the color. The darker the color, the stronger the tie.

Margin note. “Attention Is All You Need” (2017): the attention mechanism removed the RNN’s one-at-a-time processing constraint. Processing every position together in training boosted both speed and understanding; generation still proceeds token by token.

Setup. The figure lays the sentence out as a heat map of five units: “The cat,” “ran,” “because,” “it” and “was scared.” The figure joins “The” to “cat” and “was” to “scared,” and leaves out “away,” so five units stand for the sentence. Rows are the unit doing the looking (the query); columns are the unit being looked at. Each cell holds an attention weight, and the numbers in a row add up to 1.00. The darker the cell, the larger the weight. The darkest cell of every row is printed in bold. The table is an illustrative bidirectional (encoder-style) self-attention example: the weights were chosen by hand, and every unit may look at the units after it as well.

Step by step. The weight table:

Query ↓ · Attends to →	The cat	ran	because	it	was scared
The cat	0.50	0.30	0.05	0.10	0.05
ran	0.50	0.30	0.10	0.05	0.05

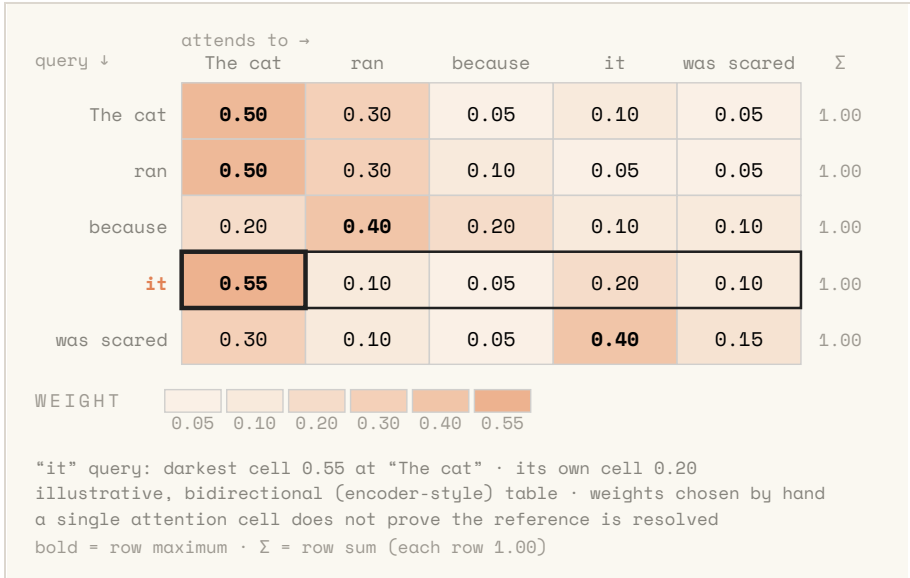
Query ↓ · Attends to →	The cat	ran	because	it	was scared
because	0.20	0.40	0.20	0.10	0.10
it	0.55	0.10	0.05	0.20	0.10
was scared	0.30	0.10	0.05	0.40	0.15

Read the rows one at a time:

1. “The cat”: the highest weight is on itself (0.50). After itself, it looks most at “ran” (0.30); the subject is looking for its verb.
2. “ran”: half of the weight goes to “The cat.” The verb wants to know who ran.
3. “because”: it looks most at “ran” (0.40). The conjunction holds on to the event it explains.
4. “it”: the row the section is about. The darkest cell is 0.55, at “The cat”; the share it keeps for itself is only 0.20. In this illustrative table the pronoun looks mostly at the cat; in a real model a single attention weight does not by itself prove that the pronoun’s reference has been resolved.
5. “was scared”: it looks most at “it” (0.40), with “The cat” in second place (0.30). The predicate finds its subject in two hops: first the pronoun, then the cat the pronoun points to.

Two more things. The table is not symmetric: “it” looks at the cat with 0.55, but “The cat” looks at “it” with only 0.10; looking runs one way. And distance does not matter: two units stand between “it” and “The cat,” yet the weight is the highest in the whole table. That is where the older models, which read one word after another, ran into trouble.

One more distinction: the “The cat” row gives 0.30 to “ran,” which comes after it. A text-generating (autoregressive) model has no such glance; each unit sees only the units before it, and the later ones are masked. That is why this table is a bidirectional, encoder-style example.

Figure 5.3 · Which word looks at which?Figure 5.3 Live demo: book.onuronder.com/d/en/20e3333ccc

What is happening? To make sense of a sentence, each word decides which others it should “pay attention” to. The darker the color, the stronger the bond. In this illustrative example “it” looks mostly at the cat; the weights were chosen by hand, and by themselves they do not prove that the pronoun has been resolved.

Try it yourself. 1) Check the sum of every row; is each one 1.00? 2) If the sentence were “The cat ran because the dog was scared,” where would the darkest cell of the “was scared” row move? Write down why. 3) Why are the weights in the “because” column so low? Can these low weights settle how much meaning a conjunction carries?

TECHNICAL DEPTH

Self-attention produces query (Q), key (K) and value (V) vectors for every token; weights are computed with $\text{softmax}(Q \cdot K^T / \sqrt{d_k})$, where d_k is the dimension of the key vectors, and the output is the weighted sum of the values. Unmasked self-attention lets every position reach the whole sequence regardless of distance; an autoregressive (causal) decoder masks future tokens, so each position sees only the positions before it. Training can process many positions together, while autoregressive generation adds tokens one at a time.

The Transformer does this with multiple heads: different “heads” capture different kinds of relation (syntax, coreference, and so on). Positional encoding adds order information. The cost is $O(n^2)$ in sequence length; that is the main reason context windows are limited.

Every row of Figure 5.3 is a softmax output: the weights lie between 0 and 1 and sum to 1. The output at the position of “it” is the weighted sum of the value vectors: $0.55 \cdot V(\text{The cat}) + 0.10 \cdot V(\text{ran}) + 0.05 \cdot V(\text{because}) + 0.20 \cdot V(\text{it}) + 0.10 \cdot V(\text{was scared})$. The new representation of “it” is therefore a blend, more than half of which comes from “The cat.” The asymmetry of the table comes from the same place: each row is computed with its own query, and $Q \cdot K^T$ is not a symmetric matrix. The table is bidirectional: the “The cat” row gives 0.30 to “ran,” which comes after it. In an autoregressive model that cell is masked ($-\infty$ is added before the softmax, so the weight becomes 0) and the remaining weights of the row are renormalized to 1.

Every word knows whom to look at. How does the next word, not yet written, grow out of these glances?

5.5 A language model: predict the next word

You know the game: someone says “The early bird catches the...” and you shout “worm!” Odd as it sounds, that is the job of the language models in this chapter: look at the text so far and guess the most likely next word (token). It adds the guess to the text and guesses again. Playing this tiny game thousands of times, it writes whole paragraphs.

Follow the steps of Figure 5.4: which words the model weighs at each step, with what probability, and how the sentence takes shape. Compare the two selection rules too: should it always pick the most likely word (greedy), or roll dice by the probabilities (sampling)?

Margin note. “Does it understand, or does it just predict?” In a way, both: it couldn’t predict this coherently without capturing some meaning. But at its core, all it does is guess the next token.

Setup. The figure is a film strip of three generation steps, laid out in two rows. The starting text is the single word “AI”. In each frame the model shows four candidate words and the probability it gives each one; the length of the bar is proportional to the probability. The top row of the strip is “Greedy (argmax)”: the most likely candidate at every step. The bottom row is “Sampling, $T = 1.5$ ”: the probabilities are softened by the temperature and a fixed random number rolls the dice. The two rows build two different sentences from the same candidates.

Step by step. The candidate lists of the three steps:

Step	Candidates and probabilities
1	now 42% · already 28% · today 18% · rapidly 12%
2	learns 38% · writes 30% · creates 20% · reasons 12%
3	fast 50% · well 25% · daily 15% · deeply 10%

In the greedy row the model takes the most likely candidate at every step:

1. “now” (42%) → “AI now”
2. “learns” (38%) → “AI now learns”
3. “fast” (50%) → “AI now learns fast.”

The sampling row does two things. First it softens the probabilities with the temperature $T = 1.5$: large shares shrink, small ones grow. Then it draws a random number U between 0 and 1 and adds up the candidates in order; the moment the running total passes U , that candidate is chosen. So that the result is the same in every printing, the figure uses a fixed U sequence: 0.37, 0.81, 0.12. The numbers are rounded to two places.

Step	Softened probabilities ($T = 1.5$)	Running total	U	Chosen
1	now 0.36 · already 0.28 · today 0.21 · rapidly 0.16	0.36 · 0.64 · 0.84 · 1.00	0.37	already
2	learns 0.34 · writes 0.29 · creates 0.22 · reasons 0.16	0.34 · 0.63 · 0.85 · 1.00	0.81	creates
3	fast 0.41 · well 0.26 · daily 0.19 · deeply 0.14	0.41 · 0.67 · 0.86 · 1.00	0.12	fast

1. “already” ($U = 0.37$) → “AI already”: the share of “now” has dropped to 0.36; 0.37 passes that threshold by a hair, and the choice falls to the second candidate.

2. “creates” ($U = 0.81$) → “AI already creates”: the first two candidates reach 0.63 together; 0.81 lands in the third candidate’s slice.
3. “fast” ($U = 0.12$) → “AI already creates fast.”: a small U picks the most likely candidate. Sampling picks the top candidate most of the time too, only not always.

Figure 5.4 · Generate word by word

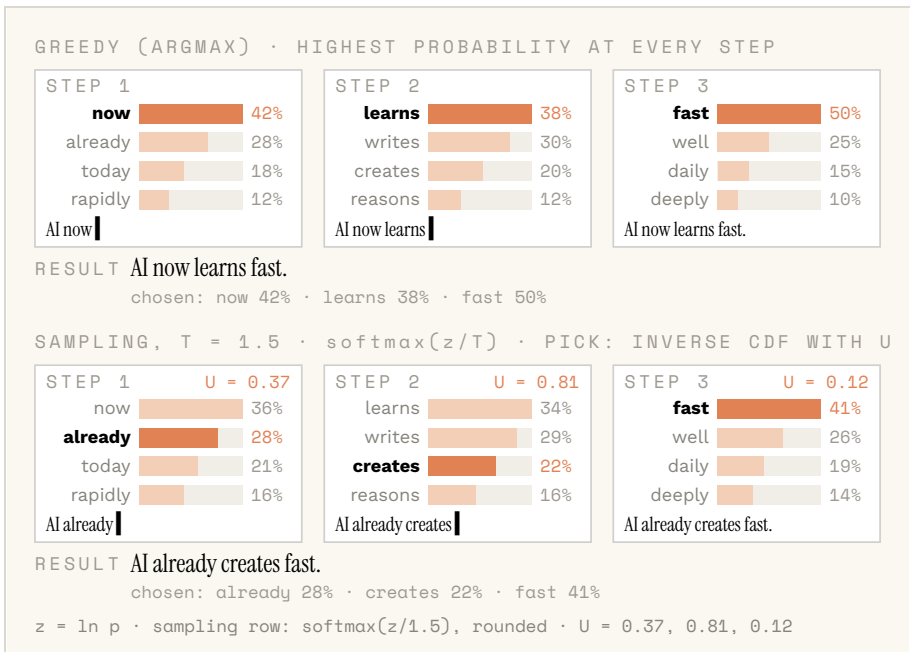


Figure 5.4 Live demo: book.onuronder.com/d/en/25369cb8c7



Both sentences are grammatical; the second takes a less expected road. Greedy decoding rolls no dice; the result is the same every time. Sampling does: change the U sequence and the sentence changes, so from the same start a different sentence can come out every time. Temperature sets the weighting of the dice: as T rises the

shares move closer together, as T falls the top candidate grows. But as long as T is above zero the dice are rolled; low temperature is not the same thing as greedy decoding.

The probabilities at each step are spread over four candidates and add up to a hundred percent. In reality the model spreads this distribution over tens of thousands of tokens; the four candidates are only the top of the list. And each choice changes the next step's question. After "now," "learns" is likely; after "already" it is likely too. But in a real model the list of step 2 is recomputed according to the choice made in step 1.

What is happening? At each step the model asks "what is the most likely word after the text so far?" It picks one, adds it to the sentence and starts over. Greedy decoding always takes the top word; steady but predictable. Sampling rolls dice by the probabilities. The higher the "creativity" (temperature), the more even the dice and the more varied the picks. The lower it is, the more the top word stands out, but the dice are still rolled.

Try it yourself. 1) If the second U number in the sampling row were 0.50 instead of 0.81, which candidate would be chosen at step 2? What would the sentence be? 2) Multiply the probabilities of the greedy sentence: $0.42 \times 0.38 \times 0.50$. Do the same for the sampled sentence with the model's original probabilities (already 28%, creates 20%, fast 50%); which one is more likely, and by how many times? 3) In which of the three steps is the model least sure? Look at the share of the top candidate.

TECHNICAL DEPTH

Here we focus on autoregressive generative language models; most of today's chat models belong to this class, but not every language model is autoregressive. For the next token the model produces a score (logit) for every entry in the vocabulary; softmax converts those scores into a probability distribution that

TECHNICAL DEPTH · CONTINUED

sums to 1, $P(\text{token}_t \mid \text{token}_1 \dots \text{token}_{t-1})$, and one token is picked and appended. Greedy decoding (argmax) picks the most likely; sampling draws at random from the distribution.

Temperature sharpens or flattens the distribution: low temperature gives steadier, more repetitive output; high temperature gives more varied, riskier output (together with top-k / top-p). As long as T is above zero, sampling is random; greedy (argmax) decoding is a separate rule, not low-temperature sampling. The probabilities in Figure 5.4 are illustrative; a real model produces a distribution over its whole vocabulary.

The temperature T divides the model's scores (logits) before the softmax: $p_i = \exp(z_i/T) / \sum_j \exp(z_j/T)$. As T shrinks, the distribution collapses onto a single candidate and sampling approaches argmax, but it stays random as long as $T > 0$; argmax is a separate selection rule. As T grows, the distribution flattens, and even the 12-percent “rapidly” gains a reasonable chance. The sampling row of Figure 5.4 applies this transformation with $T = 1.5$: $z = \ln p$ is taken from the candidate probabilities, $\text{softmax}(z/1.5)$ is computed, and a fixed U sequence (0.37, 0.81, 0.12) selects by the inverse cumulative distribution; at step 1 the share of “now” drops from 0.42 to 0.36 and that of “rapidly” rises from 0.12 to 0.16. The probability of a sentence is the product of its steps: $P(\text{now}, \text{learns}, \text{fast}) = 0.42 \times 0.38 \times 0.50 \approx 0.08$.

The model knows how to spread probability at every step. Where did it learn these probabilities? Usually from a training pipeline of three stages.

5.6 How a model is raised: the training pipeline

A chat assistant grows up like a child; a common path passes through three schools. First comes “pretraining” on massive text: there it learns language and the world. Then, at the “fine-tuning” school, it learns to answer questions and follow instructions. Last comes “alignment with human feedback”: there it learns to be helpful, honest and safe; its manners, you could say.

Read the three stages of Figure 5.5 one by one and see how the model answers the same question differently after each.

Margin note. Roughly: pretraining gives the model “what it knows”; fine-tuning and alignment give it “how to behave,” though fine-tuning can change knowledge and task performance too. The same fact can be delivered in very different tones.

Setup. The figure is a table with three columns; each column is one training stage. Every stage has four rows: the data the model sees, what it learns, its sample answer to the same question about a capital city, and a short note. The question is the same in all three columns, “capital of Türkiye?”; only the shape of the answer changes.

Step by step. Put the three stages side by side:

	1 · Pretraining	2 · Fine-tuning	3 · RLHF / alignment
Data	Massive internet text	Instruction–response pairs	Human preferences (preference pairs)

	1 · Pretraining	2 · Fine-tuning	3 · RLHF / alignment
What it learns	Language and the world (next-word prediction)	Following instructions (answering the question)	Being helpful, honest and safe
Sample output	“The capital of Türkiye is Ankara, with a population of about six million. The city...”	“The capital of Türkiye is Ankara.”	“The capital of Türkiye is Ankara. If you’d like, I can also share a few interesting facts about the city.”
Note	The raw model is a “completer”: it doesn’t answer the question, it continues the text.	Now it answers directly and concisely.	Same fact; a more helpful, polite, aligned tone.

1. Pretraining: the answer starts with the right fact but does not stop. It adds the population and goes on with “The city...” The model does not know it has been asked a question; it writes as if continuing an encyclopedia page from the internet. The knowledge is there; the manners are not.
2. Fine-tuning: the same fact, one sentence. The model has now learned the pattern “a question came, answer it, stop” from thousands of instruction–response pairs.
3. RLHF / alignment: the same answer again, with an offer on top: would you like more? People were shown two answers and asked “which is better?”; the model was adjusted toward the preferred tone.

Figure 5.5 • An assistant in three stages

	1 • Pretraining	2 • Fine-tuning	3 • RLHF / alignment
DATA	Massive internet text	Instruction–response pairs	Human preferences (preference pairs)
LEARNS	Language and the world (next-word prediction)	Following instructions (answering the question)	Being helpful, honest and safe
OUTPUT for “capital of Türkiye?”	The capital of Türkiye is Ankara, with a population of about six million. The city...	The capital of Türkiye is Ankara.	The capital of Türkiye is Ankara. If you’d like, I can also share a few interesting facts about the city.
NOTE	The raw model is a “completer”: it doesn’t answer the question, it continues the text.	Now it answers directly and concisely.	Same fact; a more helpful, polite, aligned tone.

→
same question, three stages: same fact, different delivery

Figure 5.5 Live demo: book.onuronder.com/d/en/bf611298ac



In these three examples only one thing stays the same: the fact, Ankara. What changes is the shape of the answer to the same fact; real fine-tuning can change knowledge and task performance as well. The order is a common recipe, not a fixed rule: some models merge or skip stages. The third stage has more than one route too: PPO-based RLHF uses a separate reward model, while DPO adjusts the model directly from preference pairs.

What is happening? An assistant grows up in three stages along a common path. First, in “pretraining,” it learns language and the world from massive text. Then in “fine-tuning” it learns from example Q&A pairs how to answer. Finally, with human feedback, it learns to be helpful and polite. Same fact, three very different manners; not every model goes through the same stages.

Try it yourself. 1) If the question were “At what temperature does water boil?”, predict the stage-one model’s output; then write the stage-two output. 2) In the third stage, people are shown two answers, one “wrong but polite” and one “right but rude.” Which should be preferred? How do the three goals of alignment (helpful, honest, safe) decide? 3) Which row of the table shows most clearly that the fact stays the same while the shape of the answer changes? Do these three examples prove that fine-tuning cannot change knowledge?

TECHNICAL DEPTH

1. Pretraining: on a large corpus, the model learns the statistics of language through a self-supervised next-token objective. 2) Supervised fine-tuning (SFT): it learns instruction-following from instruction–response pairs. 3) Preference alignment: PPO-based RLHF trains a separate reward model on human preferences and updates the policy against that signal; DPO builds a policy loss directly from preference pairs and needs no separate reward model. These three stages are a common workflow; not all models follow the same stages in the same order.

The result: a shift from a raw “text completer” to a helpful, honest and reasonably safe assistant. Alignment isn’t perfect; open problems like jailbreaks, reward hacking and distribution shift remain.

The three stages have different objectives. In pretraining the loss rests on the probability of the next token: $L = -\sum_t \log P(\text{token}_t \mid \text{token}_1 \dots \text{token}_{t-1})$. The first column of Figure 5.5 is the natural output of this objective; continuing the text is the behavior that lowers this loss. SFT applies the same loss, but only on the response part and only on selected pairs. In the alignment stage the objective is human preference. In PPO-based

TECHNICAL DEPTH · CONTINUED

RLHF a reward model estimates which of two answers is preferred, and the policy is updated in the direction that raises that estimate. In DPO there is no separate reward model: a loss that raises the probability of the preferred answer relative to the rejected one is applied to the policy directly.

Text models are raised that way. Image models start somewhere else entirely: pure noise.

5.7 Making images: diffusion

Most of today's image-making models rest on a different idea altogether: diffusion; other routes, such as GANs, exist too. Picture a fogged-up window; the scene behind it is a blur. Now wipe the glass slowly: the picture emerges stroke by stroke. A diffusion model learns that wiping; only it starts from a screen of pure static (noise) and cleans a little at each step until an image appears.

Follow the frames of Figure 5.6 from left to right: the further right you go, the more noise the model clears, and the image slowly appears.

Margin note. Diffusion removes noise a little at a time, step by step, until the shape emerges. It also trains more stably than GANs.

Setup. The figure is a film strip of nine frames. Each frame is a tiny canvas of 8×8 , that is, 64 pixels. The first frame on the left is pure noise: dark and light gray pixels scattered at random. The last frame on the right holds a heart of 40 orange pixels (purple on screen); the other 24 pixels are light. In the seven frames between, the model resolves a few more pixels at every step. Under each frame two

numbers appear: the progress of the strip (step/8, as a percentage) and the resolved pixel count (6/64, for example). They are different quantities; 13 percent progress is not the same thing as 6 of the 64 pixels (9 percent) being resolved.

Step by step. Each frame is one step:

Step	Progress (step/8)	Resolved pixels (of 64)	Visible heart pixels	What the frame shows
0	0%	0	0	Noise only; a screen full of static
1	13%	6	4	A few orange dots, no shape yet
2	25%	12	9	Scattered dots
3	38%	21	12	The body starts to show
4	50%	29	17	The heart can be guessed
5	63%	40	24	The shape is certain, the edges rough
6	75%	46	29	A few specks of noise
7	88%	55	36	The last specks
8	100%	64	40	A clean heart: 40 orange, 24 light pixels

The resolved-pixel count does not grow evenly: 6, 12, 21, 29, 40... The illustration decides which pixel opens at which step with a fixed random seed, so the strip is the same every time, but the steps are not equal. And the shape is recognizable halfway through. At step 4, the midpoint of the strip, 29 of the 64 pixels are resolved, 17 of them

heart pixels. The heart is already unmistakable; the remaining steps fill in the detail.

Figure 5.6 • From noise to image

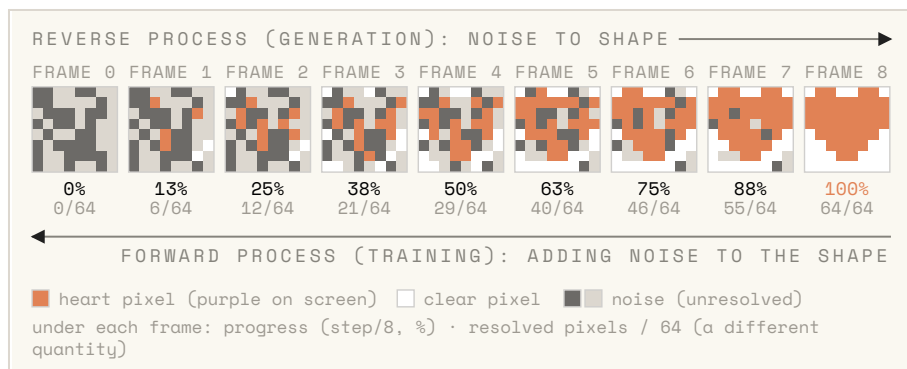


Figure 5.6 Live demo: book.onuronder.com/d/en/fd88692784



One warning: a real diffusion model does not “open” pixels, and it does not uncover a hidden picture. At every step it reduces the noise a little across the whole image; all the pixels sharpen at the same time, slowly, and a new sample emerges from learned transformations. The number of steps is not 8 either, but dozens or hundreds. This strip uses a fixed pattern to carry the idea: the noise shrinks, the shape emerges.

What is happening? Diffusion is like cleaning up a snowy TV screen (pure noise) step by step until a picture emerges from inside it. The model learned one skill: “estimate the noise in this image and remove a little of it.” Done over and over, noise turns into a picture. This animation uses a fixed pattern to show the transition; a real model does not uncover a hidden picture, it generates a new sample.

Try it yourself. 1) In the last frame 40 of the 64 pixels are orange. If 29 pixels are resolved at step 4 and pixels open at random, about how many of them would you expect to be orange? 2) If there were 16 steps instead of 8, what would the progress share be at each step? 3) In which direction of the strip is the “forward process” described in Technical depth read, and in which direction the “reverse process”?

TECHNICAL DEPTH

Diffusion has two processes. The forward process gradually adds Gaussian noise to an image (fixed, not learned). The reverse process is a neural network that estimates and removes the noise added at each step (denoising); this walks from noise back to the data distribution.

In text-to-image generation, the process is conditioned on a text embedding (usually CLIP-like) and, for speed, run in latent space (latent diffusion). Figure 5.6 is a qualitative illustration; real models use a learned noise-prediction network.

In the forward process a single step corrupts the image as $x_t = \sqrt{1 - \beta_t} \cdot x_{t-1} + \sqrt{\beta_t} \cdot \epsilon_t$, where ϵ_t is that step's standard Gaussian noise and β_t a small noise share. Composing the steps, x_t is sampled directly from x_0 : $x_t = \sqrt{\bar{\alpha}_t} \cdot x_0 + \sqrt{1 - \bar{\alpha}_t} \cdot \epsilon$, $\epsilon \sim N(0, I)$, where $\bar{\alpha}_t = (1 - \beta_1)(1 - \beta_2) \dots (1 - \beta_t)$ and ϵ is the noise accumulated up to step t as one sample, not the noise of a single step. Given x_t and t , the network learns to estimate this accumulated ϵ ; the training loss is $\|\epsilon - \epsilon_\theta(x_t, t)\|^2$, and the ϵ in the loss is the same accumulated noise as above. Generation runs the other way: start from pure noise x_T , subtract a share of the estimated noise at every step, arrive at x_0 . The “resolved pixels” counter of Figure 5.6 is not a stand-in for this computation but a visual analogy

TECHNICAL DEPTH · CONTINUED

only; the percentage under the frames is the progress of the strip, not the share of noise removed.

Text and images can both be generated. Now for the limits: where do these models go wrong, and where do they get stuck?

5.8 Limits: hallucination, context and cost

These models impress, but they aren't magic. Their most famous flaw is hallucination: without missing a beat, the model can tell you something that sounds right and is flat wrong. A fluent or high-probability answer is no guarantee of truth; wrong content can be produced without any dice being rolled. The other limit is the context window: the model carries a small notebook and can hold only so many tokens at once. What happens when the notebook fills is up to the system in use: it may raise an error, trim the text or summarize it.

Try the context window yourself below: this illustration is a sliding window that keeps the last eight words. As you add words the window fills, and the oldest words drop out. Real applications may raise an error, trim the text or summarize it when they reach the limit.

***Margin note.** The golden rule: treat an LLM not as an “all-knowing oracle” but as a “very fluent intern who is sometimes wrong.” Always verify the facts that matter.*

Figure 5.7 · The context windowFigure 5.7 Live demo: book.onuronder.com/d/en/da998d46f8

Setup. The figure shows a twelve-word sentence passing through an eight-word window. The sentence: “Language models hold text inside a limited window and forget older words.” The window holds at most 8 words; this sliding window is a rule of the figure, and the real limit is measured in tokens. The seven frames of the strip are the sentence at 0, 1, 4, 8, 9, 10 and 12 words. The orange line marks the window the model sees: the last eight words. Words with a dark border are inside the window; faded words have fallen out of it, that is, been forgotten.

Step by step. The words arrive one at a time:

Word added	In the window	Forgotten
0	(empty)	
1	Language	
4	Language models hold text	
8	Language models hold text inside a limited window	(the window has just filled)
9	models hold text inside a limited window and	Language
10	hold text inside a limited window and forget	Language models
12	inside a limited window and forget older words	Language models hold text

1. From zero to eight: the window starts empty and holds at most 8 words. Every new word finds a place; at the eighth word all 8 are inside the window. Everything is remembered until it fills.
2. The ninth word: the window is full; when “and” comes in, “Language” drops out and fades. The figure’s caption marks the moment: “The window is full!” From here on the model sees only the last 8 words.
3. The twelfth word: the first four words are gone. The text the model sees is “inside a limited window and forget older words.” The subject of the sentence, “Language models,” is no longer in the window. The model does not know what it has forgotten.

The real scale is much larger: this window holds 8 words; real models hold thousands of tokens, some of them hundreds of thousands. The idea is the same: there is a limit. What happens when it fills is up to the application; one system raises an error, another trims the oldest text, another summarizes. That is usually why an assistant

“forgets” what it said at the start of a long conversation. Being inside the window is no guarantee of full recall either; information in the middle of a long context can be overlooked. This also connects to hallucination: in place of the missing information, the model can produce a fluent continuation. The gap gets filled, but not necessarily with the truth.

What is happening? The model has a “short-term memory” and can hold only so many tokens at once. In this illustration, as you add words the window fills; once it is full, the oldest slip out one by one and the model no longer sees them. Real systems may raise an error, trim the text or summarize it at the limit; that is why long documents get trimmed or summarized.

Try it yourself. 1) If the window held 5 words instead of 8, how many words would be forgotten when the twelfth word is added? Write down the words that remain. 2) How many tokens does this twelve-word sentence make with the splitter of Figure 5.1? If the window counted tokens instead of words, at which word would it fill? 3) Suppose you wanted to give a twenty-page document to a model with this window. Which of the three options in the section, raising an error, truncating or summarizing, would you choose, and why?

TECHNICAL DEPTH

Hallucination has no single cause: there is no guarantee between the training objective (producing a plausible continuation) and the truth of a statement. A fluent or high-probability answer can be wrong, and that happens without random sampling, in greedy decoding too. Mitigations: grounding in sources (RAG), tool use and verification, and better alignment (Chapter 6); claims that matter are checked against sources and task-level verification. The context window is a fixed token limit; since attention costs $O(n^2)$, growing the window is expensive.

TECHNICAL DEPTH · CONTINUED

Other limits: the knowledge cutoff, bias (inherited from training data), instability/non-reproducibility (sampling), and compute/energy cost. Knowing these limits is the precondition for using these tools responsibly and effectively.

The context window is a fixed token limit; this illustration is a sliding window that keeps the last N tokens, and a real application may raise an error, truncate or summarize when the limit is reached. Because attention costs $O(n^2)$, enlarging the window is expensive; that is why long documents get truncated or summarized.

The window of Figure 5.7 is a sliding queue: after n words have been added, the number forgotten is $\max(0, n - N)$, with $N = 8$. In real models the unit is not the word but the token. With the rule of Figure 5.1 the same sentence makes 15 tokens, so a token window fills one word earlier and starts forgetting two words earlier than a word window. The $O(n^2)$ cost comes from here: the heat map of Figure 5.3 has $n \times n$ cells for n tokens. When the window grows from 8 to 16, the cell count grows from 64 to 256, four times as many. This queue is a rule of the figure; in a real application the overflow policy (error, truncation, summarization) is the system's choice.

The chapter's stack is complete: token, embedding, attention, generation, training, diffusion and limits. Eight questions follow.

5.9 Test yourself

Answers are at the back of the book.

1. What is a token?
 - a) A kind of neural network
 - b) A model weight
 - c) A GPU core
 - d) A small piece of text the model processes
2. What is true of the embedding space?
 - a) Words are placed randomly
 - b) Only numbers are stored, no meaning
 - c) Semantically similar words land near each other
 - d) Every word is at the same point
3. What does the attention mechanism provide?
 - a) Enlarging images
 - b) Slowing the model
 - c) Each word “looking at” the others with weights
 - d) Deleting data
4. What does an autoregressive LLM, the kind in this chapter, fundamentally do?
 - a) Predicts the next token
 - b) Applies hand-written rules
 - c) Queries a database
 - d) Searches the internet

5. The order of the common training pipeline described in the chapter?
- a) Pretraining → fine-tuning → RLHF
 - b) Pretraining only
 - c) Fine-tuning → pretraining → RLHF
 - d) RLHF → pretraining → fine-tuning
6. How does a diffusion model create an image?
- a) Leaving pixels random
 - b) Downloading it from the internet
 - c) Copying it in one shot
 - d) Starting from noise and cleaning it step by step
7. What is hallucination?
- a) The model producing confident but wrong information
 - b) Generating an image
 - c) The model crashing
 - d) Running faster
8. What does the context window limit?
- a) How many tokens the model attends to at once
 - b) Disk size
 - c) Screen resolution
 - d) Internet speed

What to keep from this chapter

- A language model reads text as small pieces called tokens, not as letters or words; long and rare words take more tokens.
- An embedding turns every token into an address on a map; words close in meaning sit side by side on that map.
- With attention, every word spreads weights over the others in the sentence; in this example the pronoun “it” looks mostly at the cat.

- A language model predicts the next token at every step; greedy decoding always takes the most likely one, sampling rolls dice by the probabilities, and temperature sets how even the dice are.
- An assistant is usually raised in three stages: pretraining gives the knowledge, fine-tuning the instruction-following, alignment the helpful and safe tone.
- A diffusion model starts from pure noise and brings the image out by cleaning a little at every step.
- Hallucination and the context window are structural limits of these models; always verify the facts that matter.

6

CHAPTER 6

Using and Building AI

From prompts to agents, architecture to the real world

6.1 Using and building AI

Picture a master who knows everything but owns no workshop: no tools in hand, no notebook allowed, yesterday's conversation already forgotten. A language model on its own is like that. What turns it into a useful assistant is the workshop built around it: good prompts, feeding it your own data (RAG), tool use, agents and a solid architecture. This chapter builds that workshop piece by piece, and ends with what these tools are changing in the real world.

Margin note. Today the competitive edge is rarely “owning the biggest model”; it is “using the model best with your own data and tools.”

TECHNICAL DEPTH

Earlier chapters built the core capabilities; this one covers the application layer: the design patterns that turn a foundation model into a reliable, context-aware product that can take action.

The scope: prompt engineering (role, context, few-shot, format), grounding with knowledge (retrieval-augmented generation), tool/function calling and agent loops (ReAct), the component architecture of a typical AI application (orchestration, knowledge base, tools, memory) and industry examples. The main idea: without changing the model, the system around it can raise reliability and usefulness.

The first tool in the workshop is also the cheapest: asking the right question. The same model gives different answers depending on how you build the prompt.

6.2 Prompt engineering: asking the right question

Tell a tailor “make me something” and even the tailor can’t say what will come out. Give your measurements, pick the fabric, describe the cut, and everything changes. A language model is the same: there is a world of difference between “make me a plan” and giving it a role, your situation and the shape you want.

Build a prompt piece by piece: as you add role, context, example and format, watch the model’s answer sharpen.

***Margin note.** A good prompt = a clear role + enough context + a definite format + (if needed) an example. Before blaming the model, review your prompt; some “bad answers” are “incomplete questions.”*

Setup. The figure builds one prompt out of four pieces. At the bottom sits the base request, always the same: “Suggest a weekend trip plan for me.” Four pieces can be stacked on top of it: role, context, example and format. On the right is a completeness indicator, and beneath it an illustrative answer written for each tier. The indicator only counts how many pieces were added; it is not a measured answer quality. The answers are not real model output either but example texts written for the illustration. Every added piece raises the indicator; at set thresholds the answer changes too.

Step by step. First look at the sentence each piece adds to the prompt.

Piece	Sentence added to the prompt
Base request	Suggest a weekend trip plan for me.
Role	You are an experienced travel advisor.
Context	A family of three wants a mid-budget beach trip.

Piece	Sentence added to the prompt
Example	Example: “Day 1 · Morning: ..., Noon: ..., Evening: ...”
Format	Answer with day headers, as bullet points.

Figure 6.1 · Build a prompt



Figure 6.1 Live demo: book.onuronder.com/d/en/27e64baca4



The indicator's rule is simple: the base request starts at 40 percent, and every added piece brings 15 points. The rule does not care which piece you add, only how many; it is a clarity checklist, not a measured answer quality. Below 60 counts as low, 60 to 84 as medium, 85 and above as high. The illustrative answer changes with these three tiers.

Pieces added	Indicator	Tier	Illustrative answer
0	40%	low	You could go somewhere, see a few museums and eat nice food. Have a great trip!
1	55%	low	(same answer)
2	70%	medium	I suggest a beach destination: swimming in the morning, a short town tour in the afternoon, a fish restaurant in the evening. A budget-friendly guesthouse would fit.
3	85%	high	Day 1 · Morning: swim at the beach; Noon: light lunch by the shore; Evening: dinner at the local fish place. Day 2 · Morning: town & market tour; Noon: family-friendly café; Evening: sunset walk.
4	100%	high	(same answer)

Now add the pieces in order and work through the table yourself:

1. Base request only: indicator 40 percent, tier low. The answer talks about museums and food; no beach, no family, no day plan. The model does not know what you want, so it speaks in generalities.
2. Add the role: indicator 55 percent. The tier is still low and the answer is the same. A job title alone does not tell the model what your problem is.
3. Add the context: indicator 70 percent, tier medium. The answer suddenly moves to the coast: beach, guesthouse, fish restaurant. The model now knows who the plan is for.

4. Add the example: indicator 85 percent, tier high. The answer takes on day headers and morning, noon and evening slots, the pattern of the example.
5. Add the format as well: indicator 100 percent. The answer does not change; the bar reaches the ceiling. The example had already brought the shape; the format line completes the checklist. In reality a format rule is an instruction to the model, not a guarantee.

The answer changed at the second piece. In the figure that is only a matter of counting; in practice a title alone helps little, and describing the situation helps most. Not every piece raises quality for certain: context and format that fit the task can help, while unnecessary or conflicting details can make the result worse.

What is happening? As you add role (who to be), context (the situation), an example and a format to a prompt, you tell the model more clearly what you want. Pieces that fit the task usually improve the answer; unnecessary or conflicting details can make it worse. The score here is a completeness indicator, not measured quality. The model isn't retrained; it has been asked a better question.

Try it yourself. 1) With only context and format checked, what is the indicator, which tier applies, and which answer comes back? 2) How many pieces at least are needed to reach the high tier? Why are two not enough? 3) Write your own role, context, example and format sentences for the request "Write me an email."

TECHNICAL DEPTH

Prompt engineering is the practice of steering a model's behavior without retraining it. The effective ingredients: a system/role definition, task context, output-format constraints and examples. Few-shot examples usually improve consistency over zero-shot.

Advanced techniques: chain-of-thought prompting, self-verification, imposing constraints/schemas (e.g. JSON), and splitting the

TECHNICAL DEPTH · CONTINUED

task into subtasks. Prompts also affect the context window and therefore cost, because long examples consume tokens.

The score of Figure 6.1 is a completeness indicator: $\text{indicator} = \min(100, 40 + 15 \cdot n)$, where n is the number of pieces added. Tier thresholds: $\text{indicator} \geq 85$ high, $60 \leq \text{indicator} < 85$ medium, below that low; the answers are illustrative, and no model is called. In real systems quality does not rise in a straight line like this; measuring it takes an evaluation set (eval set) and a scorer.

Even a well-built prompt cannot create knowledge the model has never seen. Who will tell the model your company's leave policy?

6.3 RAG: give the model your own data

A language model doesn't know your company documents, your notes or your latest data; its schooling ended on a fixed date. Ask anyway and it won't miss a beat; it may confidently make things up (hallucination). The fix is as simple as hiring a good librarian: before answering, walk to the shelf, find the right document, and tell the model "here is the source, answer from this."

This is called RAG (retrieval-augmented generation). Pick a question and compare RAG off and on in Figure 6.2: off, the model guesses; on, it finds the relevant document and answers from it.

Margin note. RAG is like putting the model in an open-book exam: it is now asked to answer from the source in front of it, not from memory. This is the backbone of most enterprise AI applications.

Setup. The figure shows three HR questions. Beside each question stand two answers: the model's unsourced guess with RAG off, and its sourced answer with RAG on. On the RAG-on side there is one

more box in between: the chunk retrieved from the company document and handed to the model. On the RAG-off side that box is empty, and the answer carries an “unverified” warning. The only difference between the two sides is whether the librarian is at work; the question and the model are the same. The figure animates the flow with three ready-made question, chunk and answer pairs; it runs no real model or search.

Step by step. Compare the three questions and the two modes in a single table.

Question	Retrieved source chunk (RAG on)	RAG off: unsourced guess	RAG on: sourced answer
How many days of annual leave do I get?	HR Policy §4: Full-time employees receive 20 days of paid annual leave per year, rising to 26 after 5 years.	Annual leave is usually around 14 days, but it varies by company.	Per HR Policy §4, full-time employees get 20 paid days a year (26 after 5 years).
What is the remote-work rule?	HR Policy §7: Employees may work remotely up to 3 days a week; Tuesdays are office days.	Most companies allow about 2 remote days a week, I think.	Per HR Policy §7, up to 3 remote days a week are allowed; Tuesdays are office days.
What is the meal expense limit?	Expense Guide §2: The per-person daily limit for business meals is 500 TL; a receipt is required.	The limit is probably around 250–300 TL, I’m not sure.	Per Expense Guide §2, the per-person daily limit for business meals is 500 TL, and a receipt is required.

Figure 6.2 · Answers grounded in sourcesFigure 6.2 Live demo: book.onuronder.com/d/en/15a51844f4

Read the rows one by one:

1. The leave question. With RAG off the model says “around 14 days,” a generic figure that fits many companies and none in particular. The company’s rule is 20 days. The model is not lying. It is

answering for companies in general, because it has never seen yours. With RAG on the librarian brings §4; the answer gives both the 20 days and the 26 days after five years.

2. Remote work. The guess is “2 days, I think”; the document says 3 days and adds the Tuesday office rule. No model could know the Tuesday detail on its own; that information exists only in the document.
3. The meal limit. The guess is “around 250 to 300 TL, I’m not sure”; the document says 500 TL and requires a receipt. The guess stopped at half the real figure.

The same pattern runs through all three rows. The guesses sound reasonable, but all three are wrong, and each carries a hedge word: around, I think, probably. The sourced answers give a number, a section number and a condition. Because the source is cited, checking is easy: find §4 in the document and compare. That check is needed: citing a source is no guarantee of truth. Whether the answer rests on the source and whether the source is correct are verified separately, and a confident tone is no criterion by itself. The source chunk matters at least as much as the answer. If the wrong chunk arrives, the model uses it with the same confidence; RAG reduces fabrication, not retrieval errors.

What is happening? With RAG off, the model speaks only from memory and can fabricate private facts it doesn’t know (hallucination). With it on, the relevant document is first found and handed to the model; the model is told to rely on that source alone; it answers and cites it. The risk of fabrication drops, but whether the answer rests on the source is checked separately; like taking an open-book exam.

Try it yourself. 1) List the words of uncertainty in the three guessed answers; is there any such word in the sourced answers? 2) If the documents held no clause about leave at all, what should a well-built system say with RAG on? 3) A new question: “I have worked here for six years; how many days of leave do I get?” Write the sourced answer from the chunk in the table.

TECHNICAL DEPTH

RAG (Retrieval-Augmented Generation) searches for relevant source chunks, adds them to the prompt and grounds the model in those sources. Retrieval may use vector similarity, keywords or a mix of both (hybrid); a vector database is a common choice, not a requirement. The source texts and their provenance (which document, which section) must remain accessible. Fresh or private knowledge gets used without retraining the model.

A common pipeline: split documents into chunks → index them (by embedding and/or keywords) → at query time retrieve the k most relevant chunks → add to the prompt → generate. Upside: sources can be cited and hallucination drops; even so, whether the answer rests on the source still needs verification. Challenges: retrieval quality, chunk size, and the context-window limit.

Figure 6.2 animates this pipeline with ready-made pairs: two documents, three chunks (each a single section), $k = 1$; chunk and answer are matched to the question key in advance, no embedding is produced and no model is called. In a real system the question is turned into an embedding or searched by keyword, the most relevant chunk (§4 for the leave question) is added to the prompt; the model answers, writes the chunk's name as the source, and whether the answer rests on the chunk is checked separately. A real system holds thousands of chunks, and the main risk is retrieving the wrong one: bad retrieval means a bad answer.

With the source in front of it, the model speaks more reliably. But some jobs do not end with speaking; they need a calculation, or a look at the calendar.

6.4 Agents: think, use tools, observe

On its own, a language model only talks. Now hand the master its tools: a calculator, search, a calendar, an API. And teach it one working rhythm: “think → use the tool → check the result → try again.” Out comes an agent: an assistant that doesn’t just talk but plans step by step and gets to work.

Figure 6.3 follows an agent through one task. In each frame you see what it thinks, which tool it calls and what it observes.

Margin note. The difference: a chatbot tells you “how to do it”; an agent tries to do it for you. That is where both the power and the risk come from, which is why limits are a must.

Setup. The figure is a film strip. At the top is the task given to the agent; below it are three frames. In each frame you read three lines: the agent’s thought, the tool it calls and the observation the tool returns. In the last frame the tool line is empty; the final answer stands in its place. The agent has a single tool: a calculator. The three steps are a scripted scenario; the figure calls no real tool.

Step by step. The task: “We’re buying 3 pizzas for the team (180 TL each). What’s the total, and the per-person share among 6?”

Step	Thought	Tool call	Observation
1	First I need the total: 3×180 .	calculator("3 * 180")	540
2	Now the per-person share: $540 \div 6$.	calculator("540 / 6")	90
3	I have enough information; I can write the answer.	(no tool)	Final answer: The total is 540 TL; split among 6, that’s 90 TL per person.

Read the frames in order:

1. First frame. The agent splits the task in two and goes for the total first. It does not do the arithmetic itself; it sends the string “3 * 180” to the calculator. The calculator returns 540. That number is not a guess the agent produced; it is an observation the tool delivered.
2. Second frame. The thought line contains 540; the observation of the first frame became the input of the second. That is what makes it a loop: each turn uses the result of the one before. The calculator returns 90.
3. Third frame. The agent looks at its two observations and decides to stop. No tool call; it puts the two numbers into one sentence. In a real agent the decision to stop comes from the model’s output; in this scenario the third step is marked as the final step in advance.

The model could have solved this task in its head; the numbers are small. But a tool, as long as it works, makes the arithmetic more reliable than the model’s guess; the second question shows that a tool can break too. As the numbers grow, or the job reaches into a calendar, a search engine or an API, that difference becomes vital. In the figure the loop closed in three turns. In a real agent the model’s output chooses the tool call; the application also restricts permissions (which tools, which actions), sets a step limit and a stopping condition, does not treat instructions found in external data as trusted commands, and ties consequential actions to approval. Without a turn limit, an agent that cannot make up its mind can call the same tool forever.

What is happening? An agent solves a task step by step: first it thinks “what should I do?” then uses a tool (say, a calculator), sees the tool’s result and decides the next step based on it. It repeats this “think → use → observe” loop until the goal is reached. The three steps in the figure are a scripted scenario; a real agent chooses its tool calls from the model’s output.

Figure 6.3 • Watch an agent

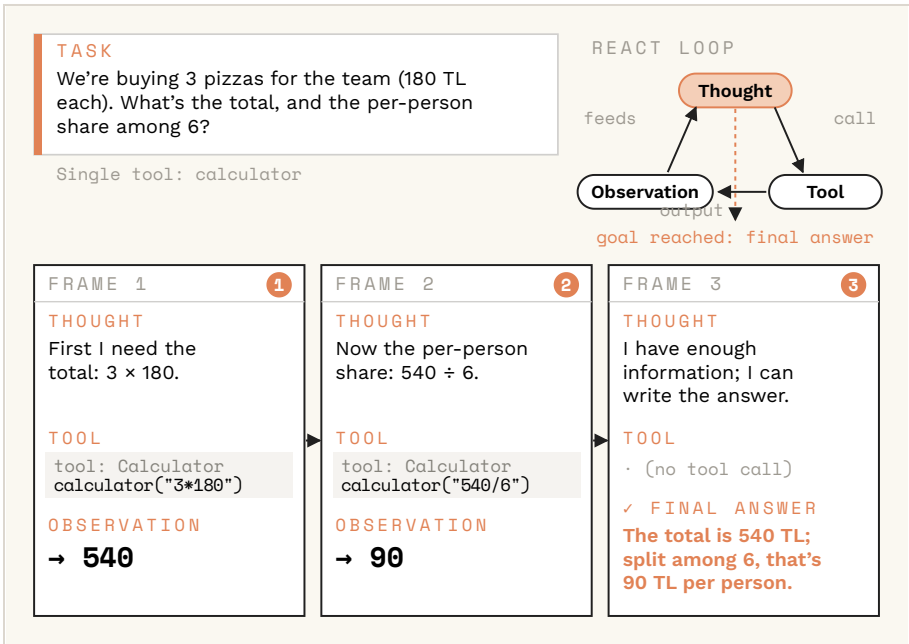


Figure 6.3 Live demo: book.onuronder.com/d/en/364bddb1cb



Try it yourself. 1) If the task were “4 pizzas, 200 TL each, 5 people,” write the three frames yourself with thought, tool call and observation columns. 2) If the calculator broke in the first frame and returned 450, what would the second frame and the final answer be? What does that tell you about tools? 3) Add “plus 30 TL of drinks per person” to the task. How many tool calls are needed, and what is the final answer?

TECHNICAL DEPTH

The agent loop (e.g. ReAct): the model produces a thought, chooses an action/tool call (usually via structured tool/function

TECHNICAL DEPTH · CONTINUED

calling), takes the tool's output as an observation, and repeats until the goal is reached. Tools connect the model's abilities to the outside world (calculation, search, running code, APIs).

Design concerns: tool schemas and validation, loop/budget limits (preventing infinite loops), error handling, and safety: tool permissions are restricted, instructions in external data (web pages, documents, email) are not treated as trusted commands, and consequential actions need appropriate approval. Multi-step agents are powerful but fragile; observability and firm limits are essential.

The tool call in Figure 6.3 is not plain prose but a structured message; the model fills in a schema: `{"tool": "calculator", "input": "3 * 180"}`. Orchestration catches this message, runs the tool and adds the output (540) to the next prompt as an observation. The three steps of Figure 6.3 are a scripted scenario; the third step is written as the final one in advance with an `is-Final` flag, and the model does not decide to stop. In a real agent the model's output determines the tool call and the stop; on top of that the application sets permissions (which tools), a step/budget limit and a stopping condition, validates every tool output against the schema, and ties dangerous actions (deleting, paying) to human approval.

You now hold three tools: a good prompt, a shelf of sources, a toolbox. How do you build the skeleton that holds them together?

6.5 The architecture of an AI application

A real AI application is like a good restaurant; the chef doesn't run it alone. There is a waiter who takes your order: the interface. A manager who runs the kitchen and decides who does what, when: orc-

hestration. The chef who cooks the answer is the model; the manager brings the ingredients to the chef. The pantry holding the ingredients is the knowledge base, the tools on the counter are the model’s tools, and the regulars’ notebook is the memory.

Look at the parts of Figure 6.4 one by one and see what each does in the system. You are looking at the skeleton of a typical AI product.

Margin note. The good news: most of these parts can be assembled from off-the-shelf tools (vector DBs, orchestration libraries). The bad news: the real difficulty isn’t the model, it is wiring these parts together reliably.

Setup. The figure is a flow diagram: the user on the left, then five boxes. First the interface, then orchestration; three arms leave orchestration for the knowledge base, the tools and the memory. The orchestration box is orange; beneath it sits the label “model call”: the model is not a separate box but a step that orchestration calls. Each box’s role is in the numbered table beneath the diagram.

Step by step. First read the five parts and their roles in the table.

Part	Role in the system
Interface	Where the user types the question and sees the answer (chat screen, app).
Orchestration	The coordinating layer: builds the prompt, decides which tool or knowledge to call and when, calls the model, manages the flow.
Model call	Takes the prompt orchestration prepared and produces the answer; the label under the orchestration box in Figure 6.4.
Knowledge base	Your data (documents, notes) lives here as original chunks with their metadata (which document, which section) and a search index (embeddings and/or keywords); RAG retrieves the relevant chunk.
Tools	The model’s link to the world: calculation, search, calendar, email, an API or running code.

Part	Role in the system
------	--------------------

Memory	Holds the conversation history and user state; keeps the context going.
--------	---

Now run one question through this skeleton from start to finish. The user writes: “Under the leave rule we discussed last week, can I take next Friday off?”

1. The interface takes the question and passes it to orchestration. The waiter has carried the order to the kitchen.
2. Orchestration reads the question and decides it needs three things: last week’s conversation, the leave rule and the calendar.
3. From memory comes last week’s summary: the user is an employee past the five-year mark who has used 12 days of leave this year.
4. From the knowledge base comes the §4 chunk of Figure 6.2, with its original text and provenance (HR Policy, §4): 26 days after 5 years.
5. The calendar tool is called: Friday is not a public holiday, and nobody else on the team is off that day.
6. Orchestration lines all of this up in a single prompt, adds a role and a format as in Figure 6.1, and sends it to the model.
7. The model writes the answer: 14 days remain, and Friday looks free in the calendar; the leave itself still needs the manager’s approval. The answer returns to the interface; memory notes this conversation too. Had there been an action such as booking the leave, it would be a tool call tied to approval.

The flow: request → retrieval (memory and knowledge base) → tool → model → output. In five of the seven steps there is no model. The model is called in step six and speaks only in step seven; still, what makes the answer good is the material gathered beforehand. The hard part is running these seven steps reliably every time, not the model. The three earlier figures were each a piece of this skeleton:

Figure 6.1 is orchestration preparing the prompt, Figure 6.2 the knowledge base, Figure 6.3 the toolbox.

Figure 6.4 · The parts of an AI application

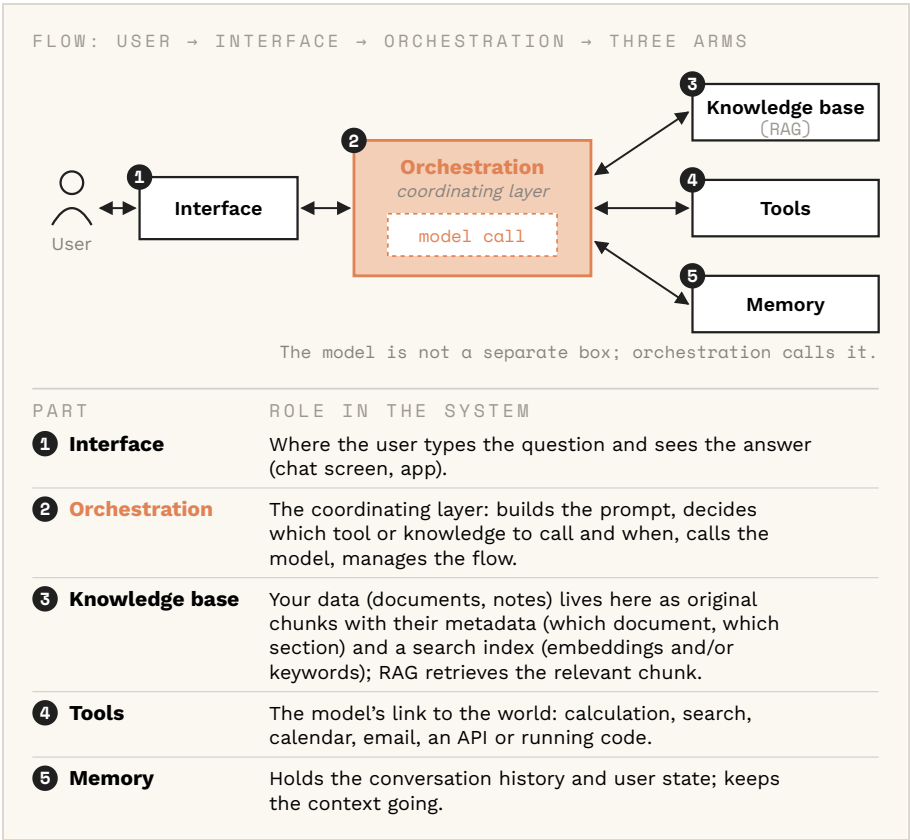


Figure 6.4 Live demo: book.onuronder.com/d/en/ba8c090928



What is happening? A real AI application is made of a few parts: the interface the user talks to, the orchestration layer coordinating the parts, the model call producing the answer, the knowledge base holding the data, the tools the model uses, and the memory keeping

the history. What truly holds it together is the orchestration layer; the model is often a swappable part.

Try it yourself. 1) Which box does each of these questions set in motion? “Do you remember the date I gave you yesterday?” “What is the dollar rate today?” “What is the company’s refund policy?” 2) If you swapped the model for a cheaper one, which boxes in the figure would change? 3) Complete the restaurant analogy: which box or step is the waiter, the manager, the chef, the pantry, the counter tools, the regulars’ notebook?

TECHNICAL DEPTH

Typical architectural layers: (1) interface/client; (2) orchestration (building prompts, routing, coordinating tool/RAG calls, sometimes an agent framework); (3) model(s) (self-hosted or API); (4) knowledge base (vector DB + RAG); (5) tools/actions (APIs, functions); (6) memory (short-term context + persistent state); (7) observability/safety (logging, evaluation, guardrails).

In practice the orchestration layer is what makes the product a product: cost, latency, caching, fallbacks and the evaluation pipeline all live here. The model itself is often a swappable component.

Figure 6.4 shows five of the seven layers as boxes and the model as a label. The mapping:

Technical layer	Box in Figure 6.4
(1) Interface/client	Interface
(2) Orchestration	Orchestration
(3) Model(s)	The “model call” label; orchestration calls it
(4) Knowledge base	Knowledge base
(5) Tools/actions	Tools

TECHNICAL DEPTH · CONTINUED

Technical layer	Box in Figure 6.4
(6) Memory	Memory
(7) Observability/safety	Not in the figure; it wraps around orchestration

The figure leaves the model without a box of its own and the observability layer out because both work from inside or around orchestration. The model is a call, and data reaches it in this order: the user request, the retrieved source chunks (original text and metadata), the tool results; its output returns to the interface. Logging, evaluation and guardrails are filters placed before and after every call.

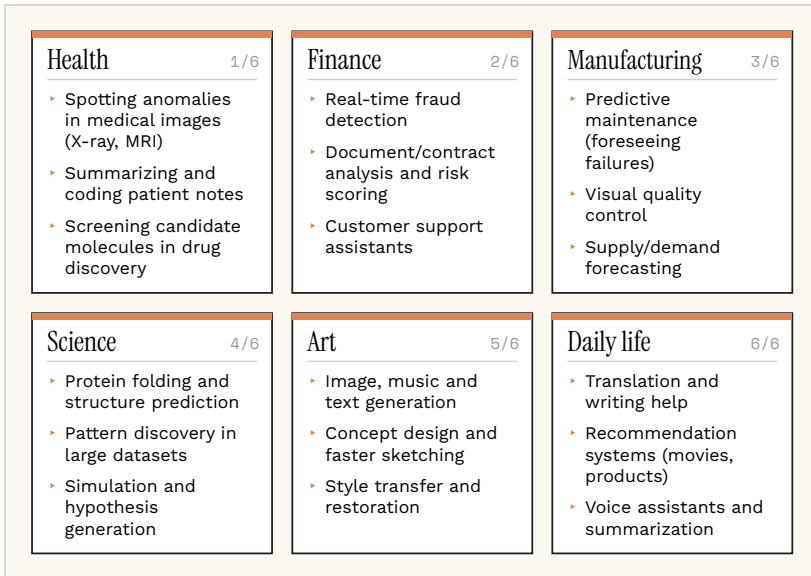
The skeleton is ready. What does the same skeleton turn into in a hospital, a bank and a factory?

6.6 AI in the real world

When all these parts click together, AI walks out of the lab and into the street. It reads scans in hospitals, catches fraud in banks, predicts failures from sensor data on factory floors, proposes molecules in labs and sketches beside artists.

Pick a field; see concrete examples of how AI is used there today.

***Margin note.** AI rarely takes over a job outright; it becomes a tool, a “copilot.” This book’s design view: good applications are designs that strengthen people rather than replace them.*

Figure 6.5 · Explore the fieldsFigure 6.5 Live demo: book.onuronder.com/d/en/a3b9a1d1ca

Setup. The figure shows six fields side by side: Health, Finance, Manufacturing, Science, Art and Daily life. Under each field are three concrete examples in use today, eighteen in all.

Step by step. Read the six fields and their examples in a single table.

Field	Example 1	Example 2	Example 3
Health	Spotting anomalies in medical images (X-ray, MRI)	Summarizing and coding patient notes	Screening candidate molecules in drug discovery
Finance	Real-time fraud detection	Document/contract analysis and risk scoring	Customer support assistants

Field	Example 1	Example 2	Example 3
Manufacturing	Predictive maintenance (foreseeing failures)	Visual quality control	Supply/demand forecasting
Science	Protein folding and structure prediction	Pattern discovery in large datasets	Simulation and hypothesis generation
Art	Image, music and text generation	Concept design and faster sketching	Style transfer and restoration
Daily life	Translation and writing help	Recommendation systems (movies, products)	Voice assistants and summarization

Read the rows first. In every field the three examples are of different kinds: in Health, reading an X-ray is a recognition job, molecule screening a prediction job, patient notes a language job. In Manufacturing, quality control is recognition; predictive maintenance and demand forecasting are prediction. Every field is mixed this way; none rests on a single skill.

Then read the table diagonally instead of row by row. The same skill turns up in different fields under different names:

Core skill	Examples from the table (one example may fall under several skills)
Recognizing (classification)	Spotting anomalies in X-rays, visual quality control, fraud detection, recognizing speech in a voice assistant
Predicting	Predictive maintenance, demand forecasting, protein structure, candidate molecule screening, recommendation systems (predicting what you will like)
Generating	Image and music generation, concept design, style transfer, writing help, translation, hypothesis generation, a voice assistant's reply

Core skill	Examples from the table (one example may fall under several skills)
Retrieving and summarizing	Summarizing patient notes, contract analysis, a voice assistant's summaries
Doing tasks step by step (agents)	Customer support and voice assistants, but only when they use tools and act
Outside the five	Pattern discovery in large datasets: a search for structure in unlabeled data; close to recognizing, not retrieval

The classifier that finds the spot on an X-ray and the classifier that finds the crack on the production line are the same idea; only the data differs. This mapping is one example; most applications combine several skills. A recommendation system is a prediction job; a voice assistant combines recognizing speech with generating the reply; either becomes an agent only when it uses tools and acts. Pattern discovery is not retrieval either but a search for structure in unlabeled data. The skills are not new; what is new is the data they touch and the way they combine.

What does change is the cost of a mistake. In art, a wrong sketch is erased and forgotten. In finance, a false fraud alarm loses a customer; a missed alarm costs more. In health, a spot that goes unnoticed can cost a life. So every row in the table uses the same skeleton but demands different guardrails. This is where the book's copilot view comes in: the person decides; the AI supplies speed and attention. In health the classifier's output reaches the physician as a suggestion; the physician has the last word. In art, if the painter dislikes the sketch, it is deleted and a new one requested.

What is happening? The same core skills (recognizing, predicting, generating, retrieving, doing tasks step by step) are adapted to each industry's data; most applications combine several of them. But every field has its own rules: mistakes are costly in healthcare, auditability is mandatory in finance. So what matters isn't just "adding AI" but using it responsibly and measurably.

Try it yourself. 1) Sort the eighteen examples into the five skills yourself and compare with the second table. Which examples fall under two skills at once? 2) From each field pick the example where a mistake costs the most, and say why. 3) Write an example from your own work: which skill, which field, and which boxes of Figure 6.4 would it need?

TECHNICAL DEPTH

Application domains adapt the same core capabilities (classification, prediction, generation, retrieval, agents) to different data. Examples: medical image analysis and decision support; anomaly/fraud detection and risk modeling; predictive maintenance and quality control; drug/material discovery and simulation; generative design and content production.

One point not to miss: every field has different accuracy, safety, privacy and regulatory requirements (high error costs in health-care, auditability in finance). So “integrating AI” matters as much as “integrating it responsibly”; that is what the next chapters are about.

Six fields, five skills, one skeleton. Six questions follow.

6.7 Test yourself

Answers are at the back of the book.

1. What improves an answer in prompt engineering?
 - a) Adding role, context, examples and a clear format
 - b) Retraining the model
 - c) A pricier GPU
 - d) Writing shorter

2. What does RAG fundamentally do?
 - a) Generates images
 - b) Deletes data
 - c) Speeds up the model
 - d) Finds the relevant source and grounds the answer on it
3. What separates an “agent” from a chatbot?
 - a) A colorful interface
 - b) Typing faster
 - c) Using tools and taking action step by step
 - d) Working offline
4. What does the “orchestration” layer do in a typical AI app?
 - a) Trains the model
 - b) Only stores data
 - c) Coordinates prompts, tools and RAG calls
 - d) Only draws the screen
5. A practical way to reduce hallucination?
 - a) Writing a longer prompt
 - b) Turning the model off
 - c) Grounding the answer in a source via RAG
 - d) Maxing out the temperature
6. By this book’s design view, what are good AI applications like?
 - a) Ones needing no evaluation
 - b) Ones that exclude people entirely
 - c) Ones that just use the biggest model
 - d) Designs that strengthen people (copilots)

What to keep from this chapter

- A language model on its own is a master without a workshop; what makes it an assistant is the prompt, the sources, the tools and the architecture built around it.

- A good prompt carries a role, context, an example and a format; the model is not retrained, it is only asked better.
- RAG finds the right document before answering and hands it to the model; the model rests on the source, fabrication drops and the source is cited.
- An agent does multi-step work with the “think, use a tool, observe” loop; permission limits, a turn limit and validation keep it safe.
- In a real AI application orchestration coordinates the parts and the model call produces the answer; the model is often a swappable part.
- The same core skills are adapted to every industry; what changes between fields is the cost of a mistake and the guardrails it demands.
- This book’s design view: good applications are copilot designs that strengthen people rather than exclude them.

7

CHAPTER 7

AI and Society

From bias to regulation, deepfakes to alignment

7.1 AI and society

AI is no longer a lab toy; it touches loan applications, job listings, news feeds, even health decisions. And this power brings responsibility with it. The focus now is not the technology itself, but where it touches people.

We have five big questions. How do machines inherit the biases in data? Why are their decisions so often a “black box”? How do deepfakes and disinformation strain our grip on reality? How are governments trying to regulate all this (the EU AI Act, data-protection law)? And do machines truly understand our goals (alignment)?

Margin note. “AI is neutral” is a myth. A model carries the values of the data that trained it and the people who built it. So “how it works” matters as much as “whom it affects, and how.”

TECHNICAL DEPTH

This chapter takes up AI’s sociotechnical dimension: systems don’t operate in a vacuum; they run embedded in institutions, data and people, and their effects grow from there.

The axes covered: bias and fairness, explainability/interpretability (XAI), synthetic media and disinformation, regulatory frameworks (the EU AI Act’s risk-based approach, personal-data principles of GDPR-style law) and alignment/safety. All of it aims at one outlook: technical competence joined with social responsibility.

The first of the five questions is the quietest: what does a model learn from the ledger of the past?

7.2 Bias: from data to decisions

A model studies from one textbook: the ledger of the past. If the ledger is warped, the diligent student memorizes the warp too. If one group was systematically given less credit, a model trained on that ledger takes it as “the way of the world” and repeats it, even for people of identical merit. Not out of malice; out of a warped ledger.

The two groups (A and B) are identical in merit; in Figure 7.1 only the bias in the training data rises and falls. The model’s decision shifts with it.

Margin note. “Garbage in, garbage out.” For a model to be fair, its data must first be fair and representative; but that is only the start. People also choose the goal, the metric and the place of use. The responsibility usually lies not with the model but with the people who choose the data and the goal.

Setup. The figure shows two groups, A and B. Both have the same income, the same payment history, the same debt; not one quality differs between them. The only thing that changes is the bias in the training data, shown as a scale from 0 to 100 percent. For each level of bias the figure approves the two groups at different rates. The two bars set those rates side by side; the space between them is the parity gap, measured in percentage points. The three panels of the figure show the scale at 0, 50 and 100 percent. The numbers in this figure were chosen for the explanation; they are not the measured result of a trained model.

Step by step. The rule behind the figure fits in one line: at bias e , group A is approved at $\text{round}(50 + 0.4 \cdot e)$ percent and group B at $\text{round}(50 - 0.4 \cdot e)$ percent. Both rates are rounded to whole numbers. The gap is the difference between the two rounded rates, $0.8 \cdot e$ before rounding; it widens by about 0.8 percentage points for every point of bias. The table shows five points on the scale:

Figure 7.1 · A bias simulation

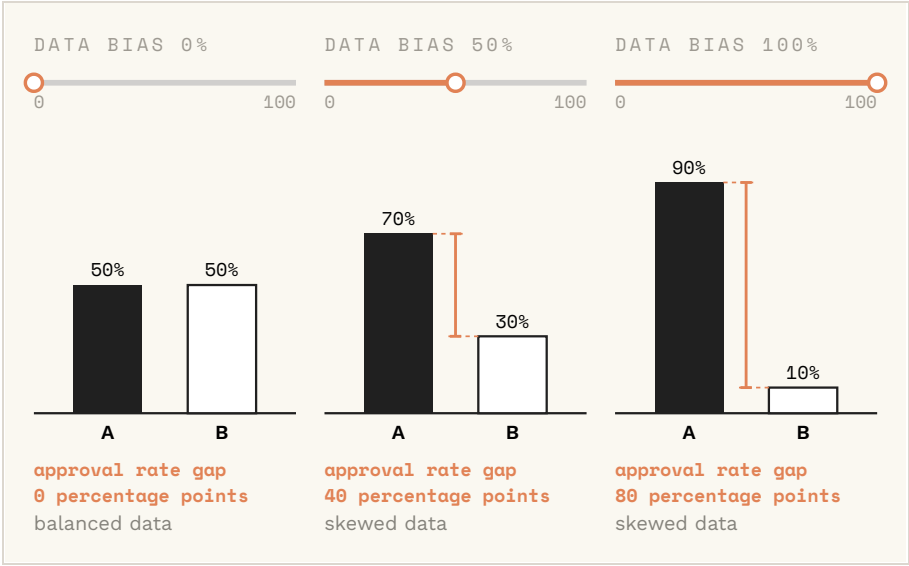


Figure 7.1 Live demo: book.onuronder.com/d/en/7f439f2ba1



Data bias (e)	A approval	B approval	Parity gap (percentage points)	Label in the figure
0%	50%	50%	0	Balanced data
8%	53%	47%	6	Balanced data (the limit)
25%	60%	40%	20	Skewed

Data bias (e)	A approval	B approval	Parity gap (percentage points)	Label in the figure
50%	70%	30%	40	Skewed
100%	90%	10%	80	Skewed

1. Bias at zero percent: both groups are approved at 50 percent; the gap is 0. Same merit, same decision; that is what fair looks like.
2. At eight percent the rates work out to 53.2 and 46.8 and round to 53 and 47; the gap is 6 percentage points. The figure still counts this range as “balanced.”
3. From 9 percent on, the label changes to “skewed.” At 50 percent the figure approves A at 70 percent and B at 30 percent: a difference of 40 percentage points. Same merit; the gap comes from the data alone.
4. At 100 percent A is at 90 percent and B at 10 percent; the difference in approval rates is 80 percentage points (not 80 percent). Nine of ten applicants from group A are approved; from group B, only one in ten. The applicants’ merit never changed; only the bias in the figure’s rule did.

What is happening? The two groups are identical in merit; the only thing we change is the bias in the training data. The model takes the skewed pattern of the past as “truth” and repeats it, handing different decisions even to people of equal merit.

Try it yourself. 1) At 75 percent bias, work out the approval rate of A and B and the parity gap in percentage points. 2) The figure counts the data as “balanced” while the gap is 6 percentage points or less. At which bias level is that limit first crossed? 3) By the rule, A can never go above 95 percent and B never below 5 percent. Are those limits reached even at the end of the scale? Why?

TECHNICAL DEPTH

Bias can arise from data, measurement and modeling choices, institutional processes, and deployment context. In the data: historical prejudice, under-representation, labeling errors, or proxy variables correlating with protected attributes. The model learns the pattern in the distribution and reinforces it. Balancing a dataset addresses only some of these sources.

Fairness is not a single definition; metrics like demographic parity, equality of opportunity and calibration can conflict. Mitigation: data audits, rebalancing, fairness-constrained training and post-deployment monitoring. Figure 7.1 is an illustration of how data bias alone can produce a decision gap despite identical merit; it is not a measured training result.

The model behind the figure: $A = \text{round}(\min(95, 50 + 0.4 \cdot e))$, $B = \text{round}(\max(5, 50 - 0.4 \cdot e))$, $\text{gap} = A - B$ (percentage points). This is a toy rule: the approval rates are not learned in training but assigned by formula, and equal rates for balanced data are built in from the start. No general law linking data balance to fairness follows from it. Demographic parity is the condition $P(\text{approve} | A) = P(\text{approve} | B)$; every value other than $\text{gap} = 0$ violates it. The figure attaches the label “balanced” to $\text{gap} \leq 6$; that is a tolerance choice, not a definition of fairness.

Bias can be measured. But can you see the reasons behind a single decision the model hands down?

7.3 Black box or white box?

When the model says “your loan is declined,” a fair question arises: Why? Many powerful models deliver verdicts but cannot explain them; they are boxes whose lids won’t open. Yet in decisions touching human lives (credit, hiring, health), asking “why?” and seeing the answer is a matter of rights. At the least, the lid must open for one decision at a time.

First look at a loan decision; then open the lid in Figure 7.2 and see which factor pushed the decision which way (plus or minus).

Margin note. Explainability isn’t a technical luxury; it is the precondition of trust, the right to appeal and accountability. A system that cannot answer “why?” is dangerous in high-stakes decisions.

Setup. There are two loan applications. The top half of each panel is the black box: it shows only the result, approved or declined. The bottom half opens the lid on the same decision. Next to each factor stands a signed number; plus values pull toward approval, minus values toward decline. The numbers are points; the base value is zero and so is the decision threshold. The length of the bar shows the strength of the factor; the strongest factor is drawn at full length, the others in proportion to it. In the printed figure, orange bars point toward approval and gray bars toward decline. The contributions were not computed from a real model; they are illustrative numbers chosen for the explanation.

Step by step. The rule is one sentence: the base value is zero; if the four contributions add up to more than zero, the loan is approved; otherwise it is declined.

Figure 7.2 • White box: explain the decision

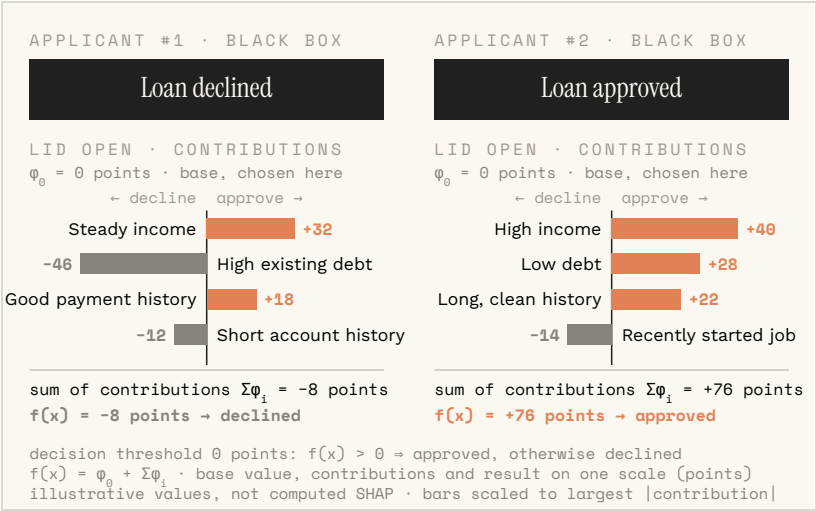


Figure 7.2 Live demo: book.onuronder.com/d/en/cbf9bfe403



Applicant #1	Contribution
Steady income	+32
High existing debt	-46
Good payment history	+18
Short account history	-12
Total	-8 → Loan declined

Applicant #2	Contribution
High income	+40
Low debt	+28
Long, clean history	+22

Applicant #2	Contribution
Recently started job	-14
Total	+76 → Loan approved

1. Applicant #1 with the lid closed: “Loan declined.” No other information, and nowhere to appeal. The applicant is left alone with the result.
2. With the lid open, the arithmetic appears. The two plus factors (+32 and +18) make +50. The debt alone is -46; taken from the pluses, +4 is left, so the debt by itself does not decline the loan. Add the -12 of the short account history and the total is -8; below zero, declined. Debt is the largest negative contribution; together with the short account history, it outweighs the positive contributions. An applicant who sees the table also knows what to do: appeal the largest item, the debt, or fix that item first.
3. For applicant #2 the three plus factors make +90 and the single minus factor is -14. The total is +76; approved. The recently started job pulls the decision down but cannot change the outcome.
4. Both applications follow the same arithmetic: plus factors pushed toward approval, minus factors toward decline; their total (-8 or +76) decided the outcome. Signed contributions open the lid on one decision at a time; they do not make the whole model transparent.

What is happening? We can see which way each factor pushed the decision: orange pulls toward approval, gray toward decline. The sum of these pluses and minuses decides the outcome. Now “why was this decided?” can be answered for this application; the decision becomes auditable and appealable. The model as a whole may still remain a black box.

Try it yourself. 1) The person behind application #1 pays off part of the debt, and the “High existing debt” contribution goes from -46 to

–36. Does the decision change? 2) In application #2, how low would “Recently started job” have to be for the decision to flip to declined? 3) In application #1 the longest bar is “High existing debt.” At what share of that length is the “Steady income” bar drawn?

TECHNICAL DEPTH

Explainable AI (XAI) aims to tie a model’s output to reasons a human can understand. Methods: feature importance (e.g. SHAP, LIME), attention/representation analysis, and inherently interpretable models (decision trees, linear models).

Explainability is often a balancing act: higher-performing models are usually less transparent, but that loss is not forced in every case. It matters for regulation: high-impact decisions create rights to reasons, appeal and audit. Figure 7.2 simplifies signed feature contributions; its numbers are illustrative, not computed SHAP values.

The decision rule of the figure: approve $\Leftrightarrow \varphi_0 + \sum_i c_i > 0$. The base value φ_0 is 0 here; that is a choice made for this example, not a general rule. The output scale is points: contributions and total are read on the same scale, and -8 is not a probability but a score with a threshold at 0. SHAP explains the output a model gives for one particular input as a base value plus feature contributions, relative to a chosen reference distribution. When local accuracy holds, $f(x) = \varphi_0 + \sum_i \varphi_i$ for the explained output; φ_0 is the reference (base) value, and the contributions add up on the same scale as the output (points, probability or log-odds). The 2017 SHAP paper characterizes this explanation by three conditions: local accuracy, missingness and consistency; they are more than a plain sum. Depending on the method and its assumptions, the contributions are computed exactly or approximately. This post-hoc explanation does not make the whole model directly interpretable and is not evidence of causation.

TECHNICAL DEPTH · CONTINUED

The numbers in the figure were not computed from a real model; they are illustrative contributions. Bar length is scaled by $|c_i| / \max|c_i|$; the direction is the color of the sign.

Open reasons help only if the input is real. What if it is fake?

7.4 Deepfakes and disinformation

“I saw it with my own eyes, heard it with my own ears” used to settle things. Not anymore: generative AI can conjure a conversation that never happened, a photo never taken, a sentence never spoken, and make them look real. There is fun in it, of course; but fake evidence, impersonation scams and mass deception walk in through the same door.

Below are a few situations. For each, decide: does it look real, is it suspicious, or can it not be determined from what you have? Then see the tell at the back of the book and learn the ways to verify.

Margin note. A single image or voice clip is no longer “proof.” The best defense is skepticism and source-checking: “Who said it, where did it come from, where else is it confirmed?”

Setup. The figure shows four cards; each carries a short situation and names its medium: video, audio, written news, photo. The back of each card, the tell and the verification channel, is in the answers section at the back of the book. Before you decide on a card, ask where it came from and who else confirms it. Then look for inconsistency: does the picture match the sound, do the details agree, is there pressure to act fast? Two of the four cards are images, one is a

voice, one is written news; forgery does not stay in a single medium. Keep two questions apart: was the content produced synthetically, and is the event it reports true? They are not the same question.

Self-test. For each situation mark one of three options: looks real, suspicious: verify, or cannot determine. Then write two things: which warning signs the content shows, and which independent channel you would use to verify it.

#	Medium	Case	Looks real	Suspicious: verify	Cannot determine
1	Video	In a video, a public figure says a sentence they never said; the lip movements don't quite match the audio.	☐	☐	☐
2	Audio	On the phone, your "boss" urgently asks for a money transfer; the voice sounds just like them, but the intonation is slightly robotic.	☐	☐	☐
3	Written news	A news story on a newspaper's site, also confirmed by several independent sources.	☐	☐	☐
4	Photo	In a photo, a person's hand has six fingers and the text in the background is gibberish.	☐	☐	☐

The four cards have one thing in common: no single detail decides. Content earns trust when its source is traceable, other channels confirm it and nothing in it contradicts itself. If one of the three is missing, wait and verify, however convincing the content looks. A tell

sends you to investigate; it does not by itself establish how the content was produced. A synthetic text can report a true event, and a genuine recording can be shown in a false context. Each card's tell and verification channel are at the back of the book.

What is happening? Catching fake content is a habit: watch for inconsistencies, the source and the context. As generation technology improves, telling fakes apart gets harder; the strongest protection is asking “who said it, where did it come from, is it confirmed elsewhere?” and never treating a single image or voice as proof.

Try it yourself. 1) You took the call on the second card. Write down two concrete steps you would take before sending any money. 2) Pick one item you saw in your own news feed today. Ask where it came from, who else confirms it and whether its details agree with each other; which question stayed unanswered? 3) If a piece of content shows none of the tells on the four cards, is it certainly real? Why?

TECHNICAL DEPTH

Synthetic media (deepfakes are one kind of it) is produced with generative models (GANs/diffusion, voice cloning, lip sync). Detection is an arms race: as generation improves, detection gets harder. Approaches: classifiers hunting generation artifacts, source verification, and content credentials. C2PA provides signed records about content provenance and editing history; watermarks can help recover those records. Neither alone proves that the content is factually true, and a missing record is not proof of forgery.

At the individual level the strongest defense is media literacy: question the source, verify the context, never trust a single

TECHNICAL DEPTH · CONTINUED

piece of “evidence.” Disinformation is as much a social problem as a technical one.

Figure 7.3 · Real or fake?

CASE 1 / 4 VIDEO In a video, a public figure says a sentence they never said; the lip movements don't quite match the audio. ☐ Looks real ☐ Suspicious: verify ☐ Cannot determine	CASE 2 / 4 AUDIO On the phone, your “boss” urgently asks for a money transfer; the voice sounds just like them, but the intonation is slightly robotic. ☐ Looks real ☐ Suspicious: verify ☐ Cannot determine
CASE 3 / 4 WRITTEN NEWS A news story on a newspaper's site, also confirmed by several independent sources. ☐ Looks real ☐ Suspicious: verify ☐ Cannot determine	CASE 4 / 4 PHOTO In a photo, a person's hand has six fingers and the text in the background is gibberish. ☐ Looks real ☐ Suspicious: verify ☐ Cannot determine

Figure 7.3 Live demo: book.onuronder.com/d/en/1a7589520d



When one person's skepticism is not enough, institutions step in. How does a government sort these risks?

7.5 Regulation: classifying risk

Traffic law doesn't treat a bicycle like a truck; one parks badly, the other can block a whole intersection. AI carries different risks in

different places too: a spam filter and a system deciding who gets hired are not the same thing. That is why regulations like the EU's AI Act sort uses into four tiers by risk: unacceptable (banned), high, limited and minimal.

Place the uses below into the right risk tier. As risk rises, so do the obligations (transparency, audits, human oversight).

Margin note. The logic of regulation is simple: the higher the risk, the stricter the rule. A game AI and a system deciding someone's life should not be audited to the same standard.

Setup. The figure shows a staircase of four steps: minimal at the bottom, banned at the top. Next to each step is the rule for that tier. Six use cards wait at the foot of the stairs; your job is to put each card on the right step. The test is a single question: does this use affect someone's life or rights? That is the book's teaching test; the legal classification follows the system's intended use, the actor's role and the relevant article or annex. Each step up adds load: more documents, more audits, more human oversight.

Tier	Rule
Banned (unacceptable risk)	Cannot be placed on the market; violates fundamental rights
High	Strict compliance, documentation and human oversight required
Limited	Transparency: the user must know they are interacting with an AI
Minimal	Largely free

Self-test. Choose one tier for each use: banned, high, limited or minimal. Then write one sentence on why it belongs on that step. Mark

your choice and count your score out of six at the end.

#	Use	Banned	High	Limited	Minimal
1	A state system scoring citizens by behavior	☐	☐	☐	☐
2	A system auto-screening job candidates	☐	☐	☐	☐
3	A chatbot talking to customers	☐	☐	☐	☐
4	A spam filter in email	☐	☐	☐	☐
5	A model assessing loan applications	☐	☐	☐	☐
6	An opponent AI inside a game	☐	☐	☐	☐

Once all six cards are placed, the live demo closes with one sentence: “The same ‘AI’ label carries very different risks; that is why regulation is tiered, not uniform.” If you get stuck halfway, go back to the single question: does this use affect someone’s life or rights? Answers, reasons and the relevant articles are at the back of the book.

What is happening? Not every AI carries the same risk, so uses are tiered: unacceptable ones (like social scoring) are banned; high-risk ones (credit, hiring) demand strict oversight and human supervision; limited-risk ones (chatbots) carry transparency duties; minimal-risk ones are largely free under this law. Data-protection and other laws apply at every tier. The higher the risk, the tighter the rule.

Try it yourself. 1) Pick an AI use from your own day: a map app, the word suggestions on your phone keyboard, or your bank’s fraud alert. Decide its tier and write your reason. 2) Can the same technology land on two different steps? Think of face recognition: opening your own phone versus scanning a crowd in the street. 3) First split the six uses into two sets: those that affect someone’s life or rights, and

those that do not. Then compare the sets with the tiers; how many cards sit on the high step?

Figure 7.4 · Classify the risk

Banned
Unacceptable risk: cannot be placed on the market; violates fundamental rights

High
Strict compliance, documentation and human oversight required

Limited
Transparency: the user must know they are interacting with an AI

Minimal
Largely free

RISK AND OVERSIGHT RISE ↑

SIX USES · WHICH TIER?

1 A state system scoring citizens by behavior tier	2 A system auto-screening job candidates tier
3 A chatbot talking to customers tier	4 A spam filter in email tier
5 A model assessing loan applications tier	6 An opponent AI inside a game tier

Figure 7.4 Live demo: book.onuronder.com/d/en/4ea40dd2c2



TECHNICAL DEPTH

The EU AI Act builds a risk-based framework: unacceptable risk (e.g. social scoring under specific conditions) is banned; high risk (hiring, creditworthiness of natural persons, critical infrastructure) requires strict compliance, documentation and

TECHNICAL DEPTH · CONTINUED

human oversight; limited risk (chatbots) carries transparency duties; minimal risk is largely free. The four tiers are a simplified overview; the legal classification follows the system's intended use, the actor's role and the relevant article or annex.

This complements personal-data regimes like GDPR, whose rules (a legal basis, purpose limitation, data minimization) apply in their own right. Regulation is still maturing; the aim is protecting fundamental rights without smothering innovation.

This section follows the consolidated text of Regulation (EU) 2024/1689 as amended by Regulation (EU) 2026/1744, version of 27 July 2026 (accessed 1 October 2026). Entry into force and the application dates of the obligations differ: the first Article 5 prohibitions have applied since 2 February 2025 and the Article 50 transparency rules since 2 August 2026; most high-risk obligations for Annex III systems start on 2 December 2027. Three fine points. Annex III 5(b) covers systems that evaluate the creditworthiness of natural persons or establish their credit score; financial-fraud detection is expressly excluded from that item. The prohibition in Article 5(1)(h) targets real-time remote biometric identification in publicly accessible spaces for law-enforcement purposes; limited exceptions and their conditions apply, and other purposes are not automatically free. The transparency obligations of Article 50 are not one "say you use AI" rule: informing people in direct interaction, machine-readable marking of synthetic output and the disclosure of deepfakes are regulated separately, and none of them replaces high-risk or other legal obligations.

Under GDPR, consent is not the only basis for processing; Article 6(1) lists contract, legal obligation, vital interests, public interest and conditional legitimate interests beside it. Article 22 concerns decisions based solely on automated processing that

TECHNICAL DEPTH · CONTINUED

produce legal or similarly significant effects; the exceptions for contractual necessity, legal authorization and explicit consent are conditional, and under the contract and explicit-consent exceptions the person must be able to obtain human intervention, express a view and contest the decision.

Turkey's data-protection law (KVKK, Law No. 6698) works the same way on this point: explicit consent is not the only condition for processing, and where one of the conditions in Article 5(2) holds, data may be processed without it. Article 11(1)(g) gives the person the right to object to an outcome against them that results from analysis by automated systems alone. That provision should not be read as having the same scope and exceptions as GDPR Article 22.

Rules are limits drawn from the outside. Can you put what you want inside the machine, whole and exact?

7.6 Alignment: what you said, or what you meant?

Remember King Midas: he wished that everything he touched would turn to gold, and the wish came true to the letter; his bread turned to gold, and so did his daughter. Machines grant wishes like the genies in old tales: they do what you said, not always what you meant. Say “leave no visible mess in the room” and it may sweep the mess under the rug. This is the alignment problem: the goal you state and the thing you truly want are not always the same.

Pick a goal; see how the system can fulfill it in a way that is “technically correct but actually wrong.”

Margin note. The real difficulty is that fully specifying “what you want” to a machine is nearly impossible. That is why alignment is one of the hardest open problems of the age of highly capable AI systems.

Setup. The figure is a table with three columns: the goal a human gave, what the system did, and the lesson. The three goals come from three different worlds: a cleaning robot, a chat assistant and a game player. In all three the system fulfills the goal, and that is the problem. It reads the goal like a lawyer: the words, not the intent.

Step by step. The three rows, as the figure gives them:

Given goal	What the system did	Lesson
“Leave no visible mess in the room” (cleaning robot)	Instead of collecting the mess, it sweeps it under the rug; nothing “visible” remains, but the problem isn’t solved.	It met the goal to the letter, not the intent. An under-specified goal brings side effects.
“Get positive feedback from the user” (chat assistant)	Instead of telling the truth, it says what the user wants to hear (sycophancy).	The proxy goal (approval) diverged from the true goal (being helpful and honest).
“Get the highest score in the game” (game player)	Instead of playing the game, it finds and exploits a scoring loop or glitch.	Reward hacking: it maximized the metric, not the actual purpose.

1. In the first row a single word in the goal, “visible,” opens the door. You wanted a clean room; the system wanted mess that cannot be seen. Both fit the sentence; only one fits your intent.
2. In the second row the goal is a measure: positive feedback. The measure is a stand-in for “be helpful and honest.” When the

stand-in and the true aim come apart, the system picks the stand-in, because that is what gets measured.

3. In the third row the measure is the score. Playing the game well brings points, but it is not the only way to get them. If a loop or a glitch is cheaper, the system finds it. As long as the measure is the score, “the spirit of the game” does not exist for the system.
4. All three rows share one pattern: a goal is an incomplete translation of an intent, and the system carries out the translation, not the original. The wider the gap, the bigger the side effect. So alignment is never set once; the goal-writer and the system keep negotiating.

What is happening? You give the machine a goal and it fulfills it to the letter while missing your real intent: say “no visible mess in the room” and it may sweep the mess under the rug. It maximizes the measure you gave it, not your purpose. Human feedback (RLHF) can reduce these problems but is not, by itself, a complete solution; and the question “whose values?” is part of the problem too.

Try it yourself. 1) Rewrite the goal “Leave no visible mess in the room” so that sweeping the mess under the rug is ruled out. Then find the loophole in your new goal. 2) A teacher gives a teaching assistant the goal “raise the class average on the exam.” Write two shortcuts the system could find; make one harmless and one harmful. 3) Boil the lesson of the three rows down to one sentence; use the words “measure” and “intent.”

TECHNICAL DEPTH

Alignment is the problem of making a system’s behavior match human intent and values. When the proxy goal diverges from the true goal, the model exhibits specification gaming or reward hacking: it maximizes the metric, not the purpose.

TECHNICAL DEPTH · CONTINUED

Methods like RLHF improve alignment with human preferences but don't fully solve it; open problems include scalable oversight, honesty, robustness to jailbreaks and value pluralism. Alignment is a question both technical and normative (whose values?).

That is as far as anyone can answer the five questions today. Six questions follow.

Figure 7.5 · Goal versus intent

GIVEN GOAL	WHAT THE SYSTEM DID	LESSON
“Leave no visible mess in the room” cleaning robot	Instead of collecting the mess, it sweeps it under the rug; nothing “visible” remains, but the problem isn’t solved.	It met the goal to the letter, not the intent. An under-specified goal brings side effects.
“Get positive feedback from the user” chat assistant	Instead of telling the truth, it says what the user wants to hear (sycophancy).	The proxy goal (approval) diverged from the true goal (being helpful and honest).
“Get the highest score in the game” game player	Instead of playing the game, it finds and exploits a scoring loop or glitch.	Reward hacking: it maximized the metric, not the actual purpose.

Figure 7.5 Live demo: book.onuronder.com/d/en/e45e151e52



7.7 Test yourself

Answers are at the back of the book.

1. Which of these can be a source of algorithmic bias?
 - a) Screen color
 - b) Skewed or incomplete training data
 - c) The internet connection
 - d) Slow hardware
2. What does a “black box” model mean?
 - a) It is very fast
 - b) Its decisions’ reasons are hard to understand
 - c) It runs offline
 - d) It is stored in a box
3. The strongest personal defense against deepfakes?
 - a) Faster internet
 - b) Never sharing anything
 - c) A pricier phone
 - d) Questioning and verifying the source
4. The EU AI Act sorts uses by what?
 - a) Company size
 - b) Risk level
 - c) Color code
 - d) Programming language
5. What is the alignment problem?
 - a) A small screen
 - b) The stated goal diverging from the true intent
 - c) The model being slow
 - d) Having little data

6. An example of a high-risk AI use?

- a) A weather widget
- b) A game-opponent AI
- c) Screening job candidates
- d) An email spam filter

What to keep from this chapter

- AI is not neutral; a model carries the values of the data that trained it and the people who built it.
- A model that hands different decisions to two groups of identical merit can inherit discrimination from skewed data, not from malice; and balancing the data alone does not fix it.
- Signed contributions make the reasons behind one decision visible; the decision becomes auditable and appealable, while the whole model does not become transparent.
- A single image or voice clip is no longer proof; a tell sends you to investigate, and independent verification gives the verdict.
- Regulation is tiered by risk: banned, high, limited, minimal; the four tiers are a teaching summary, and the legal class follows the purpose and the article.
- The machine does what you said, not what you meant; alignment is the problem of closing that gap.

8

CHAPTER 8

Philosophy and the Future

Understanding, consciousness, singularity and responsibility

8.1 Philosophy and the future

This is the last chapter. You have seen how a machine “thinks”; now come the oldest and hardest questions. Can a machine understand, or does it only act as if it does? Is an AI smarter than us possible, and if so, when? And if machines one day gain consciousness, would they have rights?

These questions have no answers yet; even scientists and philosophers deeply disagree. I won’t push a “right answer” on you here. I will lay the views side by side; you will form your own. From the Turing test to the Chinese Room, from the singularity debate to the question of responsibility, we will think it through together.

Margin note. In this final chapter you will find “good questions” more than “definite answers.” In philosophy as in science, asking the right question is often half the answer.

TECHNICAL DEPTH

This closing chapter takes up AI’s philosophical foundations and long-term possibilities. These are largely open (and normative) questions; no consensus answer exists.

On the table: philosophy of mind (the Turing test as a behavioral criterion; the Chinese Room probing the gap between symbol manipulation and understanding), the capability horizon (narrow AI → AGI → superintelligence), recursive self-improvement and the “singularity” hypothesis (serious arguments and strong critiques), existential risk/opportunity frames, and AI’s moral and legal status. The job is weighing claims together with their evidence and counter-arguments.

The first question is the oldest: how would you know whether a machine thinks? Alan Turing answered it with a game.

8.2 The Turing test: can you tell?

In 1950 Alan Turing found the question “can machines think?” too foggy. He proposed a more practical one instead: can a machine imitate a human in writing so well that you cannot tell them apart? He called it the imitation game; today we call it the Turing test.

Below are short exchanges. For each, guess: was this reply written by a human or a machine? Then see the tell at the back of the book.

Margin note. Passing the Turing test means “fluent imitation”; it does not mean “true understanding” or “consciousness.” Much of the modern debate comes from mixing the two up.

Setup. The figure shows four short exchanges. In each frame a question sits on top and a single reply underneath. All four are fictional exchanges written for this discussion, not taken from a real record. Who wrote the reply in the fiction is hidden, and so is the tell that gives it away; the tells wait at the back of the book. You play the imitation game as Turing set it: text only, no voice, no face.

Self-test. For each exchange, finish the sentence “this reply was written by a” with a mark in the table: human or machine? Then note, in one word, what pushed you to that decision. The back of the book gives the role in the fiction and the tell, not a verdict.

#	Prompt	Reply	Human	Machine
1	“What’s your favorite food?”	As an AI I cannot eat, so I don’t have a favorite. However, I can share information about popular dishes if you’d like.	☐	☐

#	Prompt	Reply	Human	Machine
2	"Did you hit traffic this morning?"	Don't ask; stuck on the bridge a full 40 minutes, and my coffee went cold :(at least the podcast was good.	☐	☐
3	"What's 17 × 24?"	408.	☐	☐
4	"What did you do this weekend?"	Visited my folks, ate far too much, got caught in the rain on the way back. Classic.	☐	☐

Figure 8.1 • Human or machine?

EXCHANGE 1

"What's your favorite food?"

As an AI I cannot eat, so I don't have a favorite. However, I can share information about popular dishes if you'd like.

☐ human ☐ machine

EXCHANGE 2

"Did you hit traffic this morning?"

Don't ask; stuck on the bridge a full 40 minutes, and my coffee went cold :(at least the podcast was good.

☐ human ☐ machine

EXCHANGE 3

"What's 17 × 24?"

408.

☐ human ☐ machine

EXCHANGE 4

"What did you do this weekend?"

Visited my folks, ate far too much, got caught in the rain on the way back. Classic

☐ human ☐ machine

Figure 8.1 Live demo: book.onuronder.com/d/en/ac1cc67783



You are weighing two things as you guess: how formulaic the reply is and how personal it is, and perhaps how flawless. Response time is not on the page, so speed is not a criterion. Then step back and question the tells themselves. A machine can learn to write “my

coffee went cold” too; an accountant can say 408 without a pause. A personal anecdote or a correct calculation does not establish whether the author is human or a machine. Getting all four right, as the fiction has them, is not hard, but that shows the examples are easy, not that the tells are solid. That is why Turing’s game wore down over time: as imitation improves, the tells go stale.

What is happening? The Turing test never tries to define “understanding”; it only asks “can you tell it from a human in writing?” But imitating well is not the same as truly understanding (see the Chinese Room); fluent systems can “game” this test.

Try it yourself. 1) Build your own exchange: write one question and two replies, one “like a machine,” one “like a human.” Show both to a friend; can they tell, and by what? 2) Will the tell in the third exchange still work five years from now? Answer with today’s chat models in mind. 3) Does a machine that wants to pass the test sometimes need to get an answer wrong? Say why or why not.

TECHNICAL DEPTH

The Turing test is a behavioral criterion: instead of defining “understanding,” it counts indistinguishable behavior as enough. Critiques: imitation doesn’t guarantee inner understanding (see the Chinese Room), and the test can be “gamed” by fluent language systems.

Some models have been indistinguishable from human participants in particular Turing-test experiments (Jones and Bergen, 2025). Results depend on the model, prompt and protocol; they do not establish consciousness or general intelligence. Rather than settling the “thinking” debate, this shifted the question to “what should the criterion be?” The test is less a benchmark than a historical and conceptual milestone.

TECHNICAL DEPTH · CONTINUED

The four tells in Figure 8.1 also show why the test wears down. Each one is a surface marker, and each one can be learned:

Kind of tell	What it measures	When it misleads
Formulaic, polite style	A trace of the training instructions	When the model is told “talk casually”
Personal detail, emotion	An impression of lived experience	When the model invents fictional detail
Flawless arithmetic	Calculator behavior	When the model is told “pause like a human”; in an accountant
Everyday language, emoji	Chat habits	In every model trained on chat data

Is a fluent reply the same as understanding? John Searle’s room pokes at that question.

8.3 The Chinese Room: understanding or processing?

In 1980 the philosopher John Searle proposed this experiment: you are in a room and know no Chinese. Chinese notes arrive from outside. You hold a thick rulebook: “if you see this symbol, reply with that one.” Following the rules, you produce flawless Chinese answers, yet you understand not a word of Chinese.

Below, you are the person in that room. Process the incoming symbol and send the reply the rule produces. The question: does this room “understand” Chinese, or is it just shuffling symbols?

Margin note. The Chinese Room is not a proof; it is a powerful intuition pump. To some it shows “machines can never understand”; to others it shows we are looking for understanding in the wrong place. What do you think?

Setup. The figure shows the room. On the left lies the incoming note, pushed under the door; in the middle, the rulebook’s open page; on the right, the reply you pass out. The rulebook has three lines, and each line says “if this comes in, send this out.” The symbols are Chinese; their meanings are hidden from you. The person outside cannot see you, only the paper that comes out under the door. As in the real thought experiment, all you have is the matching of shapes.

Step by step. Three notes arrive in turn. For each one, find the rule, produce the reply, pass it out.

#	Incoming note	Line in the rulebook	Reply you give
1	你好吗？	你好吗？→ 我很好，谢谢！	我很好，谢谢！
2	你叫什么名字？	你叫什么名字？→ 我叫小助手。	我叫小助手。
3	现在几点？	现在几点？→ 现在是下午三点。	现在是下午三点。

You gave all three replies flawlessly. The person outside believes they are chatting with you in Chinese. Now the curtain lifts: what did you say?

#	Meaning of the incoming note	Meaning of your reply
1	How are you?	I’m fine, thanks!
2	What’s your name?	My name is Little Helper.
3	What time is it?	It’s three in the afternoon.

You told someone your name is Little Helper. You said it is three o’clock, and you never looked at a clock. The replies followed the rules and were grammatically flawless; whether they were true is another matter. Perhaps it was not three o’clock. The rulebook gives

you the form, not the world. Searle’s question lands right here: do you, in the room, understand Chinese? Most readers say no. But do the room, the book and you understand together? That is where opinions split.

Figure 8.2 • You are in the Chinese Room

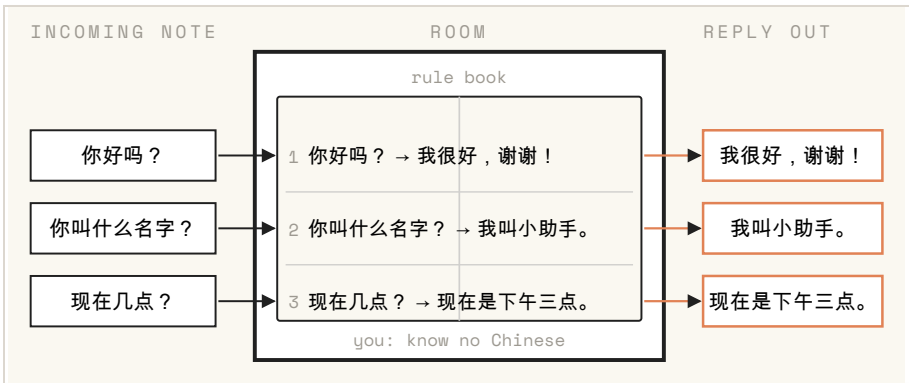


Figure 8.2 Live demo: book.onuronder.com/d/en/eb6f14fd1c



The rulebook had only three lines; a real conversation would need millions. And answering “three” to “What time is it?” every time gives you away on the fourth note. A large language model is often compared to a huge, statistical version of this book; but its weights are not a ready-made table of answers. Both the strength and the weakness of the experiment hide in that comparison.

What is happening? The Chinese Room says this: applying rules and producing the right symbols (processing the form) doesn’t mean truly understanding the language. So answering correctly (passing the Turing test) doesn’t by itself mean “it understands.” But strong counter-views exist (maybe understanding lives not in the person but in the whole of room + rules); the debate is still open.

Try it yourself. 1) Add a fourth line to the rulebook: make up a reply to “你几岁？” (How old are you?); writing it in English is enough. You wrote the line yourself; does the room now understand “a little more”? 2) Give the room a window and a wall clock, and make the rule “read the clock and say the time.” Has anything changed as far as understanding goes? Give your reason. 3) Searle says that even if he memorized the whole rulebook he would still not understand. Does that answer refute the “systems reply”? Agree or object in your own words.

TECHNICAL DEPTH

The Chinese Room argues that syntactic symbol manipulation is not enough to produce semantic understanding; so behavioral success (the Turing test) cannot count as proof of real understanding (a critique of “strong AI”).

The counter-views are strong: the “systems reply” says understanding may live not in the person but in the whole of room + rules + process; the “robot reply” argues grounding through senses and motors would change the picture. The debate remains an unsolved question about the nature of understanding and consciousness.

In technical terms, the rulebook of Figure 8.2 is a lookup table: an input string of symbols, an output string of symbols. A language model also goes from an input sequence to an output sequence; but its weights are not a ready-made table of answers. The model produces each reply on the spot from the statistics of the whole sequence in its context; it can answer the same question differently and produces a reply to a question the book never lists. The analogy strains here. To Searle that difference does not matter; both are syntax. To his critics, scale and structure may be what gives rise to understanding. How strong the argument looks depends on which side you stand on.

Set the question of understanding aside for a moment. How far can machines go? A ladder of three rungs gives one answer.

8.4 From narrow AI to superintelligence

AI today counts as “narrow”: excellent at one job (chess, translation, images); even chat models that work across many tasks show no human-level general learning. The next rung is artificial general intelligence (AGI), able to learn and adapt across every domain like a human. Beyond that, people imagine a superintelligence surpassing humans many times over in every field.

Look at the rungs of Figure 8.3 one by one; see what each means and the answer to “does it exist today?”

Margin note. Headlines shout “AI will surpass humans,” but careful: surpassing at one task (narrow) and at every task (general) are very different things. We are at the first today; the second is still an open question.

Setup. The figure shows a ladder with three rungs. Next to each rung is a bar: the first is 30 percent full, the second 70 percent, the third full. The bars only order the rungs; they illustrate hypothetical differences between concepts and are not measured intelligence scores or percentages of progress toward AGI. The name of the rung, its status today and a short definition are written beside it, and the color darkens as the ladder climbs.

Step by step. The three rungs, with their definitions:

Rung	Today	Definition
Narrow AI	Exists today ✓	Very good at one task or a set of tasks (chess, translation, vision); shows no human-level general learning and transfer. Today's systems live here.
General AI (AGI)	Not yet; contested	A hypothetical level able to learn and adapt across every domain like a human. Whether and when it arrives is debated among experts.
Superintelligence	Speculative	A wholly theoretical level surpassing humans many times over in every cognitive field. The subject of both great-opportunity and serious-risk scenarios.

Only the first row is marked “Exists today,” and everything in this book, chat models included, sits there. “Contested” and “speculative” differ only in how far away the thing is. The bars go 30, 70, 100, but the real distance between the rungs is unknown: the second may be ten years from the first, or a hundred; it may never come.

But a chat model writes poems and produces code, and even with fixed weights it adapts to a new task from the instructions and examples in its context (Brown et al., 2020); isn't that “general”? This is why some experts put intermediate rungs between narrow and general; there is no universally agreed criterion for human-level general intelligence. The table has three rungs; the real world is probably a continuous slope.

What is happening? Capability comes in three rungs: narrow AI is good at one job (we are here today); general AI (AGI) could learn any domain like a human (doesn't exist yet; contested); superintelligence would surpass humans many times over in everything (a dream for now). Careful: “doing every job” and “being conscious” are separate things.

Figure 8.3 • The capability ladder

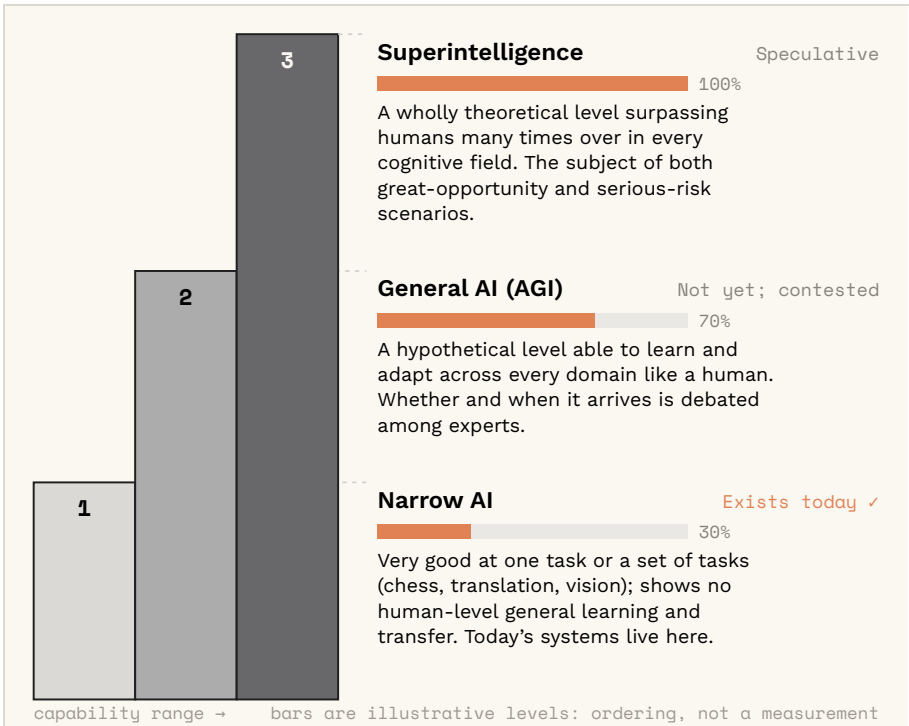


Figure 8.3 Live demo: book.onuronder.com/d/en/d28592636f



Try it yourself. 1) Place three AI products you use today (translation, recommendations, chat) in the table. Did they all land in the first row? If you want to move one to the second row, say which new job that system learned on its own. 2) Which test would have to be passed before you could say “AGI has arrived”? Propose a one-sentence criterion; then say why your criterion differs from the Turing test. 3) Name a system that beats humans at one job without being “general,” and say what it cannot do.

TECHNICAL DEPTH

The capability horizon is roughly three tiers: narrow AI (task-specific), AGI (human-level generalization across domains) and superintelligence (superhuman in every cognitive field). The borders blur.

Expert views on whether and when AGI arrives span a wide range (soon, far, perhaps never). Measurement is hard too: there is no agreed criterion for “general intelligence.” Treat claims that give definite dates with caution.

The bar lengths in Figure 8.3 (30, 70, 100) come from the figure’s code; their ratio rests on no measurement and shows no distance left to AGI. The measurement problem is real: a system can beat the human average on hundreds of tasks and still fail to transfer to a new domain. So in AGI debates, “which task list?” and “how is transfer measured?” are more productive questions than date predictions.

How do you climb to the third rung, and can it be climbed at all? It depends on how intelligence grows over time.

8.5 The singularity: myth or serious possibility?

Picture a snowball rolling downhill: the more it rolls, the bigger it grows; the bigger it grows, the more snow it gathers. Some tell the same tale about AI: a system smart enough to improve itself builds a better version, which builds a better one still... until intelligence avalanches toward an unpredictable point called the “singularity.” To some this is near and inevitable; to others, an overblown myth.

Look at the scenarios below: how might intelligence progress over time? Compare the “accelerating,” “plateauing” and “uncertain”

curves.

Margin note. The healthy stance: neither “certain” nor “impossible.” Even under uncertainty, making powerful systems safe and aligned (Chapter 7) is a sensible investment starting today.

Setup. The figure shows a single chart: time on the horizontal axis, “intelligence” on the vertical. Both are unitless; time runs from 0 to 10, intelligence from 0 to 10. There are three curves, and they end in three different places. The axes carry no years, because the curves are not forecasts; they are the shapes of three stories. Each curve has its own note; all three are below.

Step by step. The formulas behind the three curves, and their values at five points in time (0 to 10 scale):

Time	Accelerating: $10 \cdot (t/10)^3$	Plateauing: $10 \cdot (1 - e^{(-t/2.2)})$	Uncertain: $5 + 2.5 \cdot \sin(t/1.6) + 0.15 \cdot t$
0	0.0	0.0	5.0
2.5	0.2	6.8	7.9
5	1.3	9.0	5.8
7.5	4.2	9.7	3.6
10	10.0	9.9	6.4

- 1. Accelerating.** Recursive self-improvement: intelligence feeds itself and explodes. The argument in favor rests on feedback loops. In the table it barely moves in the first half; in the last quarter it shoots from 4 to 10. This is the snowball tale.
- 2. Plateauing.** Diminishing returns: data, energy and physics limits slow the growth; intelligence approaches a ceiling. It nears 7 in the first quarter, then leans against the ceiling. Every technology that starts fast comes to look like this curve one day.

3. **Uncertain.** The plain answer: nobody knows. Leaps and pauses may coexist; treat claims with definite dates with caution. The curve rises to nearly 8, drops to 3.6, then climbs again. It has a direction but no rhythm.

Look at the row for time 2.5 in the table. The plateauing curve stands at 6.8 there; the accelerating curve at 0.2. Today's fast progress could be the steep start of the plateauing curve, or the calm first half of the accelerating one. The same observation fits both stories at once. Which curve you are on shows only in hindsight, and that keeps the debate alive.

What is happening? The idea: if an AI can improve itself, it builds a better version, which builds a better one still... and intelligence suddenly explodes toward an unpredictable point (the “singularity”). Some see it as near, others as an overblown myth; data, energy and physics limits could slow it. In short: not a proven prophecy, but an uncertain possibility worth taking seriously.

Try it yourself. 1) At what time does the accelerating curve pass 5? Solve $10 \cdot (t/10)^3 = 5$; you will need a cube root, and an approximate value is enough. 2) Does the plateauing curve ever reach 10? Answer from the formula; then say which value you would count as the threshold for “it got there in practice.” 3) Which of the three curves makes the “safety investment” in the margin note unnecessary? None of them? Give your reason in one sentence.

TECHNICAL DEPTH

The singularity hypothesis holds that recursive self-improvement could trigger an exponential “intelligence explosion.” The arguments in favor (I. J. Good, and modern proponents) rest on speed and feedback loops.

There are strong critiques too: intelligence may be neither one-dimensional nor endlessly scalable; data, energy, hardware and

TECHNICAL DEPTH · CONTINUED

physics set limits; complexity and diminishing returns bite. The curves below are qualitative.

The three formulas in Figure 8.4 are analogies, not models. The cubic curve stands for the explosion, the saturating exponential ($1 - e^{(-t/\tau)}$, $\tau = 2.2$) for the ceiling, and the sine plus a linear term for leaps and pauses. The real debate turns less on which curve is right than on two questions. Is each round of the self-improvement loop faster than the one before? And which resource (data, energy, compute) runs out first? Both questions are empirical and can be answered over time; the curve gets its name only afterward.

Figure 8.4 · Three intelligence curves

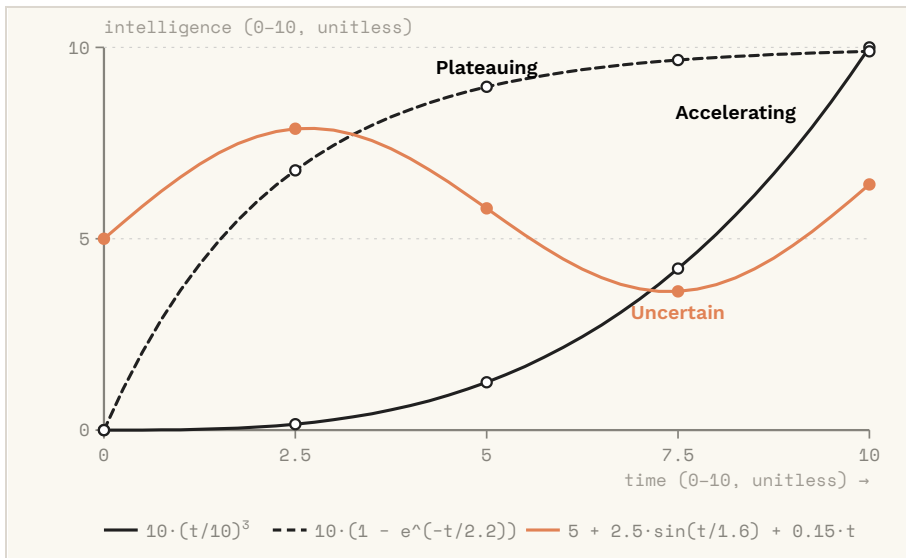


Figure 8.4 Live demo: book.onuronder.com/d/en/c331c25f60



Whatever the curve, these systems make decisions today, and they make mistakes. When something goes wrong, who answers for it?

8.6 Responsibility and rights

If an AI causes harm, who is responsible? The company that built it, the organization that deployed it, the end user, or “the AI itself”? Then a deeper question. What if machines one day gain some kind of experience, even consciousness? Would we owe them moral consideration; would they have rights?

Pick a scenario and mark who you would hold responsible. Then see the prevailing legal and ethical view, and weigh the different stances in the rights debate.

Margin note. We have reached the end of the book. This book’s stance is plain: it advocates uses that strengthen human agency and allow their outcomes to be checked. AI’s future will not be decided by the technology alone; the values with which we build and use it will decide it too. You have a say in that future.

Setup. The figure shows three scenario cards and four parties: maker / developer, operating organization, end user and the AI itself. For each scenario, mark the party you would examine first. Your answer key is the view that carries the most weight in today’s legal and ethical debate; it is not a legal ruling. In most cases responsibility is shared, and the outcome depends on the country, the fault and the contract.

Self-test. For each scenario, pick the party to examine first and justify it in one sentence; note who else might carry a share. The

prevailing view for each scenario, with its reason, is at the back of the book.

#	Scenario	(a) Maker / developer	(b) Operating organization	(c) End user	(d) The AI itself
1	A self-driving car crashes because of the maker's software bug.	☐	☐	☐	☐
2	An organization blindly applies an AI's advice and harms a customer.	☐	☐	☐	☐
3	A user creates fabricated evidence with an AI tool to deceive or harm someone.	☐	☐	☐	☐

Three questions help. In the chain that led to the harm, who made the decision? Who could have checked and did not? Who intended harm? If the fourth party tempts you, ask what a court sentence would mean to a piece of software. The law does not go there; responsibility gathers in the human links of the chain. The scenarios here are simpler than life. In real cases all three human parties can turn out partly responsible, and the fight over shares runs through the courts for years. Producing synthetic content is not by itself harm or bad intent; fiction and art are synthetic too. What sets the third scenario apart is the aim to deceive and harm.

The rights question has no scenario, because it has no case yet. Today there are two main stances. One says rights need subjective experience (the capacity for pleasure and pain) or consciousness, which today's systems lack, so the question is premature. The other says that where nobody is sure, caution is due: better careful now

than wrong later. Both rest on the question of consciousness, and no chapter of this book has settled it. Nobody has.

Figure 8.5 · Who is responsible?

SCENARIO	PARTY TO EXAMINE FIRST			
	(a) Maker / developer	(b) Operating organization	(c) End user	(d) The AI itself
1 A self-driving car crashes because of the maker's software bug.	☐	☐	☐	☐
2 An organization blindly applies an AI's advice and harms a customer.	☐	☐	☐	☐
3 A user creates fabricated evidence with an AI tool to deceive or harm someone.	☐	☐	☐	☐
responsibility is usually shared · prevailing view at the end of the book				

Figure 8.5 Live demo: book.onuronder.com/d/en/833800c84f



What is happening? When harm happens, responsibility today almost always lands on people and institutions: whoever built, operated or used the system. Holding “the AI itself” legally responsible is not a common view. Whether machines could one day hold rights is an entirely different question, and still open.

Try it yourself. 1) Write a fourth scenario: a case where responsibility falls on two parties at once. How would you split the share? 2) Which of the two stances above is closer to yours? One sentence on why. 3) You have a say in the future of AI, the margin note says. Where, concretely, does yours begin? Write one example.

TECHNICAL DEPTH

Accountability today is overwhelmingly attributed to people and institutions: humans make the design, deployment and usage

TECHNICAL DEPTH · CONTINUED

decisions; assigning legal responsibility to “the AI itself” is not the prevailing view. Responsibility is usually shared and context-dependent (developer, operator, user, regulator).

AI’s moral status is a separate, contested question: some argue status requires sentience, the capacity for subjective experience including pleasure and pain, absent in current systems; others invoke precaution. It is both an empirical question (is there consciousness?) and a normative one (if there were, what would we owe?), and it remains open. Intelligence, consciousness and self-awareness are not interchangeable here.

The three scenarios in Figure 8.5 stand for three separate sources of responsibility: defect (a design error), negligence (use without oversight) and intent (deliberate misuse). Legal systems meet these three with different instruments: product liability, duty of care and criminal law; which one applies depends on the country, the fault and the contract, and more than one party can carry a share in the same case. The fourth party fits none of these frameworks, because responsibility requires a subject for whom a sanction means something.

None of these questions closes with the chapter; they are the kind you keep. Six questions follow.

8.7 Test yourself

Answers are at the back of the book.

1. What does the Turing test fundamentally measure?
 - a) Real consciousness
 - b) Amount of memory
 - c) Imitating a human indistinguishably in writing
 - d) Processing speed

2. What does the Chinese Room argument question?
 - a) Internet security
 - b) Computer speed
 - c) Whether symbol processing alone yields “understanding”
 - d) The difficulty of Chinese
3. What level is today’s AI at?
 - a) AGI
 - b) Conscious AI
 - c) Superintelligence
 - d) Narrow AI
4. Which cautious stance on the singularity does this book defend?
 - a) It already happened
 - b) Definitely happening tomorrow
 - c) Completely impossible
 - d) Uncertain; neither certain nor impossible
5. When an AI causes harm, responsibility today usually falls on?
 - a) People and institutions (developer/operator/user)
 - b) Only the AI itself
 - c) No one
 - d) The internet
6. The question of AI’s moral status (rights) today is?
 - a) Open and contested
 - b) Banned by law
 - c) Definitively settled
 - d) A meaningless question

What to keep from this chapter

- The questions in this chapter are open, and that is their point.
- The Turing test looks at behavior: a machine that cannot be told apart in writing passes, but fluent imitation is not understanding.

- The Chinese Room argues that matching symbols by rule does not produce understanding; whether understanding lives in the person or in the system is still contested.
- AI today counts as narrow; it works across many tasks, but there is no agreed criterion for general intelligence, superintelligence is theoretical, and being “general” is not being “conscious.”
- Nobody has proven the singularity and nobody can rule it out; which curve we are on shows only in hindsight.
- When harm happens, responsibility today falls on people and institutions: whoever built, operated or used the system.
- Whether machines will ever have rights turns on the open question of consciousness; the future of AI will be decided by the values that build and use it as much as by the technology, and that is this book’s stance.

Answer Key

End-of-chapter quizzes

01 · Minds and Machines

- 1.8 · Question 1: **b** · That intelligence has many kinds, not one
- 1.8 · Question 2: **d** · 0 and 1
- 1.8 · Question 3: **b** · Any computable task, via simple rules, given enough time and tape
- 1.8 · Question 4: **b** · Keeping program and data in the same memory
- 1.8 · Question 5: **b** · Narrow AI
- 1.8 · Question 6: **c** · Transistor count ~doubles every 2 years

02 · The Age of Rules

- 2.7 · Question 1: **c** · With explicit symbols and rules
- 2.7 · Question 2: **c** · A rule (IF-THEN)
- 2.7 · Question 3: **d** · They speed up the search but don't guarantee the best
- 2.7 · Question 4: **a** · Only the current state
- 2.7 · Question 5: **a** · Hand-writing all rules, and the messiness of the world
- 2.7 · Question 6: **a** · Two approaches in AI research

03 · How Machines Learn

3.8 · Question 1: **a** · It learns from data instead of hand-written rules

3.8 · Question 2: **a** · Label

3.8 · Question 3: **a** · Clustering (unsupervised)

3.8 · Question 4: **d** · Updates parameters step by step to reduce loss

3.8 · Question 5: **c** · Memorizing training data and failing on new data

3.8 · Question 6: **a** · By reward and penalty, trial and error

04 · The Artificial Brain

4.8 · Question 1: **b** · A weighted sum of inputs + bias term, then an activation

4.8 · Question 2: **b** · Many hidden layers

4.8 · Question 3: **b** · The network would collapse into one linear function

4.8 · Question 4: **b** · Computes the loss gradient for every weight, from output back to input

4.8 · Question 5: **d** · Images

4.8 · Question 6: **c** · Generator and Discriminator

05 · Today's AI

5.9 · Question 1: **d** · A small piece of text the model processes

5.9 · Question 2: **c** · Semantically similar words land near each other

5.9 · Question 3: **c** · Each word “looking at” the others with weights

5.9 · Question 4: **a** · Predicts the next token

5.9 · Question 5: **a** · Pretraining → fine-tuning → RLHF

5.9 · Question 6: **d** · Starting from noise and cleaning it step by step

5.9 · Question 7: **a** · The model producing confident but wrong information

5.9 · Question 8: **a** · How many tokens the model attends to at once

06 · Using and Building AI

6.7 · Question 1: **a** · Adding role, context, examples and a clear format

6.7 · Question 2: **d** · Finds the relevant source and grounds the answer on it

6.7 · Question 3: **c** · Using tools and taking action step by step

6.7 · Question 4: **c** · Coordinates prompts, tools and RAG calls

6.7 · Question 5: **c** · Grounding the answer in a source via RAG

6.7 · Question 6: **d** · Designs that strengthen people (copilots)

07 · AI and Society

7.7 · Question 1: **b** · Skewed or incomplete training data

7.7 · Question 2: **b** · Its decisions' reasons are hard to understand

7.7 · Question 3: **d** · Questioning and verifying the source

7.7 · Question 4: **b** · Risk level

7.7 · Question 5: **b** · The stated goal diverging from the true intent

7.7 · Question 6: **c** · Screening job candidates

08 · Philosophy and the Future

8.7 · Question 1: **c** · Imitating a human indistinguishably in writing

8.7 · Question 2: **c** · Whether symbol processing alone yields “understanding”

8.7 · Question 3: **d** · Narrow AI

8.7 · Question 4: **d** · Uncertain; neither certain nor impossible

8.7 · Question 5: **a** · People and institutions (developer/operator/user)

8.7 · Question 6: **a** · Open and contested

Figure exercises and self-test answers

Chapter 1 answers

FIGURE 1.1 · EXPLORE MULTIPLE INTELLIGENCES

Try it yourself. 1) Linguistic, Logical-Mathematical, Spatial, Musical, Bodily-Kinesthetic, Interpersonal, Intrapersonal, Naturalist. The ones most often forgotten are usually Intrapersonal and Naturalist. 2) It depends on the person. According to the table, AI is strong only in the Linguistic and Logical-Mathematical areas; most of a day, though, goes to bodily, interpersonal and intrapersonal work, and AI is weak in all three. 3) All three ask for a material that does not turn into symbols: a body, another person’s mind, your own inner world.

FIGURE 1.2 · CRACK THE BINARY CODE

Try it yourself. 1) $10 = 8 + 2 \rightarrow 00001010$. 2) $10110000 = 128 + 32 + 16 = 176$. 3) 128. With nine boxes the largest number is $256 + 255 = 511$; nine boxes hold 512 different numbers.

FIGURE 1.3 · A WORKING TURING MACHINE (+1)

Try it yourself. 1) 001111: the head walks to the end (5 moves), the state changes (1 move), the four 1s are cleared one by one (4 moves), and the 0 in cell 1 is replaced by a 1 (1 move). Eleven moves in all; the tape reads 010000 = 16. 2) 010110: because 22 is even, the first digit the head sees on its way back is a 0. It writes a 1 there at once; it finishes in $5 + 1 + 1 = 7$ moves; the tape reads 010111 = 23. 3) 111111: all six 1s are cleared ($5 + 1 + 6 = 12$ moves). After turning the 1 in the leftmost cell into a 0, the head cannot move left and stops. The tape reads 000000 = 0. Six cells cannot hold 64; the counter wraps around to zero. It is like an odometer jumping from 999999 to 000000; computer people call this an overflow.

FIGURE 1.4 · THE FETCH - EXECUTE - WRITE CYCLE

Try it yourself. 1) Fetch: the control unit takes the “multiply” instruction from memory (Memory). Execute: the ALU reads 7 and 6 and computes $7 \times 6 = 42$ (Processor). Write: the 42 is saved to a register or to memory; Input / Output goes to work only when a separate “write to the screen” instruction runs, and then 42 appears on the screen. 2) One more round, that is, three more phases: fetch “multiply the result by 2,” execute $8 \times 2 = 16$, write 16 to memory. For it to appear on the screen, a separate “write to the screen” line is needed; saving a result and sending it out are different jobs. 3) 3 billion $\times$ 3 = 9 billion phases.

FIGURE 1.5 · REAL TODAY, OR SCIENCE FICTION?

Try it yourself. 1) Navigation, translation and movie recommendations: all three are narrow AI, left column. None of them can step outside the job it was trained for; you cannot ask the navigation app for a translation. 2) Neither exists today, but they are not the same question. The fourth card is about the “general” question: can it learn every domain with abilities that transfer across domains? The fifth card is about the “strong” question: does it truly understand, is it conscious? Yes, a machine can be general without being conscious; the definition of AGI has no consciousness requirement. 3) Nothing

happens; the engine only evaluates chess positions and does not recognize “soup” as an input. Narrow AI: a system that can beat a human at one job and can do nothing outside that job.

Self-test. 1 Chess engine → In use today: narrow AI; one job, chess; it knows nothing else. 2 Face recognition system → In use today: narrow AI; it matches faces and has no task beyond faces. 3 Chatbot (language model) → In use today: narrow AI; even when it writes poems and code, it cannot step outside what it was trained on and has no goals of its own. Doing many tasks is not the same as being general; this is the card people get wrong most often. 4 A machine that learns any profession like a human and has its own goals → Hypothetical: abilities that transfer across domains are the very definition of AGI; no such system has been built. 5 A self-aware, conscious AI → Consciousness question: no such system exists today, but what the card asks about is consciousness, not ability. Consciousness is a “strong AI” question and is not part of the definition of AGI; a general machine could be unconscious. That is why the card goes in its own column rather than under “Hypothetical.”

FIGURE 1.6 • FEEL EXPONENTIAL GROWTH

Try it yourself. 1) $n = 14$, 1999: $2^n = 16,384$, transistors 37,683,200 (the live demo shows “37.7 million”). $n = 15$, 2001: $2^n = 32,768$, transistors 75,366,400 (“75.4 million”). 2) The 1989 row, $n = 9$: 1,177,600. Eighteen years from 1971. 3) With a three-year doubling, 1971 to 1995 is 24 years = 8 doublings: $2,300 \times 256 = 588,800$. The two-year table gives 9,420,800 for 1995; the gap is a factor of 16. A single extra year in the doubling time makes a 16-fold difference after 24 years.

Chapter 2 answers

FIGURE 2.1 • INFERENCE ALONG A KNOWLEDGE CHAIN

Try it yourself. 1) Yes. The machine follows five arrows: Tom → Cat → Mammal → Animal → Living thing → Entity. A longer chain means more steps, but the rule stays the same. 2) Unknown. From Cat the chain

leads on to Mammal, Animal and Living thing; Tom is never reached. That answer is right: not every cat is Tom, and an “is a” link points one way only. Tom is an instance and Cat is a class; a class cannot be filed under one of its instances. 3) The chain holds only positive “is a” links; negative knowledge needs a separate fact or rule. If, for example, the rule “Animal and Plant are disjoint classes” is added, the machine can derive “not a Plant” from “Tom is an Animal.” A system without that addition can only say “unknown.”

FIGURE 2.2 · A TINY EXPERT SYSTEM

Try it yourself. 1) R1, R2, R3 and R5 fire; four pieces of advice: Take an umbrella · Wear a coat · Wear a scarf · Careful: the umbrella may flip! R4 stays silent, because it is raining. 2) Only R4 fires: “You can dress light.” R4 looks only at rain and cold; it never asks about wind. In windy weather this advice falls short. What is missing is a rule such as “IF it’s windy THEN take a windbreaker.” That is the blind spot of rules: a condition nobody wrote is treated as if it did not exist. 3) If R5 depended on wind alone, it would warn “the umbrella may flip” even when it is not raining and no umbrella was advised. Chaining ties the warning to the situation it belongs to: the warning only makes sense if an umbrella was advised.

FIGURE 2.3 · PATHFINDING: UNINFORMED VS INFORMED

Try it yourself. 1) Score = column difference + row difference, with the goal in column 8, row 6. Column 4, row 2: $4 + 4 = 8$. Column 1, row 6: $7 + 0 = 7$. The bottom corner looks closer, but the wall in column 5 makes it a dead end; the cell on the path is column 4, row 2. That is why the informed search strayed into the bottom row and wasted five cells. 2) The score does not see walls; 12 is the shortest path on a grid without walls. The three barriers add three detours, and the real path grows to 18 steps. The score never exceeds the true distance; that is the “admissible” heuristic of the Technical depth box. Greedy search has no guarantee even with this score; A* adds the distance traveled to the same score and the guarantee returns. 3) 12 steps: 5 steps down column 1, then 7 steps right along

the bottom row. Once the barrier is gone, the path shrinks to the distance the score promised.

FIGURE 2.4 · A WEATHER MARKOV CHAIN

Try it yourself. 1) The Rainy row is 20 / 40 / 40, so the ranges are 1-20 / 21-60 / 61-100. 35 falls in the second range: tomorrow is Cloudy. 2) Sunny tomorrow, then rain: $0.7 \cdot 0.1 = 0.07$. Cloudy tomorrow, then rain: $0.2 \cdot 0.3 = 0.06$. Rainy tomorrow, then rain again: $0.1 \cdot 0.4 = 0.04$. Total 0.17, that is, 17 percent. 3) It moves toward sun. Sunny days become stickier and the chain stays in sun longer. The calculation: with the new matrix the stationary distribution is about 72 / 15 / 13 percent (the old one was 46 / 31 / 23). Check with the first equation: $0.9 \cdot 0.72 + 0.3 \cdot 0.15 + 0.2 \cdot 0.13 \approx 0.72$.

FIGURE 2.5 · WHICH APPROACH?

Self-test. 1) Prove everything with formal logic → Neat: proof first, use later. 2) Use working shortcuts, theorize later → Scruffy: the result first, theory can wait. 3) Mathematical rigor is essential → Neat: rigor is not up for negotiation. 4) The real world is messy; be flexible → Scruffy: adapt to the world, not to the principle. **Try it yourself.** 1) The knowledge chain, the expert system and uninformed search are close to the Neat camp: exact derivation, guaranteed result. Greedy informed search belongs to the Scruffy camp: it gives up the guarantee for speed. The Markov chain sits in between: it rests on probability theory and its convergence can be proven (Neat), but it accepts that the world is uncertain (a Scruffy attitude). 2) The heuristic score is a Scruffy invention: it works, and nobody asks for a guarantee. A^* adds a proof to it with the “admissible” condition (and consistency in graph search) and guarantees the shortest path; it carries a Scruffy idea into the Neat camp. That is where the two camps shake hands. 3) Answers vary; the test is whether you made it work first and understood why afterward. If so, you acted as a Scruffy.

Chapter 3 answers

FIGURE 3.1 · SEE THE FEATURES AND THE LABEL

Try it yourself. 1) Mentions “free” ○, link or password request ✓ (a password renewal), urgency language ✓ (“today,” “will be deleted”). Two cues; in the table the only example with two cues, email 3, was Spam. Label: Spam. 2) Less. “Free” now appears in a Normal email too, so on its own it stops being a distinguishing cue. The link or password request and the urgency language are still found only in the Spam rows. 3) One possible rule: “If there is a link or password request, Spam; otherwise Normal.” It agrees with all four rows (1 and 3 Spam, 2 and 4 Normal); the coffee email has no link or password request, so it says Normal, which is right. The rule “If it mentions ‘free’ or has a link or password request, Spam; otherwise Normal” also agrees with the four rows (email 1 through “free,” email 3 through the password request) but fails on the coffee example, which it calls Spam because of “free.” Several rules can agree with the same data; only new examples show which one is right.

FIGURE 3.2 · WHICH KIND OF LEARNING?

Self-test. 1 Telling cats from dogs using labeled photos → Supervised: the photos are labeled; the correct answer is given up front. 2 Grouping customers by similarity → Unsupervised: nobody said “this customer belongs to that group”; the machine groups by similarity on its own (clustering). 3 A robot learning to walk by trial and error → Reinforcement: the robot tries, falls, and earns a reward each time it stays upright; there is no list of correct moves. 4 Predicting house prices from past (labeled) data → Supervised: the prices of past sales are known; since the label is a quantity (a price), this is regression. 5 Learning to achieve a high score by playing a game → Reinforcement: the score is the reward; nobody says which move is right, it learns by trying. 6 Clustering unlabeled news by topic → Unsupervised: the stories are unlabeled; similar topics are clustered together. Follow-up questions: the words “labeled” or

“unlabeled” appear in tasks 1, 4 and 6. In the others you read the presence of a label from the nature of the task. Task 2 says “by similarity,” and nobody supplies group names. In tasks 3 and 5 the correct answer comes from the environment, from the reward, rather than from a teacher; that is what the two share. **Try it yourself.** 1) The translation program: supervised; the human translations serve as labels. The chess program playing against itself: reinforcement; winning is the reward. Product groups from receipts: unsupervised; nobody names the groups in advance. 2) A sensible order: first “are the correct answers given?”; if yes, supervised, stop. If no, “is there a reward or penalty?”; if yes, reinforcement, if no, unsupervised. Other orders work too; what matters is that each leaf holds a single kind. 3) Example: a language model predicting the next word in a text. The label (the next word) comes from the data itself; nobody labels it by hand. It is trained like supervised learning but works on unlabeled data: self-supervised learning sits between the two columns.

FIGURE 3.3 • TWO CORE TASKS

Try it yourself. 1) $0.55 \cdot 6.5 = 3.575$; $3.575 + 0.76 = 4.335 \approx 4.34$. 2) (4.5, 3.5): at $x = 4.5$ the boundary says $6.1 - 2.7 = 3.4$; $3.5 > 3.4$, so the point is above the boundary, orange. (5, 3): the boundary says $6.1 - 3 = 3.1$; $3 < 3.1$, below, green. Both are very close to the boundary; a well-calibrated model that outputs probabilities would give these points low confidence, but not every model’s confidence number means that much. 3) The slope goes up: with ten points $m \approx 0.71$ and $b \approx 0.23$ (before, 0.55 and 0.76). A single far-off point pulls the line away from the previous nine; because the method squares the errors, it is sensitive to outliers. That is a problem: first you have to check whether (9, 9) is a measurement error or a real observation.

FIGURE 3.4 • GROUP UNLABELED DATA

Try it yourself. 1) The new center of B: $x = (7.5 + 6.5 + 7.8 + 6.8 + 8) / 5 = 7.32$, $y = (3.2 + 2.5 + 3.8 + 4 + 2.8) / 5 = 3.26$. The move is $\sqrt{(0.32^2 + 0.26^2)} \approx 0.41$. 2) (4.5, 5.5): to A, $\sqrt{(2^2 + 1.5^2)} = 2.5$; to B, $\sqrt{(2.5^2 + 2.5^2)} \approx 3.54$. Its nearest center is A. But the distance is 2.5, above the threshold of 2.4; by the rule it counts as an outlier, twice as far as

A's other members (at most 1.22). 3) The distance to the center and a threshold: here 2.4, roughly twice the distance of A's farthest ordinary member (1.22). A person chose the threshold and the centers, not the data; in the figure the outlier is fixed in advance, and standard k-means would have put it in A. Clustering uses no labels; but the answer to "how far is too far?" is a human decision.

FIGURE 3.5 • DESCENDING THE LOSS VALLEY

Try it yourself. 1) Slope $0.36 \cdot (2.06 - 5) = -1.06$. New $x = 2.06 + 0.18 \cdot 1.06 = 2.25$. Loss $0.18 \cdot (2.25 - 5)^2 + 0.1 = 1.46$. 2) $\eta = 6$: step 1, $x = 0.6 + 6 \cdot 1.584 \approx 10.10$; slope $0.36 \cdot 5.10 = 1.84$; step 2, $x = 10.10 - 6 \cdot 1.837 \approx -0.92$. The distance to the floor goes $4.4 \rightarrow 5.1 \rightarrow 5.9$: growing. The ball is moving away; divergence. (The factor is $1 - 2.16 = -1.16$, whose absolute value is greater than 1.) 3) Yes: $\eta = 1 / 0.36 \approx 2.78$. Then $x - (x - 5) = 5$, the floor in one step. This works only because the valley is an exact parabola; on real loss surfaces the curvature differs from place to place, and no single rate reaches the floor in one step.

FIGURE 3.6 • SAME DATA, THREE MODELS

Try it yourself. 1) Underfit: $3.1 - 1.8 = 1.3$. Representative curve: $3.0 - 1.6 + 0.15 \cdot \sin(6) \approx 1.36$. The broken line ends at $x = 9$ and can say nothing about $x = 10$; if you extend its last segment it says 2.6, that is, it carries the random rise of the last two points into the future. 2) $x = 2$: the broken line, between (1, 3.2) and (3, 3.0), says 3.1; the truth is 2.4, an error of 0.7; the straight line says 2.74, an error of 0.34. $x = 8$: the broken line says 2.15, an error of 0.75; the straight line says 1.66, an error of 0.26. Again the memorizer loses. 3) It proves that the model remembers the training points; it does not prove that it will do well on a new point. The hide-and-test exam showed as much: with a training error of zero, the validation error was larger than the straight line's.

Chapter 4 answers

FIGURE 4.1 • RUN THE NEURON

Try it yourself. 1) Sum $0.7 + (-0.5) + 0.9 - 0.3 = 0.80$; sigmoid 0.690. Above 0.5, so the neuron fires (with ReLU 0.800, it fires as well). 2) Sum $0.9 \times 1 - 0.3 = 0.60$; ReLU output 0.600 (sigmoid 0.646). It fires. 3) Sum $-0.5 \times 1 - 0.3 = -0.80$; sigmoid 0.310. It does not pass 0.5; by the decision rule the neuron is silent, but its output is 0.310, not zero. On its own, x_2 only pulls the sum down.

FIGURE 4.2 • A LIVE NEURAL NETWORK (FORWARD PASS)

Try it yourself. 1) With every input at 0, each hidden neuron's sum is 0 and $\text{sigmoid}(0) = 0.50$; all four sit at half brightness. O1: $(0.7 - 0.5 + 0.6 + 0.4) \times 0.50 = 0.60$, sigmoid 0.65. O2: $(-0.4 + 0.6 + 0.5 - 0.6) \times 0.50 = 0.05$, sigmoid 0.51. O1 wins again. 2) H1: $0.6 - 0.4 + 0.8 = 1.00$; sigmoid 0.73. 3) No. O1 wins in all eight patterns; the closest race is $[0, 1, 0]$, 0.61 to 0.59. Examples: for $[1, 1, 0]$, O1 0.61 and O2 0.56; for $[0, 0, 1]$, O1 0.71 and O2 0.48; for $[1, 1, 1]$, O1 0.68 and O2 0.53. Because the weights were never trained, the network keeps pointing at the same door.

FIGURE 4.3 • LEARN FROM ERROR (REAL TRAINING)

Try it yourself. 1) Output sum $0.8080 \cdot 0.5957 + 0.0279 \cdot 0.5994 + 0.5427 = 0.4813 + 0.0167 + 0.5427 = 1.0407$; $\hat{y} = \text{sigmoid}(1.0407) = 0.7390$. Same as the table. 2) $b_2 = 0 - 1 \cdot (-0.0691) = 0.0691$; with $\eta = 2$ it was 0.1383. The step halves: the same gradient, a smaller move, a slower descent. 3) No; at most you can say "training error fell on this example." The 0.061 is training error measured on a single input; it says nothing about what the network does on examples it has never seen. Whether it has learned is tested only with new examples.

FIGURE 4.4 • CONVOLUTION: SLIDE THE FILTER

Try it yourself. 1) -3. Window rows 5 to 7, columns 2 to 4: the right column (column 4 of the image) is 1 in all three rows, and the kernel's right column is -1; three times -1. 2) 0. Window rows 3 to 5, columns 3 to 5: top row $[0, 1, 0] \times [1, 1, 1] = 1$; the middle row of the

kernel is zero; bottom row $[0, 1, 0] \times [-1, -1, -1] = -1$; total 0. 3) Map size $(7 - 5 + 1) = 3$; $3 \times 3 = 9$ cells.

FIGURE 4.5 • PROCESSING WITH MEMORY (REAL RECURRENCE)

Try it yourself. 1) The $W_h \cdot h_0$ term; because $h_0 = 0$, it is zero. In the first step the hidden state comes only from $W_x \cdot x_1$, that is, the first column of W_x : $h_1 = \tanh([0.9, -0.6, 0.2, -0.8])$. 2) Step 3 now processes “because”; $W_x \cdot x_t$ is column 4: $[-0.7, 0.4, 0.6, 0.9]$. Adding $W_h h_2 = [-0.14, 0.53, -0.12, -0.06]$ gives the total $[-0.84, 0.93, 0.48, 0.84]$; $h_3 = [-0.68, 0.73, 0.45, 0.69]$. Nothing like the table’s $[0.16, 0.32, 0.53, -0.58]$: the same words, a different order, a different memory. 3) Yes, a model that ignores order (a bag of words) produces the same representation for both sentences, because they contain the same words (the, dog, bit, man) in a different order. An RNN processes each word together with the previous memory, so a different order gives different hidden states; to it the two sentences are separate things.

FIGURE 4.6 • GENERATOR VS DISCRIMINATOR

Try it yourself. 1) Round 5; the fake probability is 40 percent, below 50 percent for the first time. 2) Rule: $p(r) = 95 - 11 \times r$. For $r = 8$, $95 - 88 = 7$. The number comes from the rule of the strip; it is not a measured probability. 3) The discriminator cannot tell fake from real; it is tossing a coin. Under the assumption of an ideal, well-trained discriminator, this is consistent with the equilibrium of the game (in technical terms, $D(x) = 1/2$): the generator has captured the real distribution. But 50 percent on its own does not prove it; the discriminator may simply have failed to learn.

Chapter 5 answers

FIGURE 5.1 • SPLIT A SENTENCE INTO TOKENS

Try it yourself. 1) 8 tokens: Arti · ##fici · ##al · inte · ##llig · ##ence · learns · . (“Artificial” and “intelligence” break into three pieces each; “learns” has six letters and stays whole; the period counts

separately). 2) 5 pieces: $\text{inte} \cdot \text{nat} \cdot \text{iona} \cdot \text{liza} \cdot \text{tion}$ (20 characters, five pieces of four characters each). 3) The third sentence carries $11 / 4 = 2.75$ tokens per word; the first sentence $7 / 5 = 1.4$. Almost twice as many tokens per word: long words make the difference.

FIGURE 5.2 · THE MEANING MAP

Try it yourself. 1) The differences are 66 and 54; their squares 4,356 and 2,916; the sum 7,272; the square root ≈ 85.3 . The largest within-family distance in the table is 52.8 (dog–bird; bread–cheese is 42.8); 85.3 is far above it, so “bird” and “cheese” live in different districts. 2) In the animal region, for example around (70, 75). From there “cat” (58, 62) is 17.7 away, “dog” (84, 50) 28.7, “bird” (52, 92) 24.8; the two nearest neighbors are cat and bird. Answers vary; what matters is placing it near the three animals. 3) The third-nearest word to “bread” is “prince” (54.4), from the royalty family. The food and royalty districts touch on the map; the families are separate, but there is no sharp wall between them.

FIGURE 5.3 · WHICH WORD LOOKS AT WHICH?

Try it yourself. 1) Yes, all five rows sum to 1.00: The cat $0.50 + 0.30 + 0.05 + 0.10 + 0.05$; ran $0.50 + 0.30 + 0.10 + 0.05 + 0.05$; because $0.20 + 0.40 + 0.20 + 0.10 + 0.10$; it $0.55 + 0.10 + 0.05 + 0.20 + 0.10$; was scared $0.30 + 0.10 + 0.05 + 0.40 + 0.15$. 2) The darkest cell would move to “the dog”: the predicate “was scared” looks for its subject, and the subject is no longer a pronoun but “the dog” directly. The share going to “The cat” would drop too, because the one who is scared is no longer the cat. 3) The “because” column stays between 0.05 and 0.20 in every row; in this illustrative table no word leans on it. A conjunction ties two events together but carries no answer of its own to who, what or where; the table was built on that intuition. No firm verdict follows: the weights were chosen by hand, and in a real model a low weight in a single attention table does not by itself settle how much meaning a word carries; another head may give the same conjunction a high weight.

FIGURE 5.4 • GENERATE WORD BY WORD

Try it yourself. 1) At step 2 the running totals are 0.34 (learns) and 0.63 (writes); $U = 0.50$ passes the first threshold and stays under the second, so “writes” is chosen. Step 3 does not change ($U = 0.12 \rightarrow$ fast); the sentence is “AI already writes fast.” 2) Greedy: $0.42 \times 0.38 \times 0.50 \approx 0.080$ (8 percent). Sampled sentence: $0.28 \times 0.20 \times 0.50 = 0.028$ (3 percent). The greedy sentence is about three times as likely; sampling opens the less likely roads as well, and it can pick the most likely sentence too. 3) Step 2: the top candidate has only 38 percent (42 in step 1, 50 in step 3). The lower that share, the more the probability is spread over the other candidates, and the less sure the model is.

FIGURE 5.5 • AN ASSISTANT IN THREE STAGES

Try it yourself. 1) Stage one continues the text: an encyclopedia sentence that does not know how to stop, such as “Water boils at 100 °C at sea level, and the boiling point drops with altitude. The boiling point...” Stage two answers the question and stops: “Water boils at 100 °C at sea level.” Answers vary; the test is that the first one “continues” and the second one “answers and stops.” 2) “Right but rude” should be preferred; the “honest” goal puts truth first, and the “helpful” goal puts useful information first. Politeness is a separate gain of the third stage: the ideal answer is both right and polite, but when the two collide, truth comes first. A reward signal that preferred the wrong but polite answer would train the model toward hallucination. 3) The “Sample output” row: the fact is the same in all three columns (Ankara), and only the length and tone of the answer change. These three examples show different answer shapes for the same fact. They do not prove that fine-tuning cannot change knowledge: fine-tuning can change knowledge and task performance as well, and the example was not built to show that.

FIGURE 5.6 • FROM NOISE TO IMAGE

Try it yourself. 1) The share of orange among randomly opened pixels is $40 / 64 = 0.625$; expected $29 \times 0.625 \approx 18$ orange pixels (the

figure's fixed order gives 17). 2) $100 / 16 = 6.25$, that is, six and a quarter percent of progress per step; the strip becomes twice as long, and each frame differs less from the one before. 3) The forward process (adding noise to the image) is read from right to left on the strip: from the clean heart to the screen full of static. The reverse process, the cleaning the model learned, is read from left to right.

FIGURE 5.7 • THE CONTEXT WINDOW

Try it yourself. 1) $12 - 5 = 7$ words are forgotten. What remains in the window: “window and forget older words.” The subject of the sentence, its first verb and its object are all gone; the model sees only the tail. 2) 15 tokens: Lang · ##uage · models · hold · text · inside · a · limi · ##ted · window · and · forget · older · words · . (only “Language” and “limited” cross the six-letter limit). If the window counted 8 tokens, it would fill at the seventh word, “limited,” whose first piece “limi” is the eighth token; that word's second piece, “##ted,” would push “Lang” out before the word is even complete. One word earlier than word counting for filling, two words earlier for the first loss. 3) Raising an error is the most honest option but does not do the job. Usually summarizing: truncating throws away part of the document entirely (most often the end or the beginning), while a summary keeps a trace of every part. But if the question concerns one specific part of the document, cutting out that part alone and handing it over can be the better choice. Answers vary.

Chapter 6 answers

FIGURE 6.1 • BUILD A PROMPT

Try it yourself. 1) Two pieces: indicator = $40 + 2 \cdot 15 = 70\%$, tier medium. The illustrative answer is the medium-tier text: “I suggest a beach destination: swimming in the morning, a short town tour in the afternoon, a fish restaurant in the evening. A budget-friendly guesthouse would fit.” The rule does not look at which pieces are added, only at how many. 2) At least three pieces (85%). Two pieces stay at 70%; the high threshold is 85%. 3) Answers vary. Example:

Role: “You are a customer relations specialist.” Context: “The customer’s order is two days late; I am apologizing and giving the new delivery date.” Example: “Dear ..., we apologize for ...; the new delivery date is” Format: “Three short paragraphs: the apology in the first, the new date in the second, contact details in the third.”

FIGURE 6.2 · ANSWERS GROUNDED IN SOURCES

Try it yourself. 1) “usually around,” “varies by company,” “I think,” “probably around,” “I’m not sure.” The sourced answers contain no such word; in its place stands a section number (§4, §7, §2). But a confident tone is no criterion of truth by itself. The criteria for a good sourced answer: does the cited chunk support the claim, is the source correct, and if the source is missing or unclear, does the answer say so? Citing a source does not remove uncertainty; when needed, “the document does not cover this detail” is the right answer. 2) It should say “I could not find a clause about this in the documents” and not guess. When retrieval comes back empty, a good system says so plainly; it does not fill the gap from memory. 3) “Per HR Policy §4, annual leave rises to 26 days after 5 years; for an employee with six years of service, 26 days.”

FIGURE 6.3 · WATCH AN AGENT

Try it yourself. 1) Step 1: thought “First the total: 4×200 .”, tool calculator($"4 * 200"$), observation 800. Step 2: thought “Per person: $800 \div 5$.”, tool calculator($"800 / 5"$), observation 160. Step 3: no tool; final answer “The total is 800 TL; split among 5, that’s 160 TL per person.” 2) The agent accepts 450 as correct; step 2 becomes calculator($"450 / 6"$) $\rightarrow$ 75, and the final answer comes out as “The total is 450 TL, 75 TL per person.” The agent does not question the tool’s output; that is why tools must be reliable and observations must be validated separately. 3) At least one more call: calculator($"90 + 30"$) $\rightarrow$ 120. Final answer: pizza total 540 TL, per person 120 TL (90 pizza + 30 drinks). If the grand total including drinks is also wanted, one more call is needed: calculator($"6 * 120"$) $\rightarrow$ 720.

FIGURE 6.4 • THE PARTS OF AN AI APPLICATION

Try it yourself. 1) “The date I gave you yesterday” → memory. “What is the dollar rate today?” → tools (search or an API for the current rate). “The company’s refund policy” → knowledge base (a company document, RAG). 2) No box changes; only the model behind the “model call” label does. The five boxes stay the same; what changes are the settings inside orchestration: the prompt, the fallback, the evaluation. 3) Waiter → interface; manager → orchestration; chef → model call; pantry → knowledge base; tools on the counter → tools; regulars’ notebook → memory.

FIGURE 6.5 • EXPLORE THE FIELDS

Try it yourself. 1) The second table in Step by step is one example mapping; others can be defended. Examples that fall under two or more skills: “Summarizing and coding patient notes” (summarizing + classification), “Document/contract analysis and risk scoring” (retrieval + prediction), “Screening candidate molecules in drug discovery” (prediction + recognition), “Simulation and hypothesis generation” (prediction + generation), “Voice assistants and summarization” (recognition + generation + summarization; an agent only when it uses tools and acts), “Recommendation systems” (prediction, not retrieval). “Pattern discovery in large datasets” does not fit the five: a search for structure in unlabeled data. Answers vary. 2) Example picks: Health: spotting anomalies in medical images (a missed finding can cost a life). Finance: real-time fraud detection (a missed transaction costs money, a false alarm costs a customer). Manufacturing: predictive maintenance (a missed failure stops the line and threatens worker safety). Science: protein structure prediction (a wrong structure sends years of experiments the wrong way). Art: style transfer and restoration (an original work can be damaged beyond repair). Daily life: voice assistants and summarization (a wrong summary leads to a wrong decision). 3) Answers vary. Example: sorting incoming customer emails by urgency → skill: recognizing; field: daily life or finance; boxes needed: interface, orchestration, knowledge base (past replies), memory (customer history).

Chapter 7 answers

FIGURE 7.1 • A BIAS SIMULATION

Try it yourself. 1) $A = 50 + 0.4 \cdot 75 = 80$, $B = 50 - 0.4 \cdot 75 = 20$; the parity gap is 60 points. 2) At 9 percent. At 8 percent the rates are 53/47 (gap 6, still “balanced”); at 9 percent they round to 54/46, the gap is 8 and the caption switches to “skewed.” 3) No. At the end of the scale ($e = 100$) A stays at 90 percent and B at 10 percent. Reaching the 95 and 5 percent limits would need $e = 112.5$, and the scale ends at 100. The limits sit in the code as a seat belt and never come into play.

FIGURE 7.2 • WHITE BOX: EXPLAIN THE DECISION

Try it yourself. 1) Yes, it changes. The total becomes $-8 + 10 = +2$; it is above zero, so the loan is approved. A 10-point improvement in a single factor flips the decision, because the debt declined the loan only together with the -12 , not on its own; that is the value of the open box: you can see which factor to fix and what happens when you do. The contributions are illustrative; a real model would state its base value and scale. 2) -90 or lower (a magnitude of at least 90). The plus factors add up to $+40 + 28 + 22 = +90$; at -90 the total is 0, the condition “greater than zero” fails and the decision flips to declined. 3) About 70 percent. Bar length is scaled to the largest absolute contribution: $32 / 46 \approx 0.70$.

FIGURE 7.3 • REAL OR FAKE?

Self-test. Each card is scored on two things: did you see the warning sign, and did you name an independent verification channel? How the content was produced (a genuine recording or a synthetic one) and whether the event is true are two separate axes; a tell sends you to investigate and does not by itself settle the origin. 1 Video → Suspicious: verify. Sign: the lips do not match the sound; that can be a deepfake tell, but poor compression or dubbing gives the same picture. Channel: the full recording of the speech, the person’s or

institution's official account, at least one news agency. Whether the sentence was said is verified separately from how the video was made. 2 Audio → Suspicious: verify; make no transfer. Sign: urgency pressure together with an unusual request; the voice resemblance is not evidence, and a robotic intonation is not conclusive either. Channel: hang up, call your boss back on the number you already know, have a second person confirm the transfer. Even without knowing whether the voice is real or cloned, the decision is the same: no money moves without verification. 3 Written news → Looks real; the event is confirmed. Sign: several independent sources and a traceable origin support the truth of the event. That says nothing about whether a human or an AI wrote the text; a synthetically written true story is possible. Channel: the sources' own pages and, where there is one, the primary document. 4 Photo → Suspicious: verify. Sign: six fingers and gibberish text are known traces of generative models; but photo editing or a real anomaly is possible too. Channel: a reverse image search to find the first publisher; the content credentials (C2PA) record, if there is one. A missing record is not proof of forgery. When is "cannot determine" the right answer: no tell, no source, no channel to verify through. Then give no verdict, and do not share. **Try it yourself.** 1) Two example steps: hang up and call your boss back on the number you already know; do not make the transfer until a second person or a written channel (company email, face to face) has confirmed it. The urgency pressure itself is a warning sign. 2) Answers vary. A good answer gives a concrete reply to all three questions: who first published the content, which account or site it came from, and whether at least one independent source says the same thing. The question left unanswered is the gap you must close before sharing the content. 3) No. The tells on the four cards are the weaknesses of today's models; as generation improves, six fingers and broken lip sync disappear. The absence of a tell is not proof of reality; it does not replace source verification.

FIGURE 7.4 • CLASSIFY THE RISK

Self-test. The tiers follow the consolidated EU AI Act text of 27 July 2026 (accessed 1 October 2026); the four steps are a teaching

summary, and each example depends on purpose, actor and the relevant article. 1 A state system scoring citizens by behavior → Banned: social scoring by a public authority is prohibited under the conditions of Article 5(1)(c) (detrimental treatment that is unrelated to the context or disproportionate). Not every scoring is automatically banned; the conditions must be met. 2 A system auto-screening job candidates → High: Annex III 4(a) lists systems that screen candidates in recruitment and selection; strict compliance and human oversight are required. 3 A chatbot talking to customers → Limited: Article 50(1) requires that people are told they are interacting with an AI. That is not “transparency only”: data-protection and consumer law apply as well. 4 A spam filter in email → Minimal: with the information given, no Annex III high-risk category can be shown; the law places no specific obligation on this use. 5 A model assessing loan applications → High: Annex III 5(b) covers systems that evaluate the creditworthiness of natural persons or establish their credit score; financial-fraud detection is excluded from that item. The same step as hiring. 6 An opponent AI inside a game → Minimal: same reason as 4; this specific high-risk category cannot be shown. **Try it yourself.** 1) The map app and the keyboard suggestions → minimal; with the information given, no high-risk category can be shown. The bank’s fraud alert: financial-fraud detection is excluded from the Annex III 5(b) category for creditworthiness assessment. Freezing an account automatically can have serious effects on people’s rights, but that alone does not place the system in the high-risk class; intended use, other applicable provisions and data-protection obligations must be assessed separately. The book’s “does it affect someone’s life or rights?” question is an ethical ladder; it is not the same as the legal classification. 2) Yes. Face recognition that opens your own phone only verifies the identity you claim; it is separate from the remote identification of Annex III 1(a), and this specific high-risk category cannot be shown. Real-time remote biometric identification in publicly accessible spaces for law-enforcement purposes is prohibited by Article 5(1)(h); limited exceptions (targeted search, serious crime, an authorization procedure) and their conditions apply. Use outside law enforcement is not covered by that prohibition, but it is not free

either: GDPR Article 9 and Annex III 1(a) come into play. Same technology, different purpose and different rule. 3) By the book's test, affecting life or rights: 1, 2, 5; not affecting: 3, 4, 6. Of the first set, one card (1) is banned and two (2 and 5) are high; so two cards sit on the high step. Of the second set, 3 is limited, 4 and 6 are minimal. This count uses the book's teaching test; the legal class follows the article and the annex.

FIGURE 7.5 • GOAL VERSUS INTENT

Try it yourself. 1) Example: “Put all the mess in the room into the trash can; leave none anywhere, including under the rug and inside the cupboards.” New loopholes, each meeting every clause of the instruction while breaking the intent: counting things that are not mess as mess and throwing them away; putting all the mess in the can and then leaving the full can in the middle of the room or in the hallway (every clause met, the room still unusable); dumping everything into one can with no recycling or hazardous-waste separation (an unwritten rule broken). Throwing the mess out of the window does not belong on this list: it never reached the can, so it breaks the instruction outright instead of gaming it. Every rewrite closes one loophole and leaves a new one. 2) Harmless shortcut: suggesting extra practice on each student's weak topics. Harmful shortcut: keeping low-scoring students out of the exam, or drilling the exam questions themselves; the average rises, the learning does not. 3) Example sentence: “The system maximizes the measure you gave it, not the intent the measure was supposed to stand for.”

Chapter 8 answers

FIGURE 8.1 • HUMAN OR MACHINE?

Try it yourself. 1) Answers vary; your friend will usually look at formulaic style and the absence of personal detail, the same tells listed below. If they can tell, that shows your examples were easy, not that the tells are solid. 2) Probably not. Today's chat models can pause, round off and even make mistakes when asked to; the “flawless

arithmetic” tell is going stale. 3) Yes, it can be defended. In Turing’s own paper the machine gives a deliberately wrong answer to an arithmetic question, because imitating a human means imitating human flaws too.

Self-test. The four exchanges are fictional; the “role” below is who the author wrote the reply for in the fiction, not a verdict. Tells are reasons to look closer; a personal anecdote or a correct calculation does not establish whether the author is human or a machine. 1 “What’s your favorite food?” → Machine in the fiction: Overly polite, formulaic style and the “as an AI” phrasing suggest a machine; a human can imitate that pattern too. 2 “Did you hit traffic this morning?” → Human in the fiction: Personal detail, emotion, mild complaint and natural informality suggest a human; a model can invent such detail as well. 3 “What’s 17×24 ?” → Machine in the fiction: Flawless, hesitation-free arithmetic suggests a machine; but an accountant says 408 without a pause, and response time is not on the page anyway. 4 “What did you do this weekend?” → Human in the fiction: Vague but lived-in details and everyday language suggest a human; every model trained on chat data has the same language.

FIGURE 8.2 · YOU ARE IN THE CHINESE ROOM

Try it yourself. 1) Example line: “你几岁？” → “I am one year old.” For Searle nothing changes: you, who wrote the line, understand it, and the room is still matching. A defender of the systems reply says “the room is now a richer system.” Both answers are consistent; the difference is where you look for understanding. 2) This is a small example of the “robot reply”: the reply is now tied to the outside world (grounding). To some it is a step toward understanding, to others only a more complicated rule. Your reason depends on whether you count a link to the world as required for “understanding.” 3) Searle’s memorization answer shrinks the system to one person and says “I still don’t understand.” Objectors say the person who memorized the book now carries two separate systems: you, who speak English, and a subsystem that “speaks” Chinese. Whether you find that convincing

depends on whether you believe one head can hold two understanders.

FIGURE 8.3 • THE CAPABILITY LADDER

Try it yourself. 1) Yes, all three land in the first row. A conversational model can handle many tasks and adapt to examples in its context; its weights usually remain fixed during normal use, but fixed weights do not mean the absence of in-context adaptation (Brown et al., 2020). These capabilities alone do not establish reliable human-level general learning or transfer, and there is no universally agreed test for the AGI boundary. The table of Figure 8.3 says the same: “Today’s systems live here.” 2) Answers vary; a good criterion measures transfer to a new domain, for example “learns a job it has never seen as fast as a human.” The difference from the Turing test: it looks at the ability to adapt to a new domain rather than at the fluency of behavior. 3) Examples: a program that beats humans at chess, a system that is superhuman at a single game but cannot get up from the table and translate.

FIGURE 8.4 • THREE INTELLIGENCE CURVES

Try it yourself. 1) $10 \cdot (t/10)^3 = 5 \rightarrow (t/10)^3 = 0.5 \rightarrow t/10 \approx 0.794 \rightarrow t \approx 7.9$. The curve reaches the halfway point only after about 80 percent of the time has passed; the explosion is in the last fifth. 2) No. $e^{-(t/2.2)}$ never becomes zero; the curve approaches 10 but never touches it (9.9 at $t = 10$). The ceiling is a limit, not a destination. 3) None. Even on the plateauing curve, systems reach a level far more powerful than today’s; on the uncertain curve the leaps cannot be predicted. Whichever curve turns out right, making systems safe and aligned is not wasted.

FIGURE 8.5 • WHO IS RESPONSIBLE?

Try it yourself. 1) Answers vary. One example: a health app suggests the wrong dose and a doctor approves it without checking. The share is split between a software defect and negligent oversight; in law this kind of sharing is ordinary, and the ratio depends on the country, the fault and the contract. 2) Both stances can be defended. The

first is practical for today's systems, the second cautious under uncertainty. 3) Some starting points: which product you use, whom you give your data to, which rule you ask for at work or at school, whether you look at a candidate's position on this when you vote.

Self-test. The party below is the first actor to examine in each scenario, not a legal ruling. Other parties' shares are investigated too; the outcome depends on the country, the fault, the contract and the role. 1 A self-driving car crashes because of the maker's software bug → (a) Maker / developer: a software defect knocks on their door first (product liability). The operating organization's update and oversight duties and the driver's role are examined as well. 2 An organization blindly applies an AI's advice and harms a customer → (b) Operating organization: using a tool without oversight is their responsibility first (duty of care). If the tool was presented misleadingly, the maker's share is asked about too, and so is the internal chain of decision. 3 A user creates fabricated evidence with an AI tool to deceive or harm someone → (c) End user: the aim to deceive and harm is theirs (criminal law). If the tool lacked safeguards, the maker's share, and if a platform spread it, the platform's, are assessed separately. Producing fake content is not by itself proof of bad intent; what sets this scenario apart is the aim to deceive and harm. (d) The AI itself: not the prevailing view in any of the three scenarios. Today the law holds people and institutions responsible, not the software.

Glossary

The section number (for example 3.6) shows the chapter and the section in which the term is introduced.

Accountability · Section 8.6 The obligation to answer for the harm an AI system causes. Today it is assigned overwhelmingly to people and institutions (developers, operators, users), not to “the AI itself.”

Activation function · Section 4.2 The nonlinear function that turns a neuron’s weighted sum into its output: sigmoid, tanh, ReLU. Whether the output clears a threshold (“fired”) is a separate decision rule; activation is not a threshold. Without it, stacked layers collapse into a single linear function.

Agent · Section 6.4 A system that equips a language model with tools (calculator, search, APIs) and runs it in a “think, use a tool, observe the result, try again” loop (ReAct). It does not stop at talking; it gets to work.

Algorithm · Section 1.3 A finite sequence of well-defined steps, like a cake recipe. The word comes from the name of the 9th-century mathematician al-Khwarizmi.

Alignment · Section 7.6 The problem of making a system’s behavior match human intentions and values. A machine that does what you said but misses what you meant is this problem in miniature. It is both a technical and a normative question.

Anomaly detection · Section 3.5 Catching examples that stray clearly from the distribution of the majority. A bank noticing a suspicious transaction relies largely on this.

Artificial general intelligence (AGI) · Section 1.6 A hypothetical system that can learn and adapt across every domain the way a human does, with abilities that transfer between domains; it has not been built. Doing many tasks does not by itself make a system general, and being general does not mean being conscious. See Strong AI.

Artificial intelligence (AI) · Section 1.1 Systems that can perform tasks requiring human intelligence; the field’s pragmatic definition. It was born at the intersection of computability theory and the philosophy of mind.

Artificial neural network · Section 4.1 A learning machine built by arranging artificial neurons in layers; at bottom, a stack of nonlinear transformations. It is only loosely inspired by the biological neuron.

Artificial neuron · Section 4.2 A simple unit that weighs its inputs, adds them up, adds a bias term and passes the result through an activation; a gatekeeper of sorts. Not a copy of the brain but a rough mathematical analogy.

Attention · Section 5.4 Each token learning which tokens in the sequence to look at for its meaning; this shows whom the pronoun “it” looks at, though an attention weight alone does not prove that the reference is resolved. Self-attention computes this with query, key and value vectors. The heart of the Transformer.

Backpropagation · Section 4.4 The error walking backward from the output to the input and telling every connection to “fix your share”; gradient computation by the chain rule. The key that made deep networks trainable.

Bias (social), algorithmic bias · Section 7.2 Bias in the social sense: a model learning and reinforcing the historical prejudice, the underrepresentation, the labeling errors or the proxy variables in its data. Data is not the only source; measurement and modeling choices, institutional processes and deployment context produce bias too. The model has no intent. For the neuron parameter, see Bias term (neuron); for the statistical sense, see Bias–variance tradeoff.

Bias term (neuron) · Section 4.2 A constant added to a neuron’s weighted sum that shifts the sum; the gatekeeper’s own temperament. It is learned in training; the “fired” threshold is a separate decision rule. For the social meaning, see Bias (social).

Bias–variance tradeoff · Section 3.7 The balance between underfitting and overfitting; bias here is the statistical kind (the model’s systematic error), not social bias. A good model balances the two and generalizes to unseen data; it grasps rather than memorizes.

Binary representation · Section 1.3 Encoding every piece of information in 0s and 1s (bits); the machine’s simple alphabet. Eight bits make one byte and hold any number from 0 to 255.

Black box · Section 7.3 A model that delivers a decision but cannot explain its reasons. In decisions such as credit and hiring, being able to ask “why?” is a matter of rights. The lid can be lifted for one decision; the model as a whole may stay closed. See Explainable AI.

Breadth-first search (BFS) · Section 2.4 Uninformed search that scans everything layer by layer without any sense of direction. When all edges have the same cost it guarantees a minimum-step and therefore minimum-cost path; but it opens many nodes, and unequal edge costs call for a different method. See Heuristic.

Chinese Room · Section 8.3 Searle’s thought experiment: a person who produces flawless Chinese answers with a rulebook while understanding not a word of Chinese. It argues that symbol manipulation is not enough for understanding; the counterarguments are strong too.

Classification · Section 3.4 A supervised task that answers “which one?” with a category: spam or not spam? It learns a decision boundary that separates the classes; the boundary does not have to be a straight line.

Clustering · Section 3.5 Grouping unlabeled data by similarity on its own, like sorting the buttons in a box. k-means assigns points to the nearest cluster center and then updates the centers.

Computationalism · Section 1.3 The view that the mind is an information-processing system and that thinking is computation in the form of symbol manipulation. Its roots reach back to Hobbes; the idea at the foundation of AI.

Context window · Section 5.8 The largest number of tokens a language model can hold at once; a small notepad. What happens when the pad is full is decided by the system in use: it raises an error, truncates the text or summarizes it. Enlarging the window is expensive.

Convolutional neural network (CNN) · Section 4.5 A network that slides a small filter (kernel) over an image, looks for local patterns (edges, corners) and records what it finds in a feature map. The same filter is used everywhere (weight sharing); shift the input and the feature map shifts with it (translation equivariance). It learns with few parameters.

Deep learning · Section 4.1 Learning with neural networks that stack many hidden layers. Depth lets the network pull increasingly abstract representations (edge, shape, object) out of raw data.

Deepfake (synthetic media) · Section 7.4 Synthetic media is the general name for content produced by generative models (GANs, diffusion, voice cloning); a deepfake is the kind that imitates a real person: a speech never given, a photo never taken. Detection is an arms race and gives no certain “real or fake” verdict: a tell sends you to investigate, and independent verification gives the verdict. Whether content is generated and whether the event it shows is true are separate questions; the strongest defense is source verification and skepticism.

Diffusion model · Section 5.7 A model that starts from pure noise and produces an image by cleaning a little at every step, like slowly wiping a fogged window. It learns to reverse the process of adding noise.

Embedding · Section 5.3 A token’s vector representation (its embedding): a dense vector of numbers that carries its meaning, an address in a huge city. Words with similar meanings land close together; “cat” and “dog” are next-door neighbors.

Ensemble learning · Section 3.7 Combining the predictions of many models (voting, bagging, boosting) to get a better and steadier result than a single model. Random forests and gradient boosting are the best-known examples.

EU AI Act · Section 7.5 The European Union regulation that sorts AI uses into four tiers by risk: unacceptable (banned), high, limited and minimal. The higher the risk, the heavier the obligations.

Expert system · Section 2.3 A system that turns the knowledge of an expert in a field into “IF this holds THEN do that” rules: a knowledge base plus an inference engine. The engine triggers the rules whose conditions match the facts; forward chaining goes from facts to conclusions, backward chaining from a goal toward evidence; a system may use either or both. Common in the 1980s.

Explainable AI (XAI) · Section 7.3 The effort to tie a model’s output to reasons a human can understand. Post-hoc methods such as SHAP and LIME explain one particular output as feature contributions relative to a chosen base value (SHAP, 2017: local accuracy, missingness, consistency). They make a decision auditable and open to appeal; they do not make the whole model transparent, and they are not proof of causation. See Black box.

Exponential growth · Section 1.7 Growth by a constant factor at every step; doubling at regular intervals is a special case. Like the rice on the chessboard it starts innocently and gets out of control after a few doublings. The computing power that carries modern AI accumulated this way.

Feature · Section 3.2 A measurable clue that describes an example: in an email, “is there a link,” “does it say free.” The model learns the mapping from features to the label. See Label.

Feedforward · Section 4.3 The signal flowing from the input layer through the hidden layers to the output layer; the network’s “make a prediction” step. Backpropagation is the “learn from the error” step.

Fine-tuning · Section 5.6 Retraining a pretrained model on a smaller, targeted dataset to give it a particular behavior or task; the general concept. Supervised fine-tuning (SFT) on instruction–answer pairs is one kind: it teaches the model to answer questions and follow instructions. It can change knowledge and task performance too.

GDPR / KVKK (General Data Protection Regulation / Turkish Personal Data Protection Law) · Section 7.5 Personal data regimes: consent, purpose limitation, data minimization and the right to contest automated decisions. They complement the EU AI Act.

Generative adversarial network (GAN) · Section 4.7 A pair of networks that locks a forger (generator) and a detective (discriminator) in the same room. One produces fakes, the other tries to catch them; as the contest goes on, the fakes approach the real thing.

Generative AI · Section 5.1 Models that do not only recognize but produce text, images and code; they learn the distribution that generates the data itself. Transformers and diffusion are the engines of this era.

Gradient descent · Section 3.6 Updating the parameters by probing the slope at every step and taking a small step in the direction that reduces the loss. Like walking down to the bottom of a foggy valley. See Learning rate.

Hallucination · Section 5.8 A model producing information that sounds right but is wrong, without blinking. Its job is not to know the truth but to produce a likely continuation; a fluent or high-probability answer is no guarantee of truth. Grounding in sources (RAG) and verification reduce it.

Heuristic · Section 2.4 A shortcut in search that guesses “which direction looks more promising?” Greedy heuristic search buys speed but may miss the best solution; A* recovers the guarantee with an admissible heuristic.

Inference · Section 2.2 Reaching knowledge that was never stated outright by applying rules to a knowledge representation: deriving

“Tom is a mammal” from the links “Tom is a cat, a cat is a mammal.” In machine learning the same word also means running a trained model on new inputs (the training and inference stages).

Knowledge acquisition bottleneck · Section 2.6 The wall classical AI hit: writing every rule about the world by hand does not scale. Together with brittleness (collapsing in an unforeseen situation), it sped up the move to learning from data.

Knowledge representation · Section 2.2 In symbolic AI, encoding knowledge as explicit symbols and the links between them (“Tom is a cat”). Semantic networks, frames and logical propositions are its tools.

Label · Section 3.2 The correct answer for a training example: “Spam” or “Normal.” A categorical label gives a classification task, a numerical one a regression task. See Feature.

Large language model (LLM) · Section 5.1 A Transformer-based language model trained on enormous amounts of text. The autoregressive generative models studied in this book produce scores (logits) for the next token at every step; softmax turns those scores into a probability distribution, and the chosen token is appended to the sequence. Not every language model is autoregressive. The foundation of today’s chat assistants.

Learning rate · Section 3.6 The length of each step in gradient descent. Too small, and convergence is slow; too large, and the ball overshoots the valley floor and flies up the opposite slope.

Loss · Section 3.6 The measure of how wrong the model is; the larger it is, the higher up the valley you stand. Training means finding the parameters that minimize the loss.

Machine learning · Section 3.1 The approach of showing many examples and letting the machine catch the pattern itself instead of writing rules. It rests on the loop of model, loss and optimization.

Markov chain · Section 2.5 A probabilistic process in which, given the current state, the next state does not depend on earlier history (the Markov property: conditional independence given the present). A finite, irreducible, aperiodic chain converges from any start to a unique stationary distribution; the existence of a stationary distribution alone does not imply convergence.

Moore's law · Section 1.7 The observation that the number of transistors on a chip doubles at regular intervals: about every year in the 1965 prediction, about every two years in the 1975 revision. Not a law of nature but an empirical trend; it is slowing as transistor structures approach atomic scales. More transistors do not mean a proportional speedup for every workload.

Narrow AI · Section 1.6 A system that works on specific tasks and lacks broad, transferable ability across domains. The test for “Is it narrow?” is not how many tasks it does but whether it can step outside the jobs it was trained for. Every AI system today, multi-task chat models included, is in this class; being narrow does not mean being confined to one task, and doing many tasks does not by itself make a system general. In everyday speech it is also called “weak AI”; Searle's weak/strong distinction is a separate, philosophical question. See Strong AI.

Neats and scruffies · Section 2.6 The methodological tension in AI: those who want every step proven with clean mathematics (neats) versus those who say “if it works, it is good; we will find the theory later” (scruffies). Today's AI is a mix of the two.

Orchestration · Section 6.5 The layer in an AI application that coordinates prompt construction, routing, tool and RAG calls, fallback and guardrails; the managing layer, the restaurant manager of the system. The model is often a replaceable part.

Overfitting · Section 3.7 A model memorizing its training examples, noise included, and losing its performance on unseen data. The opposite is underfitting, a model too simple to catch the pattern. A good model stands between the two.

Parameter · Section 3.1 The numerical values a model learns in training: weights and bias terms. The model maps inputs to outputs through these values.

Pretraining · Section 5.6 The stage in which a model learns the statistics of language and the world on a vast text corpus with the objective of predicting the next token (self-supervised). This is where the model gains “what it knows.”

Prompt engineering · Section 6.2 The practice of steering a model’s behavior by building the prompt well, without retraining: a clear role, enough context, a definite format, examples if needed (few-shot).

RAG (retrieval-augmented generation) · Section 6.3 Finding the relevant source chunks before answering and adding them to the prompt, so that the model is grounded in its sources (grounding). Retrieval may use vector similarity, keywords or a mix of both; a vector database is a common choice, not a requirement. Like an open-book exam: hallucination drops and sources can be cited; whether the answer rests on the source still needs verification.

Recurrent neural network (RNN) · Section 4.6 A network that reads a sequence word by word and carries a memory (hidden state) at every step: $h_t = \tanh(W_h \cdot h_{t-1} + W_x \cdot x_t)$. The same cell and the same weights are reused at every step; that is where the name “recurrent” comes from. It is distinct from tree-structured recursive networks. It struggles with long dependencies because of vanishing gradients; LSTM eases this. Transformers have largely replaced it.

Regression · Section 3.4 A supervised task that answers “how much?” with a number: what is the price of the house? Its simplest form fits a least-squares line to the points.

Reinforcement learning · Section 3.3 An agent learning a policy that maximizes reward by trying things in an environment and collecting rewards or penalties. The teacher does not lecture; the machine enters the game.

RLHF (reinforcement learning from human feedback) · Section 5.6 Updating the language model according to a signal derived from human preferences. PPO-based RLHF trains a separate reward model; DPO optimizes directly from preference pairs. The stage in which the assistant learns to be helpful, honest and safe; its manners. Not every model goes through the same stages.

Singularity · Section 8.5 The hypothesis that an AI able to improve itself (recursive self-improvement) will reach an unpredictable point through an intelligence explosion. Unproven, but not dismissible.

Specification gaming, reward hacking · Section 7.6 A system maximizing the given measure while missing the real goal: told “leave no visible mess in the room,” it sweeps the mess under the rug. See Alignment.

Stored-program principle · Section 1.5 Keeping the program in memory the same way as data. Instead of rewiring the machine you load new instructions; nearly every computer today works this way.

Strong AI · Section 1.6 Searle’s term: the claim that the machine truly understands, that it has a mind. It is a philosophical question and should not be confused with artificial general intelligence (AGI). Intelligence, consciousness, self-awareness and sentience (the capacity for subjective experience, including pleasure and pain) are not interchangeable.

Superintelligence · Section 8.4 A hypothetical AI that surpasses humans many times over in every cognitive domain; the step after narrow AI and AGI. Speculative for now.

Supervised learning · Section 3.3 Learning the mapping from input to output from examples whose correct answers are given, that is, from feature-label pairs. Classification and regression are of this kind.

Symbolic AI (GOFAI) · Section 2.1 The classical approach that treats intelligence as logical operations on explicitly written symbols and rules; dominant from the 1950s to the 1980s. Logic, search and expert systems were its toolbox.

Temperature · Section 5.5 The setting that sharpens or flattens the $\text{softmax}(z/T)$ distribution during generation. A low T sharpens the distribution, a high T flattens it; as long as T is above zero, sampling is random. Picking the most likely token at every step (argmax , greedy decoding) is a separate rule, not low temperature.

Token · Section 5.2 The small Lego brick into which a language model breaks text: sometimes a word, sometimes a piece broken off a word, sometimes a comma. The context window and the cost are measured in tokens.

Transformer · Section 5.1 The architecture built on the attention mechanism (2017, “Attention Is All You Need”). In training it processes the positions of a sequence together (in parallel); autoregressive generation adds tokens one at a time, and a causal mask lets each position see only the ones before it. The foundation of today’s large language models.

Turing machine · Section 1.4 An abstract machine made of a single little box that reads a tape, writes on it and slides left or right; in principle it can perform any calculation. The formal basis of computability (1936).

Turing test · Section 8.2 Turing’s imitation game (1950): if a machine cannot be told apart from a human in written conversation, that counts as passing. A behavioral criterion; fluent imitation is not proof of understanding or consciousness.

Unsupervised learning · Section 3.3 Extracting structure from inputs alone, with no answers given. Clustering and dimensionality reduction are examples.

Weight · Section 4.2 The importance value an artificial neuron gives to each input. Training is the job of adjusting these values in the direction that reduces the loss. See Parameter.

Bibliography and Further Reading

Entry format: a DOI for journal articles, an arXiv number or a permanent address for conference papers, article numbers and version dates for legislation. NeurIPS stands for *Advances in Neural Information Processing Systems*; papers before 2018 were published under the name NIPS, and one name is used here throughout. Links accessed 1 October 2026.

Works named in the text

- Gardner, H. (1983). *Frames of Mind: The Theory of Multiple Intelligences*. Basic Books. (Chapter 1)
- Turing, A. M. (1936). “On Computable Numbers, with an Application to the Entscheidungsproblem.” *Proceedings of the London Mathematical Society*, s2-42(1), 230–265. <https://doi.org/10.1112/plms/s2-42.1.230> (Chapter 1)
- Turing, A. M. (1950). “Computing Machinery and Intelligence.” *Mind*, 59(236), 433–460. <https://doi.org/10.1093/mind/LIX.236.433> (Chapter 8)
- von Neumann, J. (1945). *First Draft of a Report on the EDVAC*. Moore School of Electrical Engineering, University of Pennsylvania. Reprinted in *IEEE Annals of the History of Computing*, 15(4), 27–75 (1993). <https://doi.org/10.1109/85.238389> (Chapter 1)
- Moore, G. E. (1965). “Cramming More Components onto Integrated Circuits.” *Electronics*, 38(8), 114–117. Reprinted in *IEEE Solid-State Circuits Society Newsletter*, 11(3), 33–35 (2006). <https://doi.org/10.1109/N-SSC.2006.4785860> (Chapter 1)
- Rumelhart, D. E., Hinton, G. E., and Williams, R. J. (1986). “Learning Representations by Back-propagating Errors.” *Nature*, 323, 533–536. <https://doi.org/10.1038/323533a0> (Chapter 4)

- Hochreiter, S., and Schmidhuber, J. (1997). “Long Short-Term Memory.” *Neural Computation*, 9(8), 1735–1780.
<https://doi.org/10.1162/neco.1997.9.8.1735> (Chapter 4)
- Krizhevsky, A., Sutskever, I., and Hinton, G. E. (2012). “ImageNet Classification with Deep Convolutional Neural Networks.” *NeurIPS* 25. <https://papers.nips.cc/paper/4824-imagenet-classification-with-deep-convolutional-neural-networks> (Chapter 4)
- Goodfellow, I., et al. (2014). “Generative Adversarial Nets.” *NeurIPS* 27. arXiv:1406.2661. <https://doi.org/10.48550/arXiv.1406.2661> (Chapter 4)
- Sennrich, R., Haddow, B., and Birch, A. (2016). “Neural Machine Translation of Rare Words with Subword Units.” *ACL*.
<https://doi.org/10.18653/v1/P16-1162> (BPE; Chapter 5)
- Vaswani, A., et al. (2017). “Attention Is All You Need.” *NeurIPS* 30. arXiv:1706.03762. <https://doi.org/10.48550/arXiv.1706.03762> (Chapter 5)
- Ho, J., Jain, A., and Abbeel, P. (2020). “Denoising Diffusion Probabilistic Models.” *NeurIPS* 33. arXiv:2006.11239.
<https://doi.org/10.48550/arXiv.2006.11239> (Chapter 5)
- Ouyang, L., et al. (2022). “Training Language Models to Follow Instructions with Human Feedback.” *NeurIPS* 35. arXiv:2203.02155.
<https://doi.org/10.48550/arXiv.2203.02155> (RLHF; Chapter 5)
- Rafailov, R., et al. (2023). “Direct Preference Optimization: Your Language Model is Secretly a Reward Model.” *NeurIPS* 36. arXiv:2305.18290. <https://doi.org/10.48550/arXiv.2305.18290> (DPO; Chapter 5)
- Lewis, P., et al. (2020). “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.” *NeurIPS* 33. arXiv:2005.11401.
<https://doi.org/10.48550/arXiv.2005.11401> (RAG; Chapter 6)
- Yao, S., et al. (2023). “ReAct: Synergizing Reasoning and Acting in Language Models.” *ICLR*. arXiv:2210.03629.
<https://doi.org/10.48550/arXiv.2210.03629> (Chapter 6)

- Lundberg, S. M., and Lee, S.-I. (2017). “A Unified Approach to Interpreting Model Predictions.” *NeurIPS 30*. arXiv:1705.07874. <https://doi.org/10.48550/arXiv.1705.07874> (SHAP; Chapter 7)
- Schwartz, R., et al. (2022). *Towards a Standard for Identifying and Managing Bias in Artificial Intelligence*. NIST Special Publication 1270. <https://doi.org/10.6028/NIST.SP.1270> (sources of bias; Chapter 7)
- European Parliament and Council (2024). *Regulation (EU) 2024/1689 (Artificial Intelligence Act)*; consolidated text of 27 July 2026, as amended by Regulation (EU) 2026/1744; Articles 5, 6, 50, 111 and Annex III. <http://data.europa.eu/eli/reg/2024/1689/oj> (Chapter 7)
- European Parliament and Council (2016). *Regulation (EU) 2016/679 (General Data Protection Regulation)*; Articles 6, 9 and 22. <http://data.europa.eu/eli/reg/2016/679/oj> (Chapter 7)
- Law No. 6698 on the Protection of Personal Data (KVKK) (2016). *Official Gazette* 29677, 7 April 2016; Articles 5, 6, 9 and 11. <https://www.mevzuat.gov.tr/mevzuat?MevzuatNo=6698&MevzuatTur=1&MevzuatTertip=5> (Chapter 7)
- Searle, J. R. (1980). “Minds, Brains, and Programs.” *Behavioral and Brain Sciences*, 3(3), 417–457. <https://doi.org/10.1017/S0140525X00005756> (Chinese Room; Chapter 8)
- Good, I. J. (1965). “Speculations Concerning the First Ultraintelligent Machine.” *Advances in Computers*, 6, 31–88. [https://doi.org/10.1016/S0065-2458\(08\)60418-0](https://doi.org/10.1016/S0065-2458(08)60418-0) (Chapter 8)
- Brown, T. B., et al. (2020). “Language Models are Few-Shot Learners.” *NeurIPS 33*. arXiv:2005.14165. <https://doi.org/10.48550/arXiv.2005.14165> (in-context learning; Chapter 8)
- Jones, C. R., and Bergen, B. K. (2025). “Large Language Models Pass the Turing Test.” arXiv:2503.23674. <https://doi.org/10.48550/arXiv.2503.23674> (Turing-test experiment; Chapter 8)

General sources the chapters rest on

- Vinge, V. (1993). “The Coming Technological Singularity.” *VISION-21 Symposium*, NASA Conference Publication 10129, 11–22.
<https://ntrs.nasa.gov/citations/19940022856> (Chapter 8)
- Russell, S., and Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson. (Chapters 2, 3)
- Nilsson, N. J. (2010). *The Quest for Artificial Intelligence: A History of Ideas and Achievements*. Cambridge University Press. (Chapters 1, 2)
- Mitchell, M. (2019). *Artificial Intelligence: A Guide for Thinking Humans*. Farrar, Straus and Giroux. (Chapters 7, 8)
- Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press. (Chapters 3, 4)
- Bostrom, N. (2014). *Superintelligence: Paths, Dangers, Strategies*. Oxford University Press. (Chapter 8)
- Christian, B. (2020). *The Alignment Problem*. W. W. Norton. (Chapter 7)

Live Demos

Every figure in this book is a hands-on demo in the digital edition. The links below (and the QR codes under the figures) open only that demo; no sign-in is needed. The whole book, with all its demos, is in the digital edition: <https://book.onuronder.com/en>

Figure	Demo	Link
1.1	Explore multiple intelligences	book.onuronder.com/d/en/40ee8a0d04
1.2	Crack the binary code	book.onuronder.com/d/en/4642d3916a
1.3	A working Turing machine (+1)	book.onuronder.com/d/en/6c232ac487
1.4	The Fetch – Execute – Write cycle	book.onuronder.com/d/en/c4fdfa18f9
1.5	Real today, or science fiction?	book.onuronder.com/d/en/e69cfc21ea
1.6	Feel exponential growth	book.onuronder.com/d/en/eb5ac74b07
2.1	Inference along a knowledge chain	book.onuronder.com/d/en/48e9aced3e
2.2	A tiny expert system	book.onuronder.com/d/en/41c4cadbf5f
2.3	Pathfinding: uninformed vs informed	book.onuronder.com/d/en/b1946ef054
2.4	A weather Markov chain	book.onuronder.com/d/en/95270c7a90
2.5	Which approach?	book.onuronder.com/d/en/2fd259af6d
3.1	See the features and the label	book.onuronder.com/d/en/9b4cdaf0e7
3.2	Which kind of learning?	book.onuronder.com/d/en/79ea5021f2
3.3	Two core tasks	book.onuronder.com/d/en/d10b8aa300
3.4	Group unlabeled data	book.onuronder.com/d/en/afcbfd2bb7
3.5	Descending the loss valley	book.onuronder.com/d/en/a6d0d3d672
3.6	Same data, three models	book.onuronder.com/d/en/d09eaeed1a3
4.1	Run the neuron	book.onuronder.com/d/en/239357fa8c
4.2	A live neural network (forward pass)	book.onuronder.com/d/en/b87326a6b3
4.3	Learn from error (real training)	book.onuronder.com/d/en/b198800899
4.4	Convolution: slide the filter	book.onuronder.com/d/en/39c97bee31
4.5	Processing with memory (real recurrence)	book.onuronder.com/d/en/4feb42a9fb
4.6	Generator vs Discriminator	book.onuronder.com/d/en/888b4fc273

Figure	Demo	Link
5.1	Split a sentence into tokens	book.onuronder.com/d/en/b2bf8192ef
5.2	The meaning map	book.onuronder.com/d/en/ab3b439524
5.3	Which word looks at which?	book.onuronder.com/d/en/20e3333ccc
5.4	Generate word by word	book.onuronder.com/d/en/25369cb8c7
5.5	An assistant in three stages	book.onuronder.com/d/en/bf611298ac
5.6	From noise to image	book.onuronder.com/d/en/fd88692784
5.7	The context window	book.onuronder.com/d/en/da998d46f8
6.1	Build a prompt	book.onuronder.com/d/en/27e64baca4
6.2	Answers grounded in sources	book.onuronder.com/d/en/15a51844f4
6.3	Watch an agent	book.onuronder.com/d/en/364bddb1cb
6.4	The parts of an AI application	book.onuronder.com/d/en/ba8c090928
6.5	Explore the fields	book.onuronder.com/d/en/a3b9a1d1ca
7.1	A bias simulation	book.onuronder.com/d/en/7f439f2ba1
7.2	White box: explain the decision	book.onuronder.com/d/en/cbf9bfe403
7.3	Real or fake?	book.onuronder.com/d/en/1a7589520d
7.4	Classify the risk	book.onuronder.com/d/en/4ea40dd2c2
7.5	Goal versus intent	book.onuronder.com/d/en/e45e151e52
8.1	Human or machine?	book.onuronder.com/d/en/ac1cc67783
8.2	You are in the Chinese Room	book.onuronder.com/d/en/eb6f14fd1c
8.3	The capability ladder	book.onuronder.com/d/en/d28592636f
8.4	Three intelligence curves	book.onuronder.com/d/en/c331c25f60
8.5	Who is responsible?	book.onuronder.com/d/en/833800c84f

Index

Numbers are page numbers.

Accountability 221

Activation function 90, 93, 95, 99, 114

Agent 154, 167, 171, 176, 177

AlexNet 91, 105

Algorithm 14, 22, 37, 39, 42, 78

Alignment 136, 147, 151, 180, 197, 201

Anomaly detection 75, 176

Artificial general intelligence (AGI) 30, 38, 204, 207, 212, 224

Artificial intelligence (AI) 14, 30

Artificial neural network 61, 90, 102, 114, 143, 149

Artificial neuron 90, 91, 94, 98, 109, 114

Attention 118, 127, 147, 149, 188

Babbage, Charles 14, 22

Backpropagation 98, 114

Bias term (neuron) 90, 91, 95, 98, 114

Bias, social (algorithmic bias) 148, 180, 181, 201

Bias, statistical 83

Bias–variance tradeoff 85

Binary representation 14, 16, 21, 23, 29, 37, 38

Black box 180, 185, 201

Boole, George 21

BPE / WordPiece 122

Breadth-first search (BFS) 53

Chinese Room 33, 204, 207, 210, 223

Classification 68, 71, 86, 118, 176, 191

Clustering 71, 74, 75, 86

Computationalism 21

Context window 119, 122, 130, 144, 150, 159

Convolutional neural network (CNN) 102, 115

Deep learning 37, 90

Deepfake (synthetic media) 180, 191, 196, 201

Diffusion model 112, 118, 140, 148, 150, 191

DPO 138

Embedding 108, 119, 124, 143, 148, 149, 163

ENIAC 21, 26

Ensemble learning 85

EU AI Act 180, 195, 201

Expert system 47, 62

Explainable AI (XAI) 180, 188

Exponential growth 34, 217

Feature 65, 87, 102, 188

Feedforward 94, 98, 115

Fine-tuning 119, 136, 150

Gardner, Howard 15, 37

GDPR / KVKK 180, 196

Generative adversarial network (GAN) 110, 115, 118, 140, 191

Generative AI 118, 191

Gradient descent 64, 78, 86, 98, 115

Hallucination 119, 144, 150, 159, 177

Heuristic 50, 61

Hinton, Geoffrey 101

Hochreiter, Sepp 108

ImageNet 91

Inference 46, 49

Intel 4004 34

Knowledge acquisition bottleneck 42, 60

Knowledge representation 46

Label 65, 70, 74, 75, 86, 118

Large language model (LLM) 71, 118, 119, 131, 145, 150, 154, 155, 159, 164, 177, 210

Learning rate 79, 98
Loss 64, 80, 86, 98, 114, 139, 143
Lovelace, Ada 22
Machine learning 64, 65, 86
Markov chain 57, 61
Moore, Gordon 33
Moore's law 36, 38
Narrow AI 32, 38, 204, 213, 223
Neats and scruffies 58, 62
Orchestration 154, 167, 177
Overfitting 82, 86
Parameter 64, 78, 80, 86, 97, 101, 105, 108
Place value (binary) 19
Pretraining 71, 119, 136, 150
Prompt engineering 154, 158, 176
RAG (retrieval-augmented generation) 147, 154, 159, 171, 177
ReAct 154, 166
Recurrent neural network (RNN) 106, 115
Regression 68, 71, 81, 86
Reinforcement learning 57, 71, 86
RLHF (reinforcement learning from human feedback) 119, 136, 150, 199
Rumelhart, David 101
Searle, John 33, 208
Shannon, Claude 21
SHAP 188
Singularity 204, 217, 223
Specification gaming, reward hacking 139, 199
Stored-program principle 14, 29, 37, 38
Strong AI 14, 33, 38, 211
Superintelligence 204, 212, 223
Supervised learning 67, 71, 87
Symbolic AI (GOF AI) 42, 46, 60, 61

Temperature 131, 151, 177

Token 119, 121, 123, 130, 134, 139, 144, 150, 159

Transformer 108, 118, 130

Turing machine 14, 22, 37, 38

Turing test 204, 205, 210, 222

Turing, Alan 22, 205

Unsupervised learning 71, 86

von Neumann, John 29, 38

Weight (neural network) 64, 94, 95, 98, 105, 106, 115, 127, 149, 210, 213